

UPV – Inteligencia Artificial & Big Data Analytics

Herramientas de última generación para BigData

Franzi Kröger y Carlos Ferrer

Franzi Kröger

- **Business Chemistry** from the Heinrich Heine University of Düsseldorf & **Master on Big Data Analytics** from EDEM Escuela de Empresarios de València
- From optimizing processes to BI insights to coding data pipelines – engineering my path in tech
- Working in GFT as a **Data Engineer** and **Python Backend Developer**
- Part of **Data Offering Development** team, covering “Big Data”, “IoT”, and “Data in Cloud” technologies



[franziska-sophie-kroeger](https://www.linkedin.com/in/franziska-sophie-kroeger)



franzi.kroeger@gft.com

Carlos Ferrer

- **International Business and Marketing Management** from ESIC ESIC Business & Marketing School & **Master on Big Data Analytics** from EDEM Escuela de Empresarios de València

✦ Experience in **Business Management** and **Finance**

✦ Working in GFT as a **Data Engineer** and **Cloud Engineer**

✦ Part of **Data and AI Offering Development** team, covering “**Big Data**”, “**AI**”, and “**Data in Cloud**” technologies



[Carlos Ferrer](#)



carlos.ferrer@gft.com

GFT at a glance



ABOUT GFT

GFT is a digital transformation pioneer. By leveraging next-generation technologies, we enable clients to boost their productivity with intelligent software solutions. We focus on Digital Finance, Enterprise AI & Data Solutions, and Platform Modernisation.

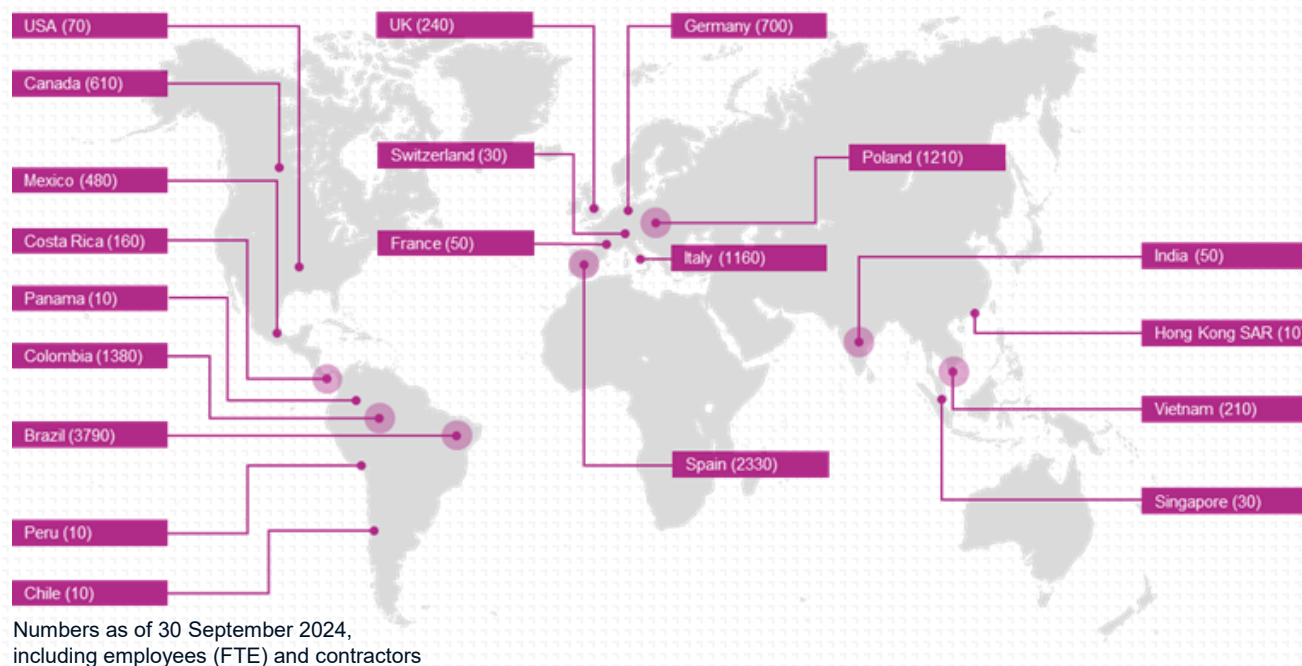
OUR END-TO-END SERVICE PORTFOLIO

- › NextGen Finance
- › Enterprise AI & Data
- › Platform Modernisation
- › Engineering Services & Regulatory

OUR PRODUCTS

- Project Portfolio Management Software “Engenion”
- › Compliance Suite “Smaragd”
- ›

GLOBAL PRESENCE



REVENUE FY 2023

€789m

GLOBAL TEAM OF

12,000+ talents

Trusted
by
market
leaders



Partnering
with
the best
└

Modern platforms



Google Cloud



Microsoft Azure



FINASTRA



70x

Independent software vendors



Backbase



RuleX



SCALED AGILE



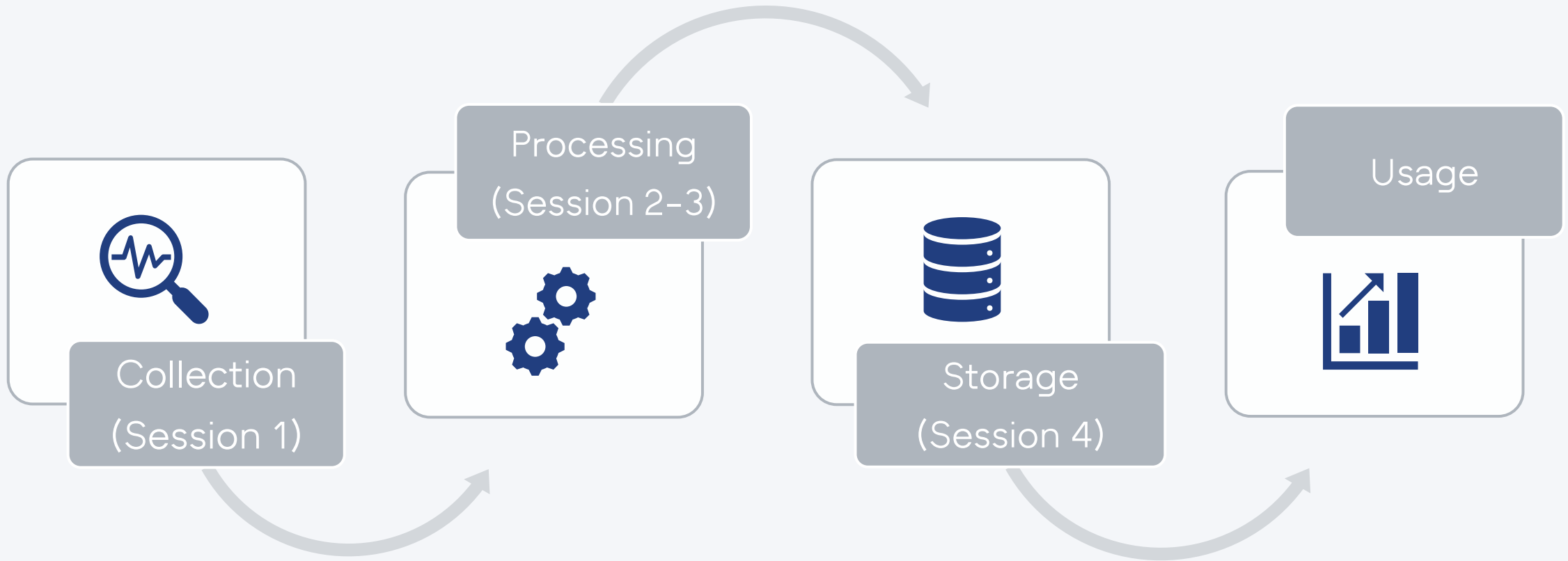
Consulting firms



McKinsey
& Company



From Raw Data to Intelligent Insights: The Modern Data Lifecycle



Session 1: Data Ingestion & Spark SQL

Franzi Kröger

Agenda

Session 1: Data Ingestion & Spark SQL

- Data ingestion concepts & tools (Airbyte, Mage)
- Spark SQL overview & hands-on exercises

Session 2: Single-Node Processing

- Single-node vs. cluster processing
- Tools: Polars, NVIDIA Rapids, DuckDB

Session 3: Processing with dbt

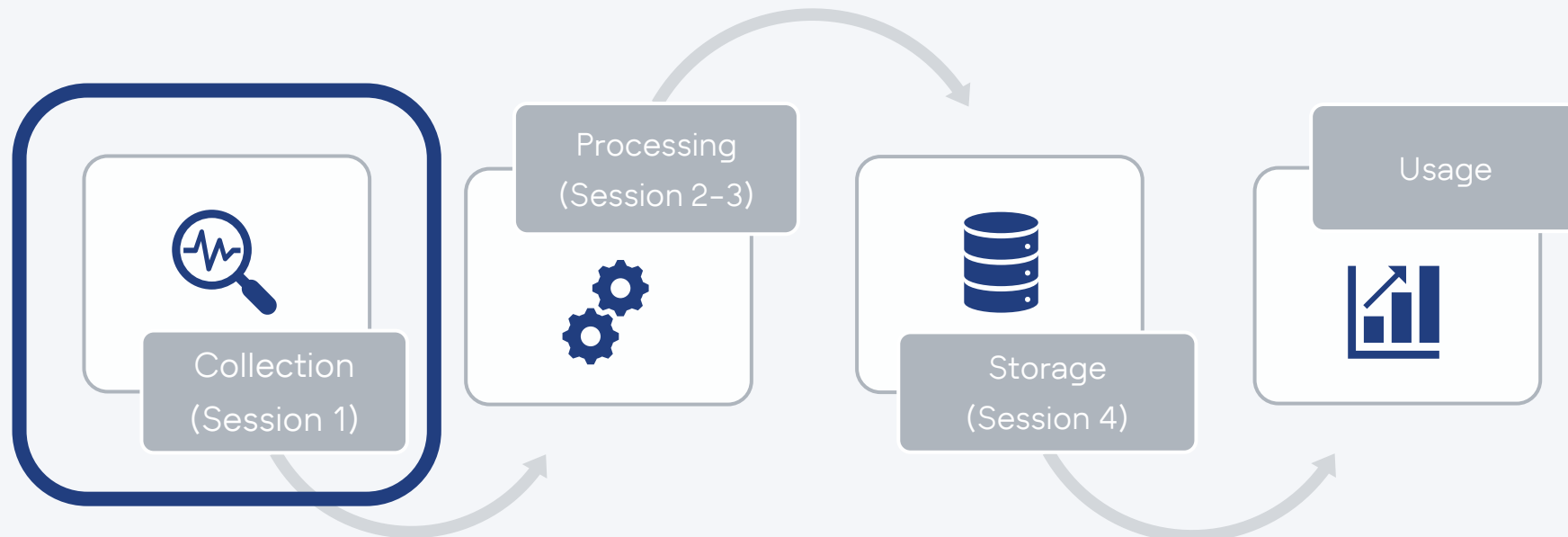
- dbt fundamentals (DuckDB-based)
- End-to-end data transformation exercise

Session 4: Vector Storage & Semantic Search

- Semantic search & vector databases
- End-to-end pipeline with Qdrant

Session 1: Data Ingestion & Spark SQL

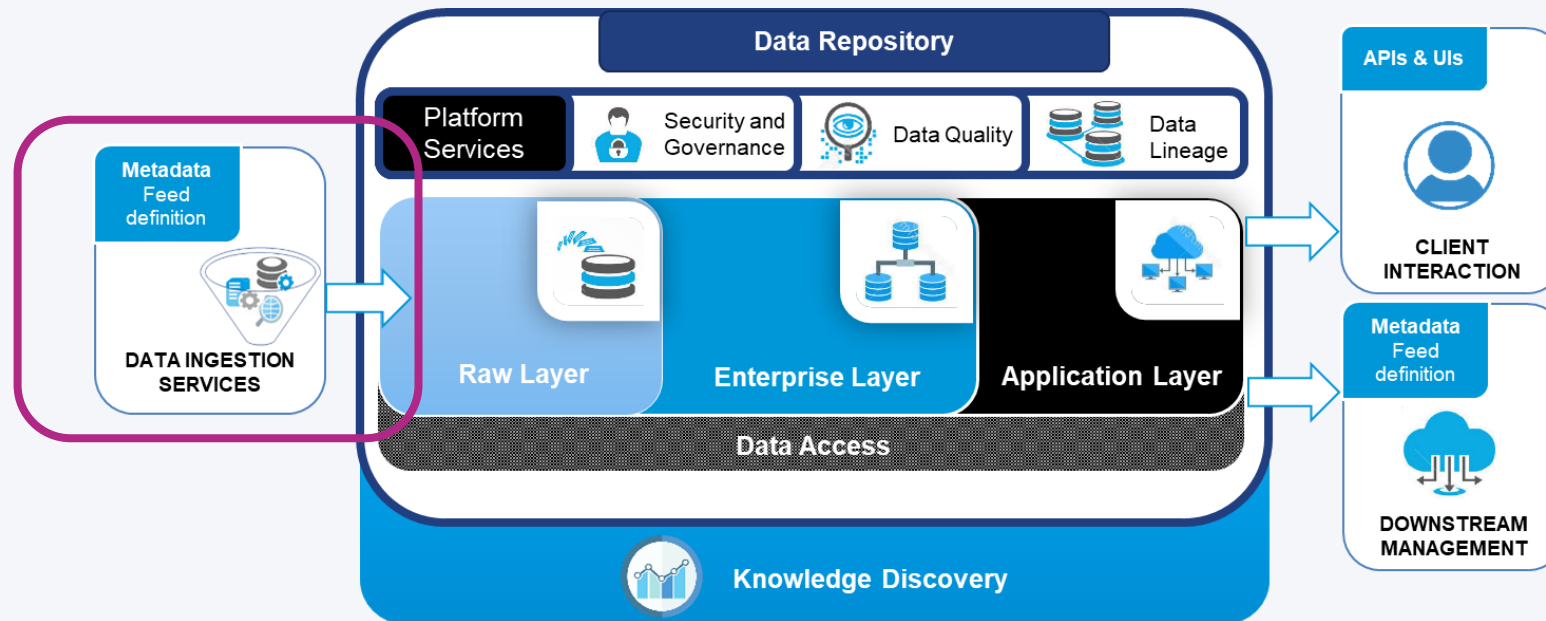
- Data ingestion concepts & tools (Airbyte, Mage)
- Spark SQL overview & hands-on exercises



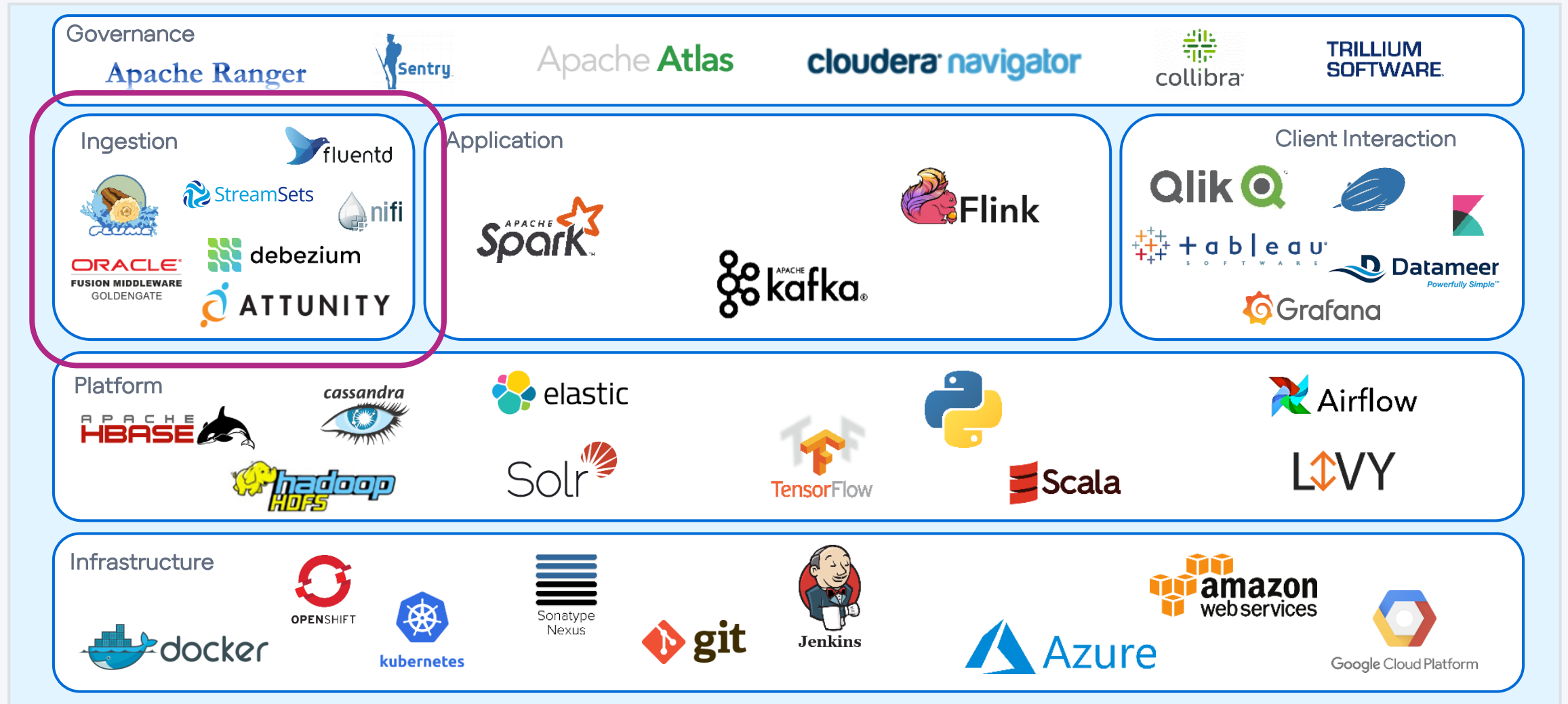
What is Data Ingestion

Data Ingestion is the process of **acquiring data** from source systems and **sending (and/or storing)** in a target system. A typical example of Data Ingestion is the sourcing of data into our **Data Lake**.

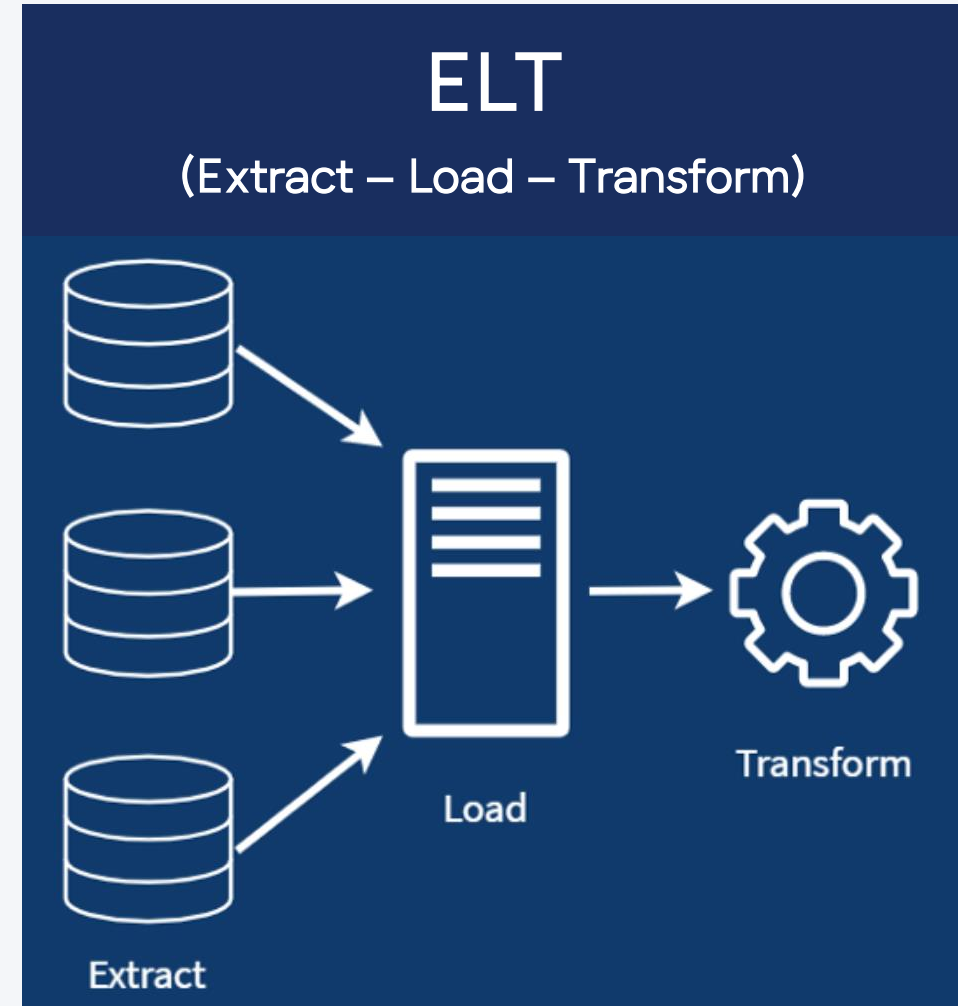
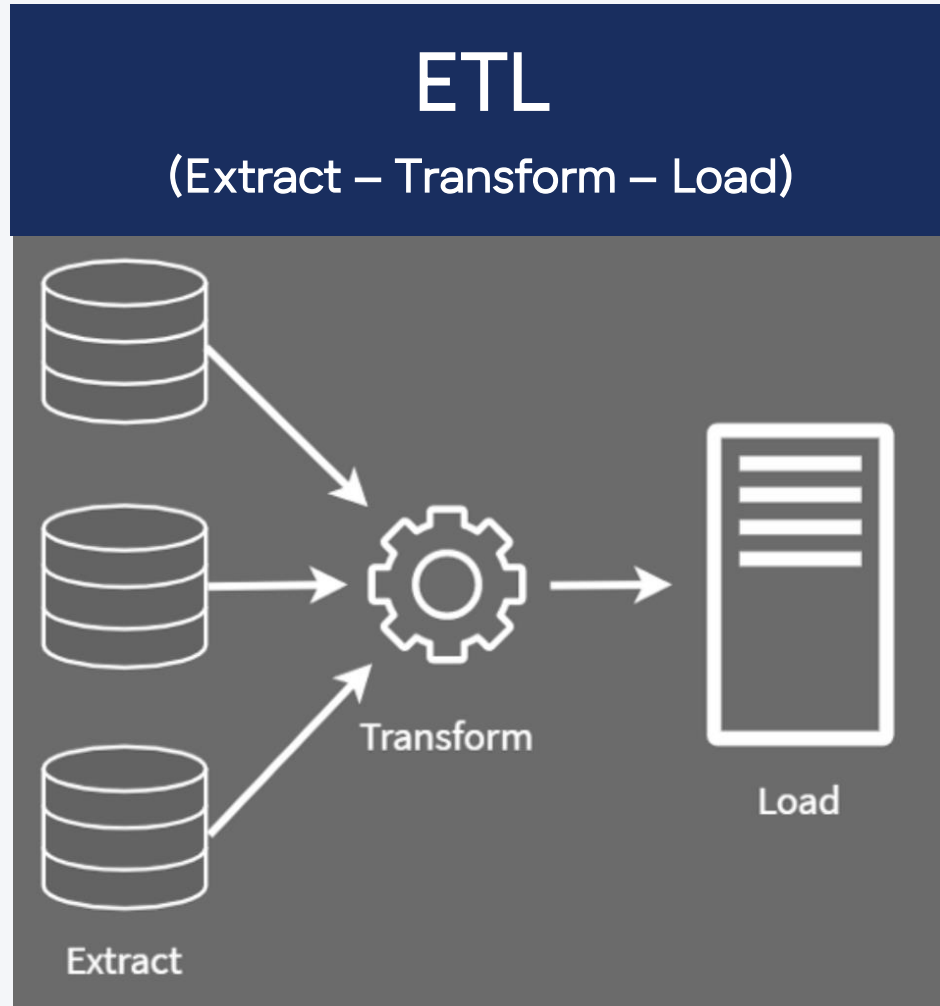
Proper Data Ingestion is key to **obtain better and quicker insights**, reducing our time-to-market.



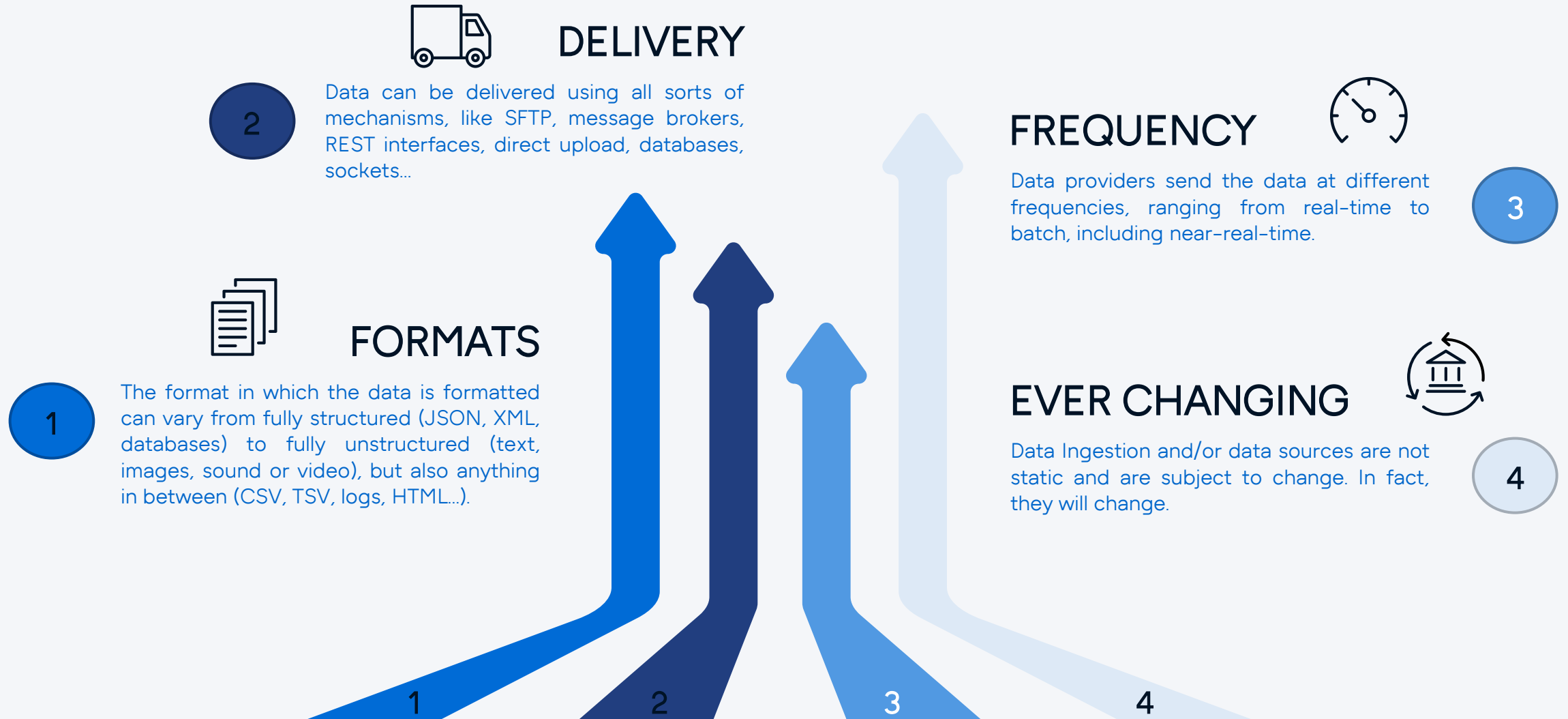
Technology Landscape



Approaches to Data Ingestion



Common pitfalls



Features of an Ingestion Tool

Given the above mentioned pitfalls we need to overcome, what **features** should a data ingestion tool have?



Easy to add new
sources/sinks



Easy to use
(graphical interface)



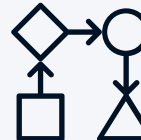
Good component
catalogue



Scalability

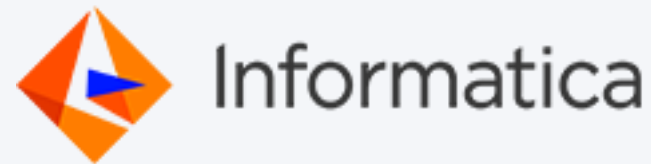


Monitoring



Traceability
(lineage)

Data Ingestion Tools

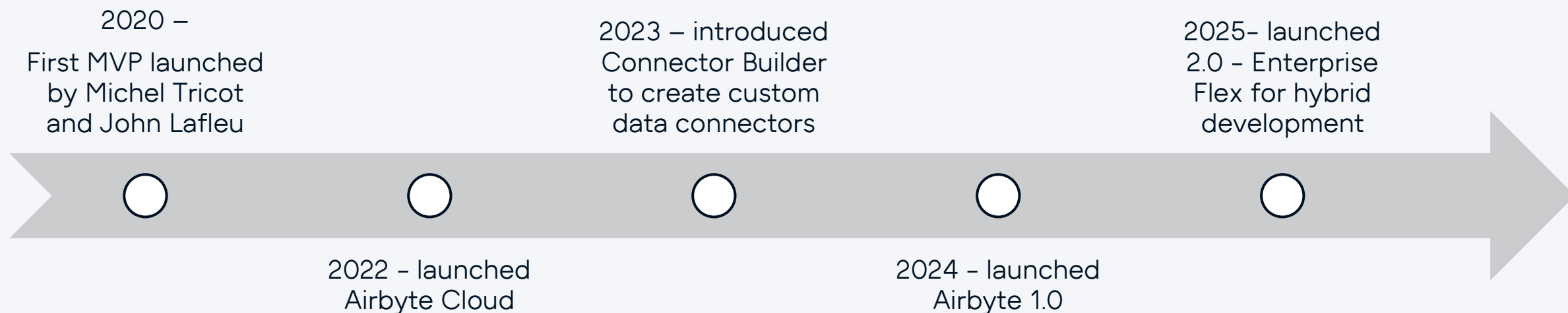




Airbyte

Introduction

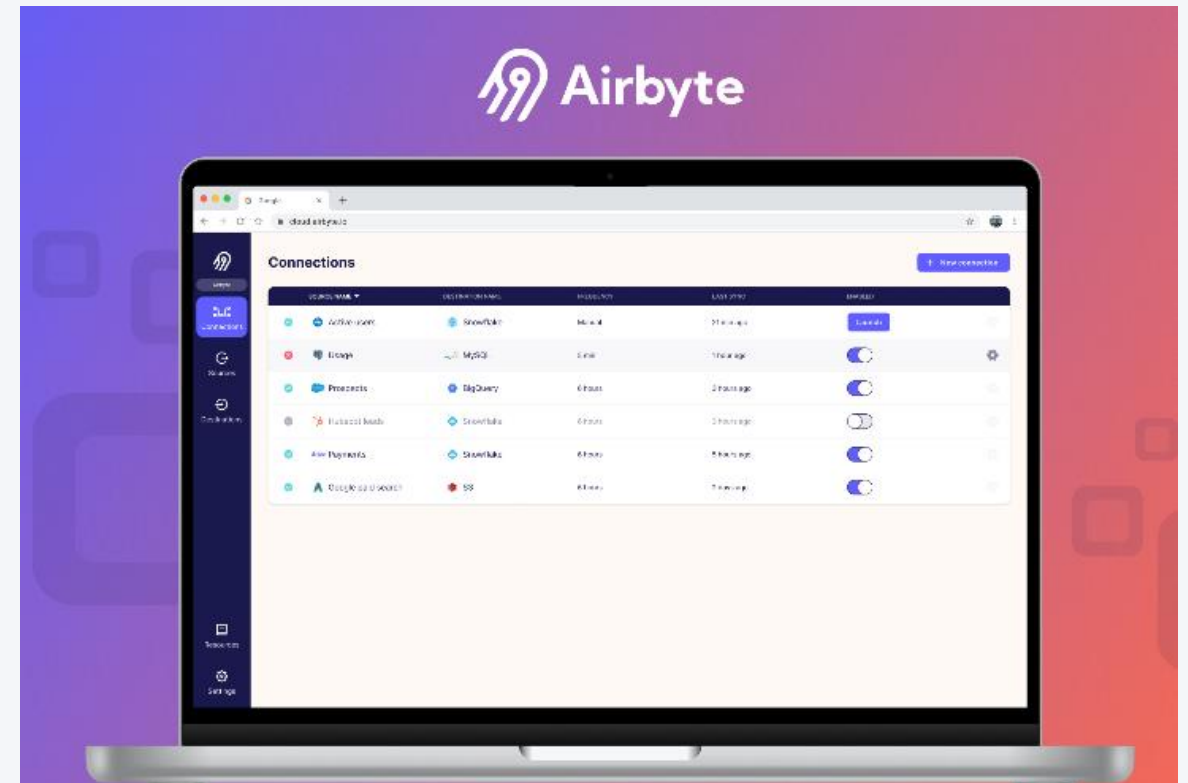
- **Open-source** data integration platform for easy-to-operate **ETL/ELT**
- Syncs data **from sources** (APIs, databases, apps) **to destinations** (warehouses, lakes, analytics tools)
- Modular, **connector-based** architecture



Features

Airbyte allows you to unify your data integration pipelines under one fully managed platform. It is provided **as a service** (cloud) or **on-prem** open-source version.

- **Reliable:** Up to date connectors developed by Airbyte and the community, so you focus on insights.
- **Extensible:** With open-source, you can edit pre-built connectors and build new ones.
- **Integrated:** Full integration with your orchestration systems, such as K8s, Airflow, dbt, etc.
- **Transparent:** Pricing based on volume, easy to understand and predict.



Basic concepts

- **Connector** – Component that extracts or loads data (source/destination)
 - **Source** – Where data comes from (API, database, SaaS app)
 - **Destination** – Where data is loaded (warehouse, lake, database)
- **Connection** – A configured pipeline linking a source to a destination
- **Sync** – A run of the pipeline; can be manual or scheduled
- **Sync Modes** – How Airbyte reads from a source and writes to a destination
 - **Incremental** – Read records added to the source since the last sync job.
 - **Full** refresh – Read everything in the source

Connectors

Types

- ☒ Sources
- ☒ Destinations

Categories

All

New & Noteworthy

Requests

Advertising

Analytics

CRM

Collaboration

Communication

Content

Customer Success

Data Lake

Data Warehouse

Databases

Dev Tools

E-Commerce



Facebook
Marketing
Advertising



Google Ads
Advertising



Google Analytics
Analytics



Google Sheets
Files



HubSpot
Marketing Automation



MSSQL - SQL
Server
Databases



MySQL
Databases



PostgreSQL
Databases



Redshift
Data Warehouse



S3
Data Lake



Salesforce
CRM



Snowflake
Data Warehouse



Webflow
Communication



AWS CloudTrail
Event Streaming



AWS Lake Formation
AWS Datalake
Data Lake



Airtal
Collaboration



Amazon Ads
Advertising



Amazon Seller
Partner
E-Commerce

Sources and Destinations

New Source

Set up the source

Source type

Type to search for a connector

 Greenhouse

 Harvest

 HubSpot

 Instagram

 Intercom

 Iterable

 Klaviyo

+ Request a new connector

New Source

Set up the source

Source type

 Instagram ✓

Source name ⓘ

Instagram

Authenticate your Instagram account

Start Date ⓘ

Set up source

Connections

Create a source

Create a destination

Set up connection

Connection name*

Pick a name to help you identify this connection

PokeAPI <> End-to-End Testing (/dev/null)

Transfer

Replication frequency*

Set how often data should sync to the destination

Every 24 hours

Streams

Destination Namespace*

Define the location where the data will be stored in the destination

Mirror source structure

Destination Stream Prefix (Optional)

Add a prefix to stream names (ex. "airbyte_" causes "projects" => "airbyte_projects")

prefix

Activate the streams you want to sync

Refresh source schema

Search stream name

	Sync	Source ⓘ	Sync mode ⓘ	Cursor field ⓘ	Primary key ⓘ	Destination ⓘ	
		Namespace	Stream name	Source Destination		Namespace	Stream name
☐	>	☒	No namespace	pokemon	Full refresh Overwrite	<source schema>	pokemon

Set up connection

Execution



echiner@gma...

Connections

Sources

Destinations

400

Resources

Support

Settings

You are using a trial of Airbyte. Your trial ends in **14 days**.

Connection

PokeAPI

ALPHA

→

End-to-End Testing (/dev/null)

ALPHA

ENABLED 

Status

Replication

Transformation

Settings

Sync History

Reset your data

Sync now

 Sync Succeeded

296.08 KB | 1 emitted record | 1 committed record | 52s

1:32PM 10/23

/tmp/workspace/743839/0/logs.log



23 2022-10-23 11:33:09 replication-orchestrator > Setting up replication worker...

24 2022-10-23 11:33:10 replication-orchestrator > Using default value for environment variable LOG_CONNECTOR_MESSAGES: 'false'

25 2022-10-23 11:33:10 replication-orchestrator > Running replication worker...

26 2022-10-23 11:33:10 replication-orchestrator > start sync worker. job id: 743839 attempt id: 0

27 2022-10-23 11:33:10 replication-orchestrator >

28 2022-10-23 11:33:10 replication-orchestrator > ----- START REPLICATION -----

29 2022-10-23 11:33:10 replication-orchestrator >

30 2022-10-23 11:33:10 replication-orchestrator > configured sync modes: {null.pokemon=full_refresh - overwrite}

31 2022-10-23 11:33:10 replication-orchestrator > Running destination...

32 2022-10-23 11:33:10 replication-orchestrator > Attempting to start pod = destination-dev-null-write-743839-0-kjipa for airbyte/destination-dev-null:0.1.1 with resources io.airbyte.config.ResourceRequirements@4fe9adfb[cpuRequest=0.5,cpuLimit=0.5,memoryRequest=500Mi,memoryLimit=500Mi]

33 2022-10-23 11:33:10 replication-orchestrator > destination-dev-null-write-743839-0-kjipa stdoutLocalPort = 9879

34 2022-10-23 11:33:10 replication-orchestrator > destination-dev-null-write-743839-0-kjipa stderrLocalPort = 9878

35 2022-10-23 11:33:10 replication-orchestrator > Creating stderr socket server...

36 2022-10-23 11:33:10 replication-orchestrator > Creating stdout socket server...

37 2022-10-23 11:33:10 replication-orchestrator > Creating pod destination-dev-null-write-743839-0-kjipa...

38 2022-10-23 11:33:13 replication-orchestrator > Waiting for init container to be ready before copying files...

39 2022-10-23 11:33:13 replication-orchestrator > Init container present..

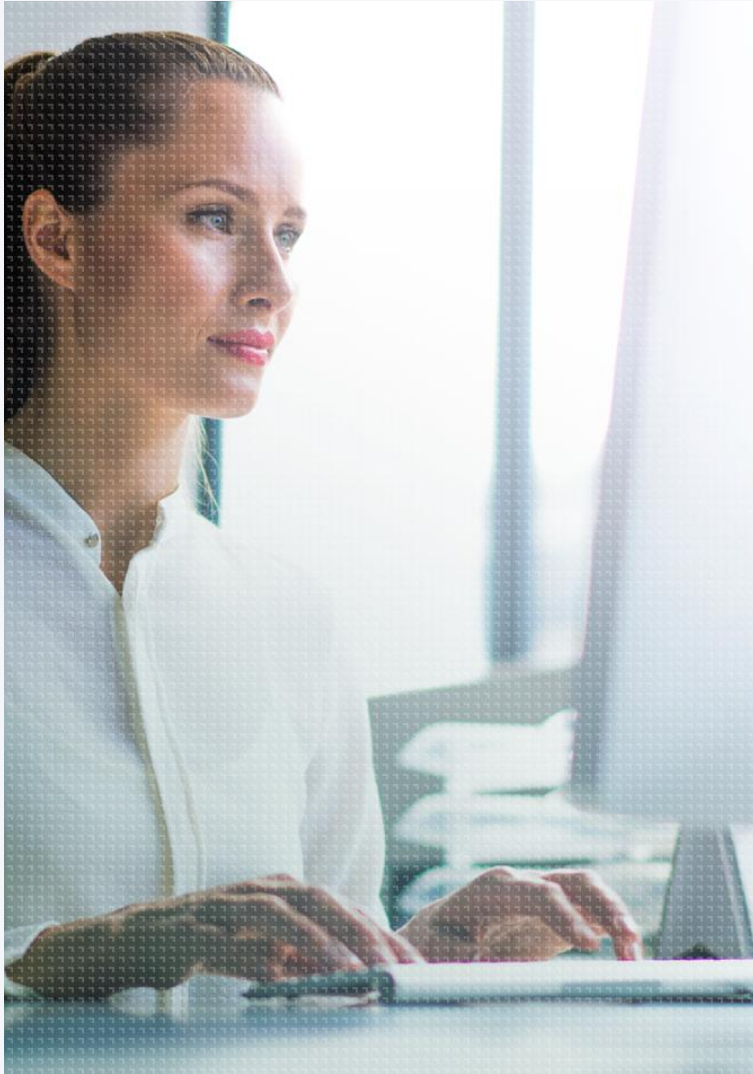
40 2022-10-23 11:33:15 replication-orchestrator > Init container ready..

41 2022-10-23 11:33:15 replication-orchestrator > Copying files...

GFT ■

Herramientas de última generación para BigData

12/19/2025



Exercise 1-2: Setting up Airbyte to ingest data into PostgreSQL

- We want to **ingest a CSV file into a PostgreSQL** database.
- Optional: Try other destination sources.
- Open GitHub repository and follow the instructions
<https://github.com/frkroe/state-of-the-art-tools-for-big-data/>
- Let's go!!



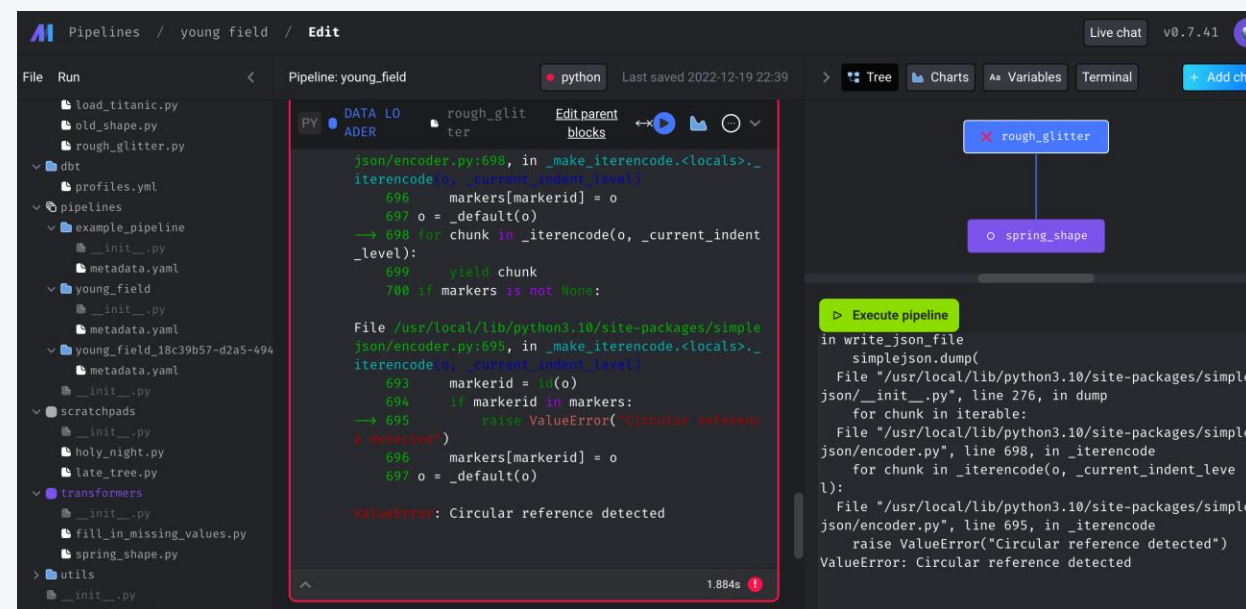
Mage



Introduction

Mage is available in two versions: As fully managed platform **as a service** (cloud) or **self-hosted on-prem** self-hosted version with Docker.

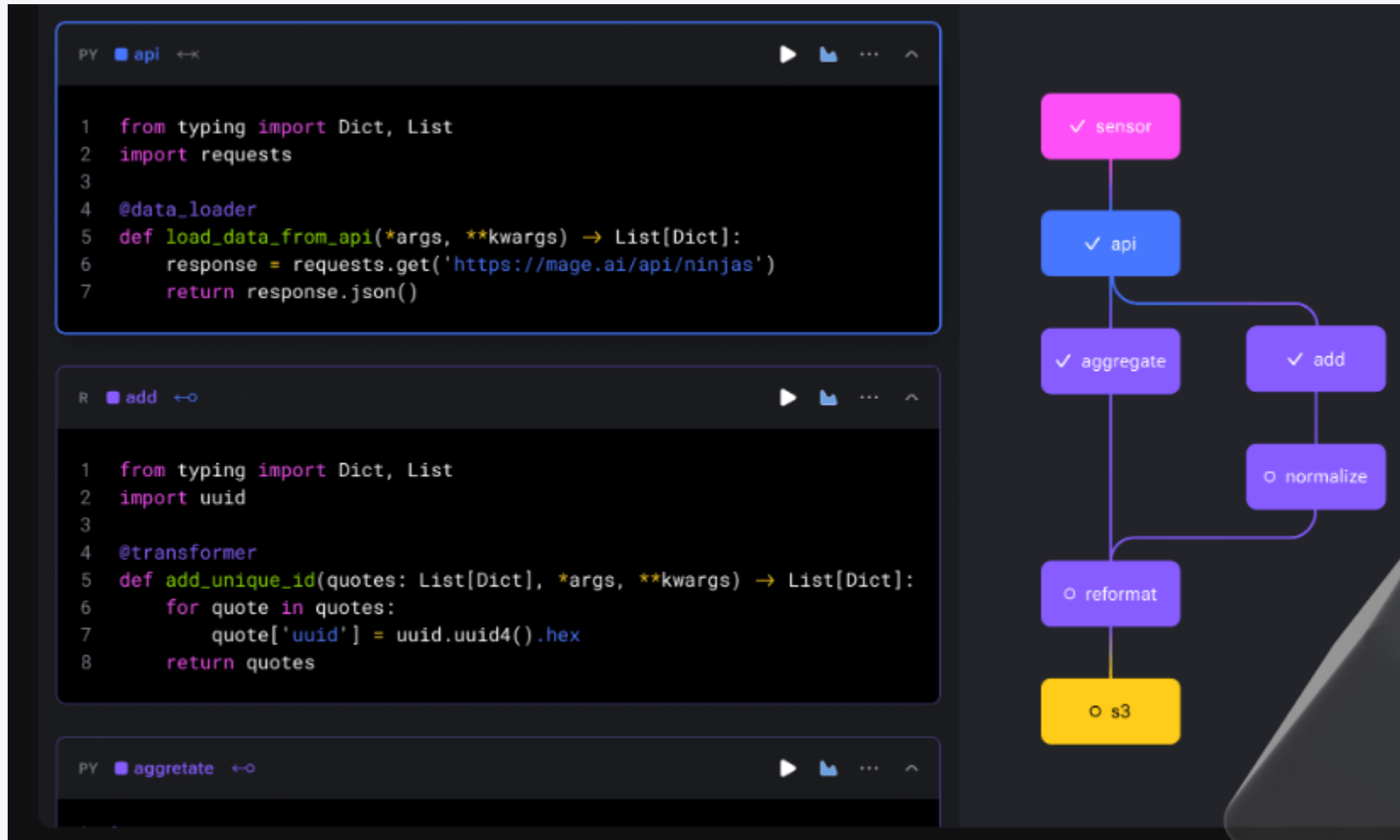
- **Orchestration**: Schedule and manage data pipelines with observability.
- **Notebook editor**: Interactive Python, SQL, & R editor for coding data pipelines.
- **Data integration**: Synchronize data from 3rd party sources to internal destinations.
- **Streaming**: Ingest and transform real-time data.
- **dbt**: Build, run, and manage your dbt models with Mage.

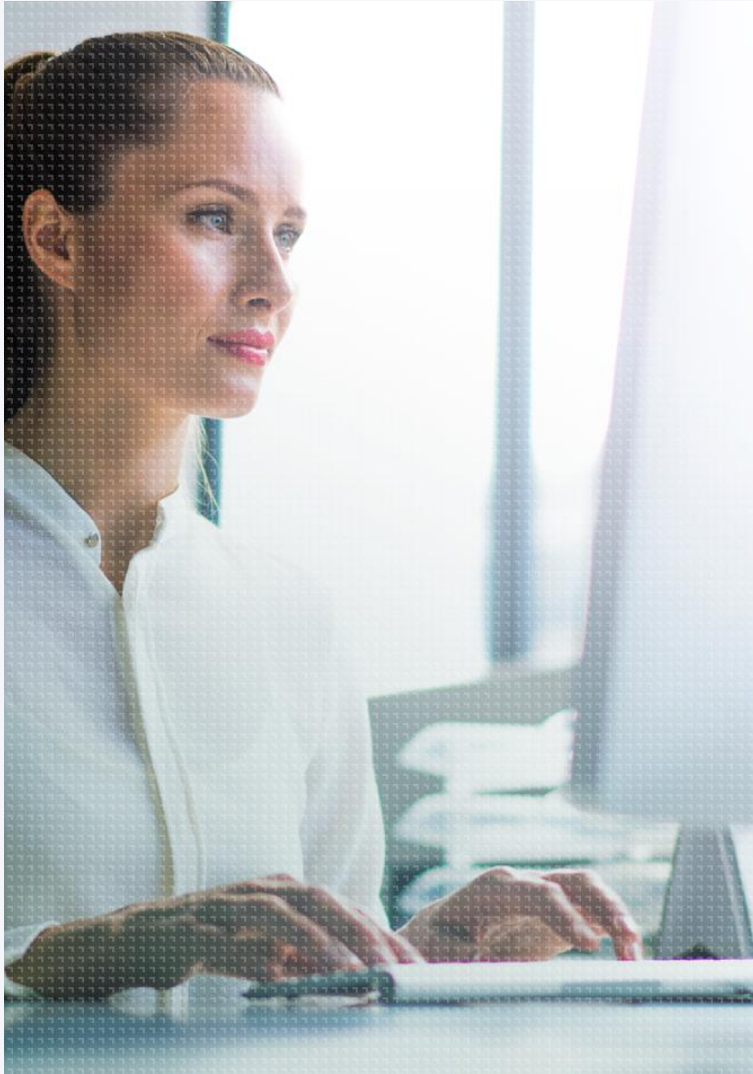


Basic concepts

- **Project**: Like a repository on GitHub; this is where you write all your code.
- **Pipeline**: Contains references to all the blocks of code you want to run, charts for visualizing data, and organizes the dependency between each block of code.
- **Block**: A file with code that can be executed independently or within a pipeline.
- **Data product**: Every block produces data after it's been executed. These are called data products in Mage.
- **Trigger**: A set of instructions that determine when or how a pipeline should run.
- **Run**: Stores information about when it was started, its status, when it was completed, any runtime variables used in the execution of the pipeline or block, etc.

Mage Pipeline





Exercise 3-6: Get familiar with Mage AI

- **Let's get familiar** with Mage AI by understanding and creating pipelines.
- Open GitHub repository and follow the instructions
<https://github.com/frkroe/state-of-the-art-tools-for-big-data/>
- Let's go!!



Spark SQL

Recap I: What is Apache Spark?

- **Apache Spark** is an open-source cluster computing framework
- Requires
 - **Cluster manager**
 - Standalone – a simple cluster manager included with Spark
 - Apache Mesos – a general cluster manager
 - Hadoop YARN – the resource manager in Hadoop 2
 - **Distributed storage system**
 - Hadoop Distributed File System (HDFS),
 - Cassandra
 - Amazon S3, Google GCS etc...
- Also supports pseudo-distributed mode for development and testing
 - Local file system and one worker per CPU core

Recap II: What is Apache Spark?

Spark is an analytics engine for real-time and batch data processing with the following key **features**:



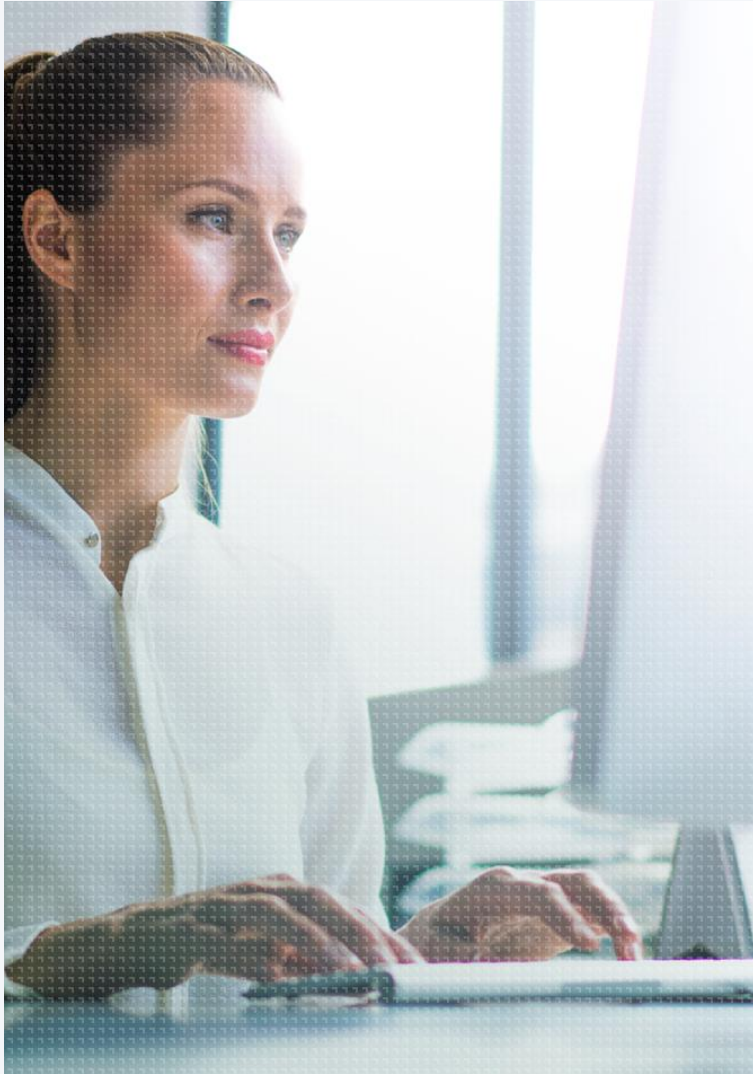
Batch & streaming
data workloads



In-memory processing
for high performance



Scalability
across clusters

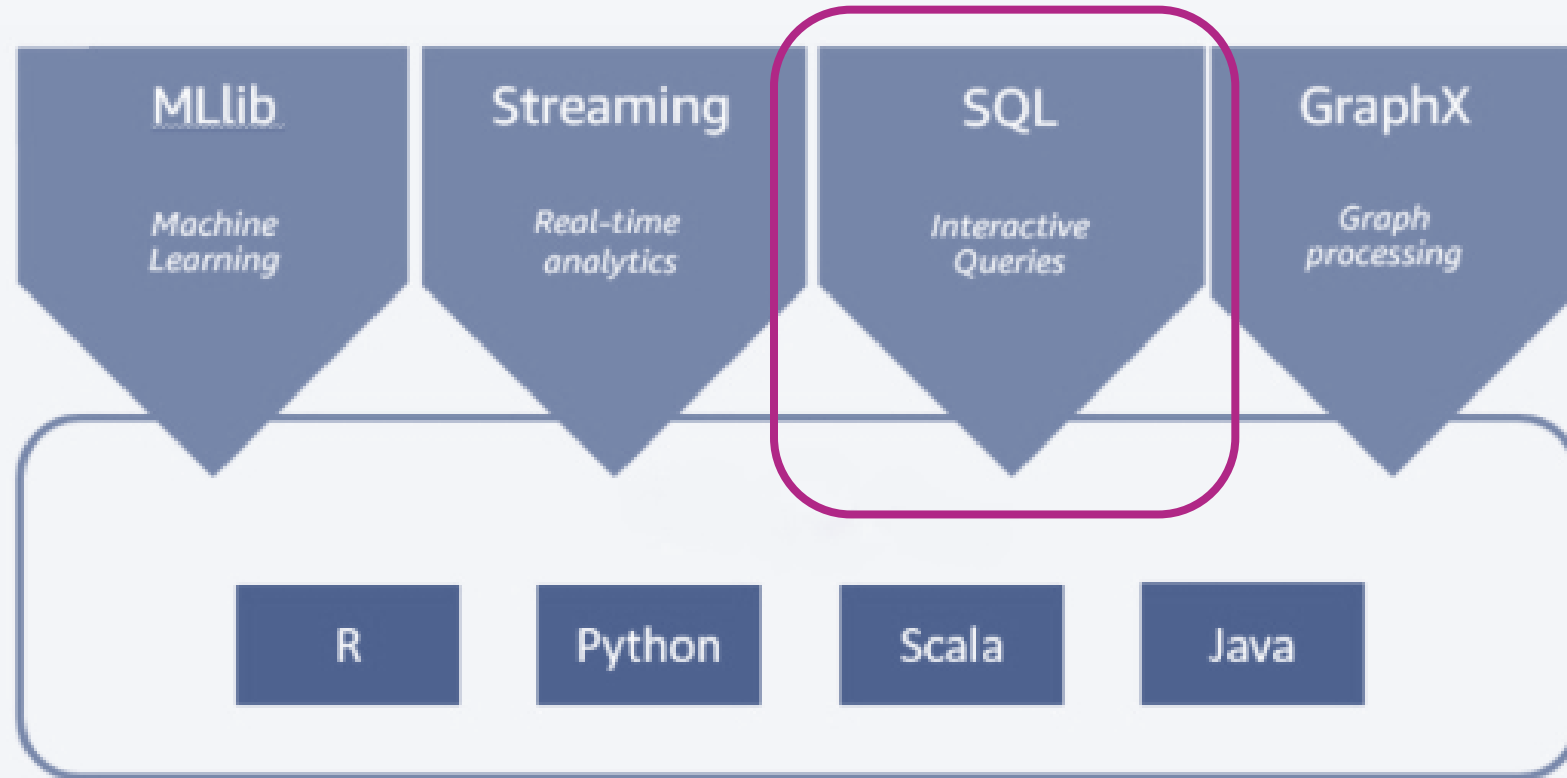


Exercise 7: Recap on PySpark

- Launch spark and open the [*01-getting-started.ipynb*](#) notebook in the Jupyter environment. Follow the instructions provided in the notebook.
- You will learn/ recap on how to
 - Build spark session
 - Save data as DataFrames
 - Perform basic operations, like filtering and aggregation
- Let's go!!

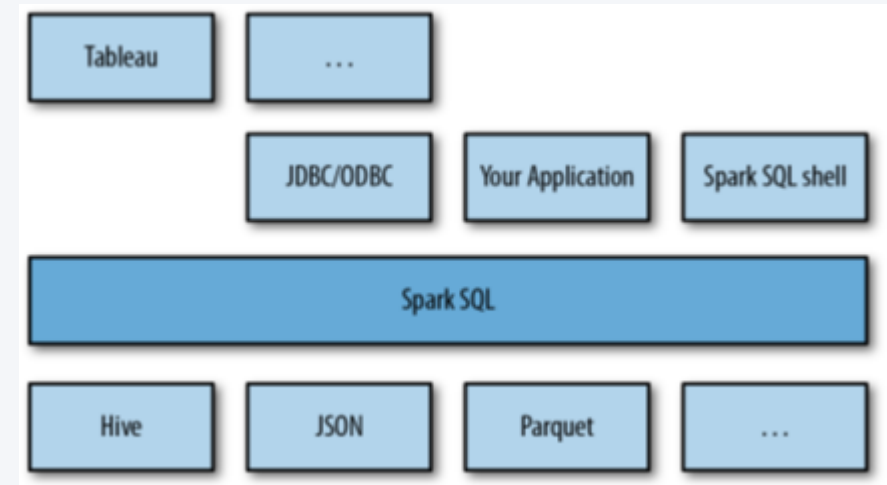
Recap III: What is Apache Spark?

Its **rich ecosystem** of native language APIs and built-in libraries enables end-to-end development for analytics, machine learning, streaming, and graph workloads.



Introduction to Spark SQL

- Spark SQL was first released in Spark 1.0 (May 2014) by Databricks
- Interface for working with structured (schema) and semi-structured data
- Spark SQL applies structured views to data stored in different formats
- Three main capabilities:
 - **DataFrame abstraction** for structured datasets.
 - Like tables in Relational database
 - Read & write in **structured formats** (JSON, Hive, Parquet,...)
 - **Query data using SQL** inside Spark program & from external tools using JDBC/ODBC



RDD vs DataFrame vs Dataset

RDD (Resilient Distributed Dataset)

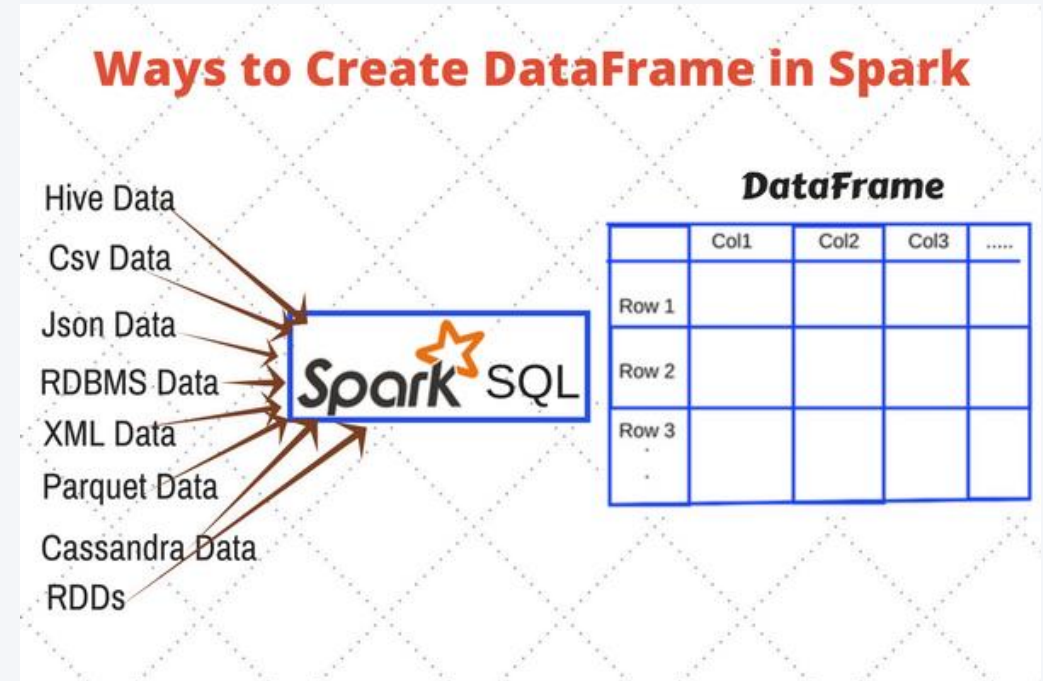
- Low-level distributed collection of objects.
- No schema → less optimization. Good for unstructured or complex data.
- Supports functional ops (map, filter).

DataFrame

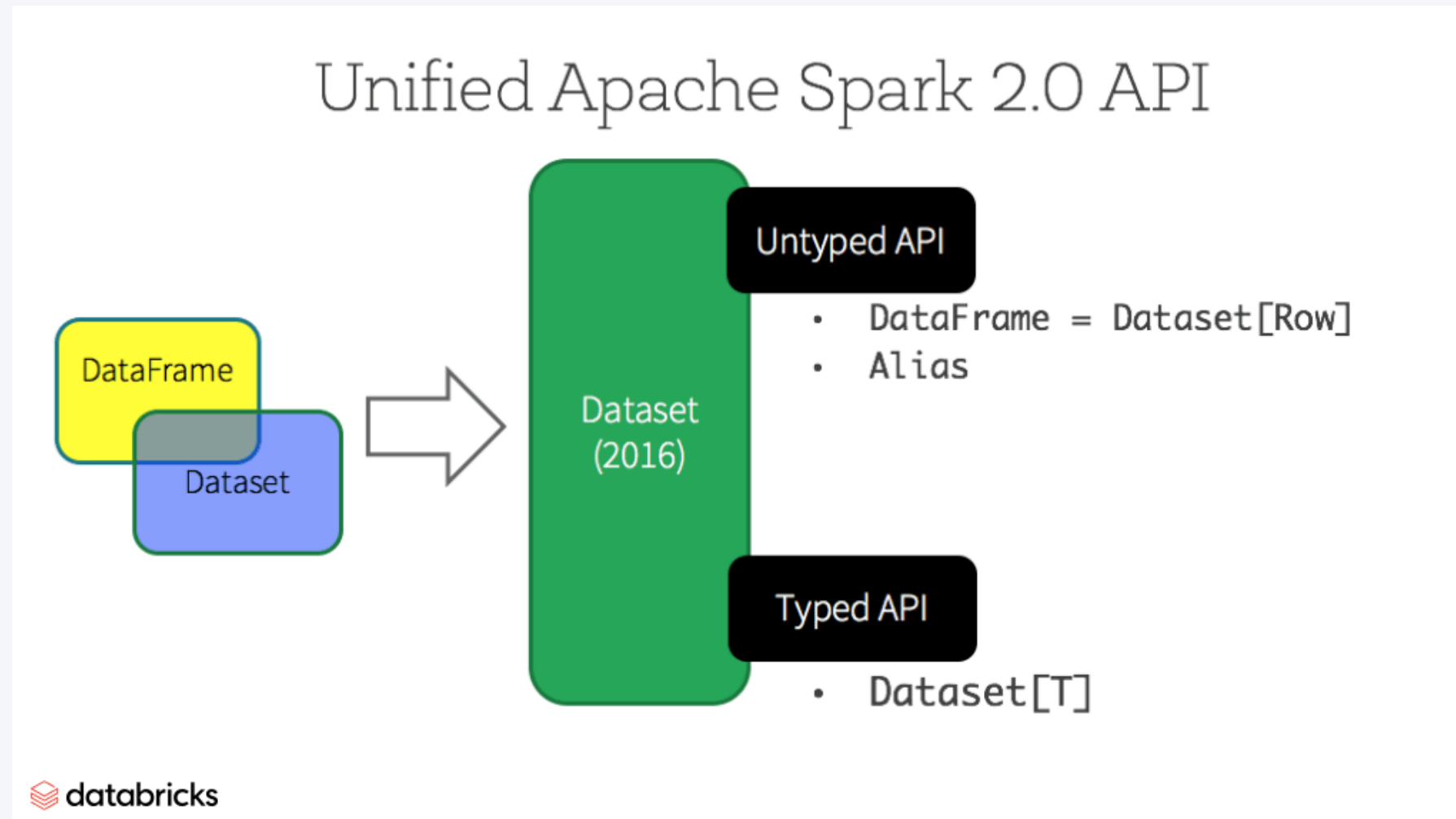
- Table-like structure with rows and named columns.
- Has a **schema** and uses **Catalyst** for optimization. Best for **structured data** and analytics.
- Supports **SQL queries**.

Dataset (Scala/Java)

- **Typed** version of a DataFrame.
- Combines **type safety** (like RDD) + **optimization** (like DataFrame).



RDD vs DataFrame vs Dataset



DataFrame & Dataset

```
1 fifa_df = spark.read.csv("path-of-file/fifa_players.csv", inferSchema = True, header = True)
2
3 fifa_df.show()
```

RoundID	MatchID	Team Initials	Coach Name	Line-up	Player Name	Position	Event
201	1096	FRA	CAUDRON Raoul (FRA)	S	Alex THEPOT	GK	null
201	1096	MEX	LUQUE Juan (MEX)	S	Oscar BONFIGLIO	GK	null
201	1096	FRA	CAUDRON Raoul (FRA)	S	Marcel LANGILLER	null	G40'
201	1096	MEX	LUQUE Juan (MEX)	S	Juan CARRENO	null	G70'
201	1096	FRA	CAUDRON Raoul (FRA)	S	Ernest LIBERATI	null	null
201	1096	MEX	LUQUE Juan (MEX)	S	Alfredo SANCHEZ	null	null
201	1096	FRA	CAUDRON Raoul (FRA)	S	Augustin CHANTREL	null	null
201	1096	MEX	LUQUE Juan (MEX)	S	Efrain AMEZCUA	null	null

```
1 fifa_df.printSchema()
```

```
root
|-- RoundID: integer (nullable = true)
|-- MatchID: integer (nullable = true)
|-- Team Initials: string (nullable = true)
|-- Coach Name: string (nullable = true)
|-- Line-up: string (nullable = true)
|-- Player Name: string (nullable = true)
|-- Position: string (nullable = true)
|-- Event: string (nullable = true)
```

only showing top 20 rows

DataFrame & Dataset

- DataFrames are Datasets of a special Row object
- Row is a generic untyped JVM object. Dataset is a collection of strongly-typed JVM.
- Python doesn't support Spark Datasets

```
> case class Person(name: String, age: Int)

val personDS = Seq(Person("Max", 33), Person("Adam", 32), Person("Muller", 62)).toDS()
personDS.show()
```

```
+-----+----+
|  name|age|
+-----+----+
|   Max| 33|
|  Adam| 32|
|Muller| 62|
+-----+----+
```

DataFrame & Dataset

- DataFrame

```
df.groupBy( "dept").avg("age")
```

- SQL

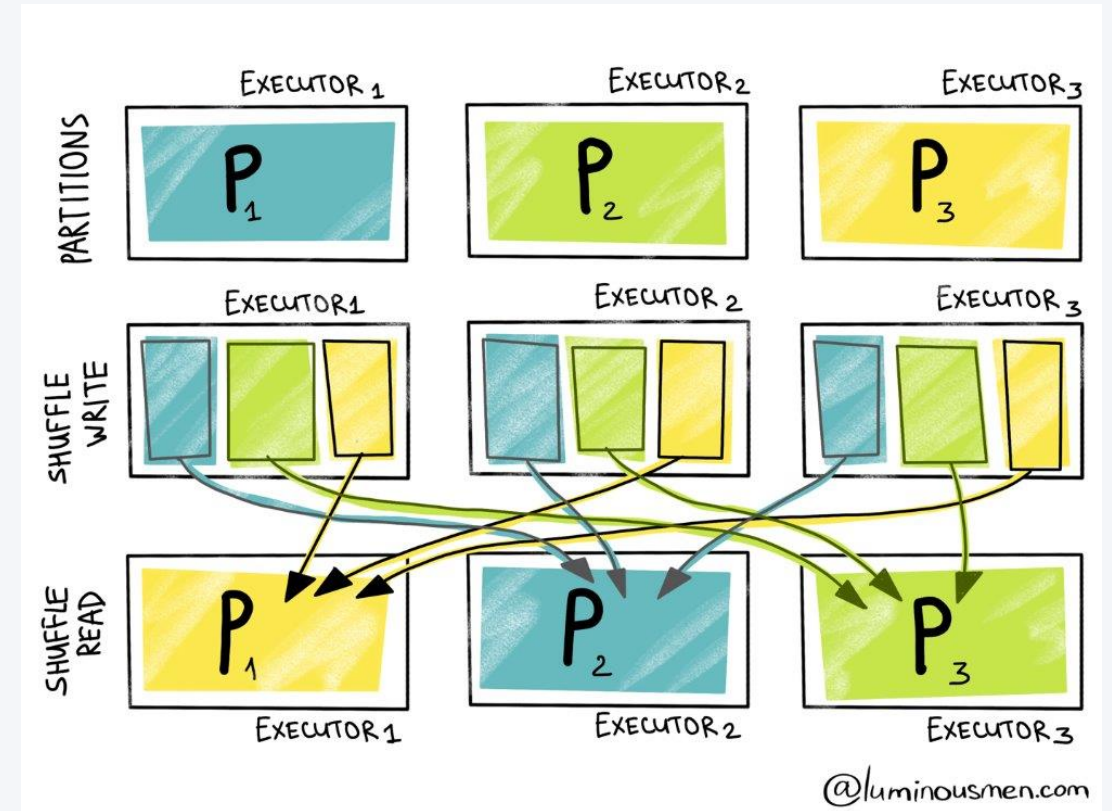
```
spark.sql("select dept, avg(age) from data group by dept")
```

- RDD

```
rdd.map { case (dept, age) => dept -> (age,1) }  
  .reduceByKey { case ( (a1, c1), (a2, c2) ) => (a1 + a2, c1 + c2) }  
  .map { case (dept, (age, c)) => dept -> age/ c }
```

DataFrame Partitioning

- Data Partitioning is critical to data processing, especially for large volume of data
- Spark assigns **one task for each partition**
 - Few partitions, application won't utilize all cores
 - Too many partitions → Overhead



Repartition vs Coalesce

- Repartition can be used to either increase and decrease the number of partitions
- It's a full shuffle operation. Partitions are equally distributed

```
val new_df = df.repartition(100)
```

```
val new_df = df.repartition(100, $"id")
```

- Coalesce reduces the number of partitions.
- Avoids shuffling.

```
val new_df = df.coalesce(10)
```

Repartitioning

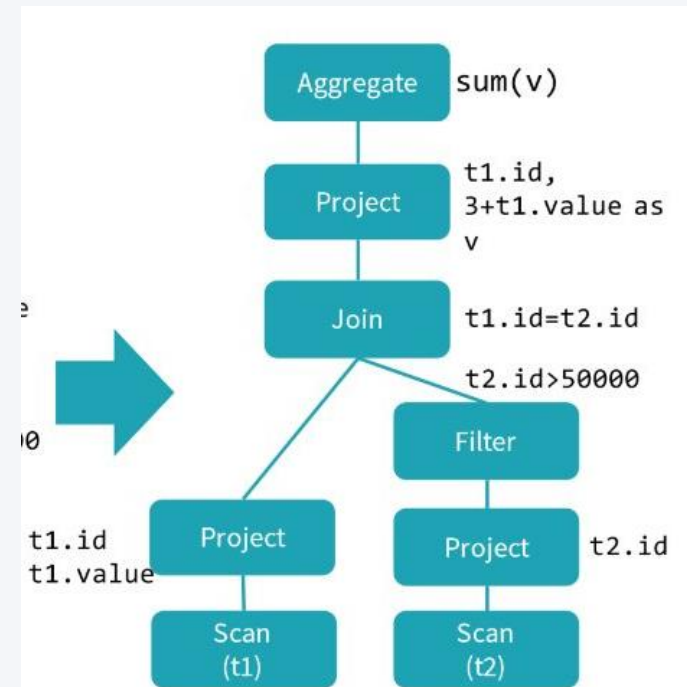
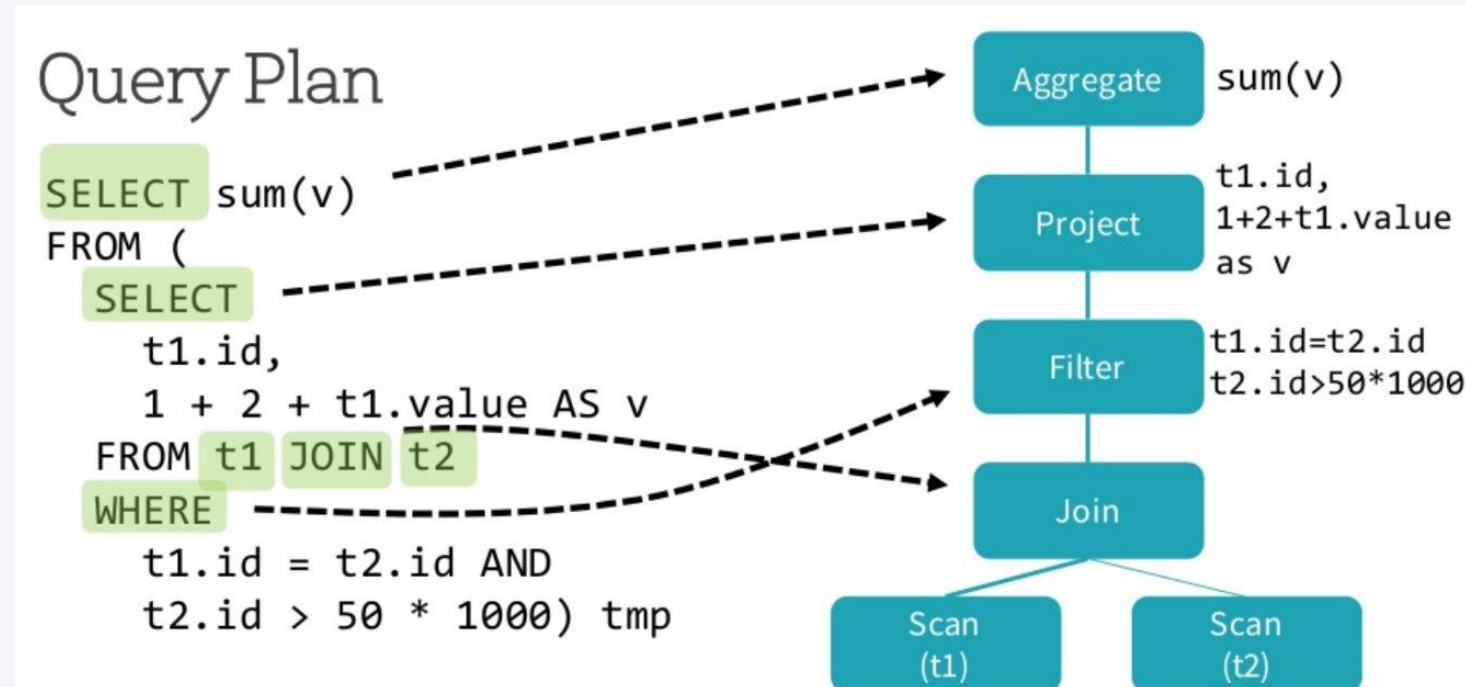
```
19M repartition/part-00000  
19M repartition/part-00001  
19M repartition/part-00002  
19M repartition/part-00003  
19M repartition/part-00004  
19M repartition/part-00005  
19M repartition/part-00006  
19M repartition/part-00007  
19M repartition/part-00008  
19M repartition/part-00009
```

Coalesce

```
33M coalesce/part-00000  
29M coalesce/part-00001  
30M coalesce/part-00002  
31M coalesce/part-00003  
32M coalesce/part-00004  
33M coalesce/part-00005
```

Catalyst

A catalyst finds out the most efficient plan to execute the query.



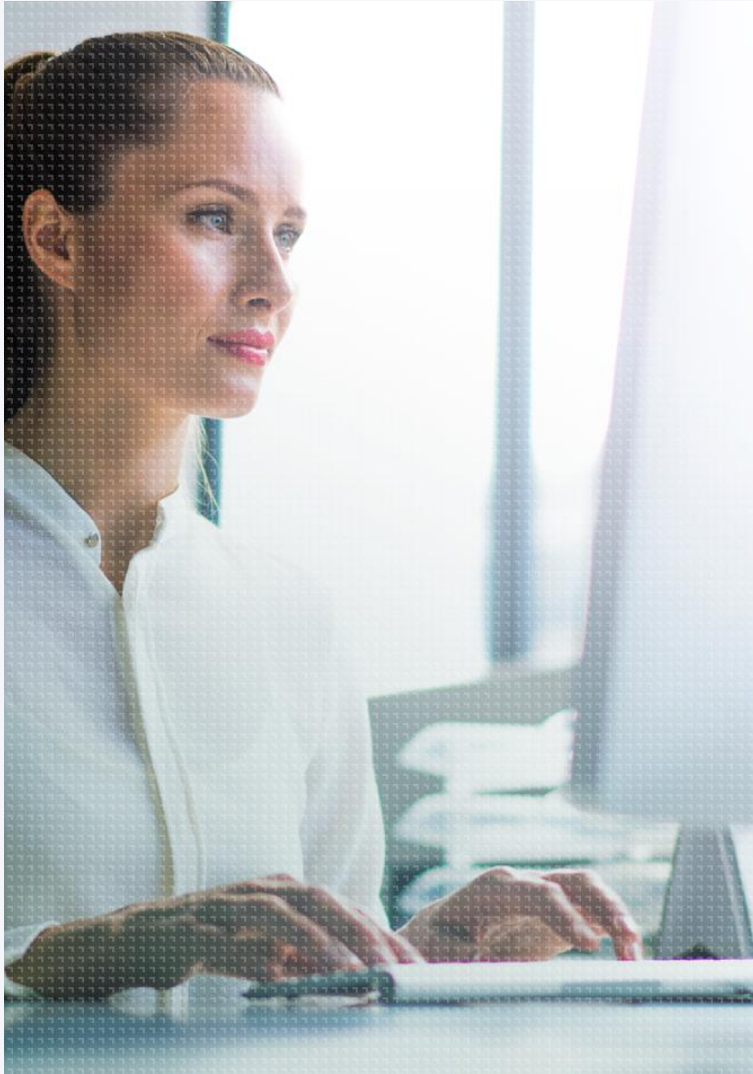
UDF (User Defined Functions)

- Allows you to extend Spark SQL with custom (business) logic not available in built-in functions
- To implement domain-specific operations (e.g., text cleaning, scoring models)
- Lets you apply Python/Scala code to Spark DataFrame columns
- Executes in parallel across the cluster

```
val squared = (s: Long) => { s * s }  
  
spark.udf.register("square", squared)  
  
spark.sql("select id, square(id) as id_squared from test")
```

```
import org.apache.spark.sql.functions.{col, udf}  
  
val squared = udf((s: Long) => s * s)  
  
df.select(squared(col("id")) as "id_squared"))
```

But note:
Performance is crucial!
UDF are slower than built-in function



Exercise 8 : Spark SQL

- Launch spark and open the [02-spark-sql.ipynb](#) notebook in the Jupyter environment. Follow the instructions provided in the notebook.
- You will learn/ recap on how to
 - Build spark session
 - Load csv data as DataFrame
 - Perform basic operations, like filtering and aggregation as before but also as SQL queries
 - Use UDF
- Let's go!!

Session 2: Single-Node Processing

When a single machine beats a cluster

Carlos Ferrer

Agenda

Session 1: Data Ingestion & Spark SQL

- Data ingestion concepts & tools (Airbyte, Mage)
- Spark SQL overview & hands-on exercises

Session 2: Single-Node Processing

- Single-node vs. cluster processing
- Tools: Polars, NVIDIA Rapids, DuckDB

Session 3: Processing with dbt

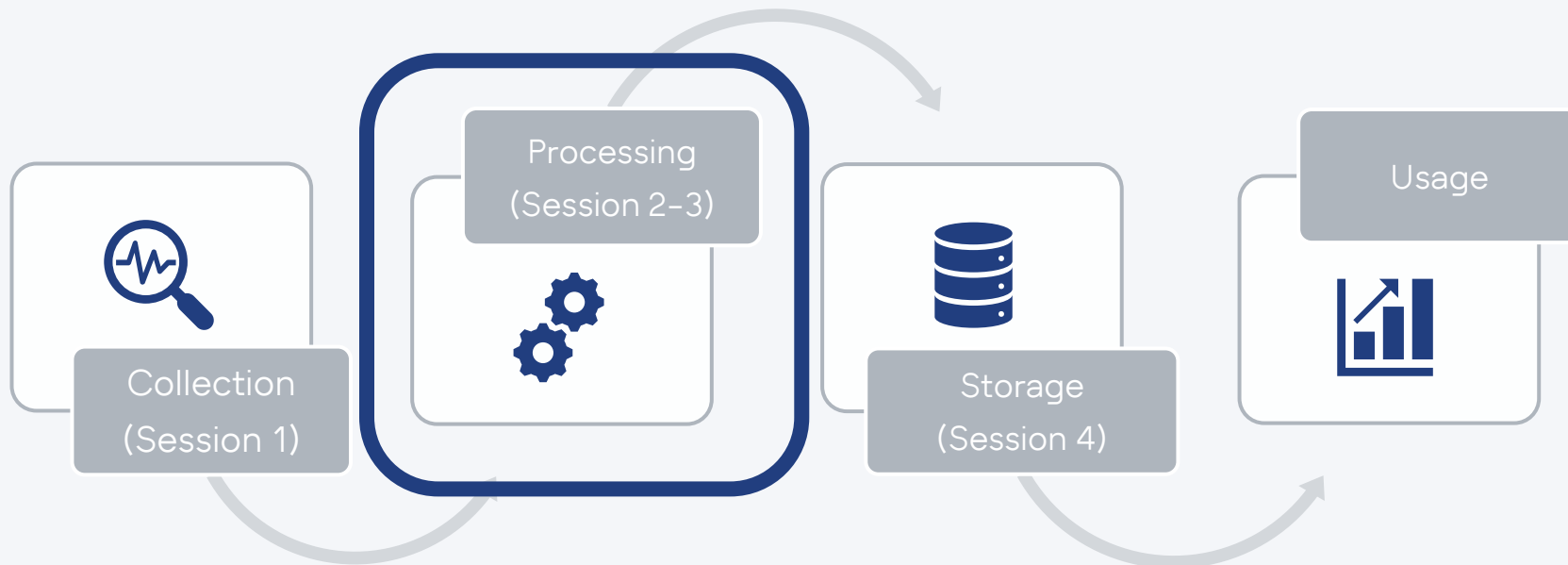
- dbt fundamentals (DuckDB-based)
- End-to-end data transformation exercise

Session 4: Vector Storage & Semantic Search

- Semantic search & vector databases
- End-to-end pipeline with Qdrant

Session 2: Single-Node Processing

- Single-node vs. cluster processing
- Tools: Polars, NVIDIA Rapids, DuckDB



Introduction and Session Goal

80% of a data engineer's daily workloads fit in a single machine

- Modern hardware (CPU + NVMe + RAM) handles mid-large datasets easily.
- New engines (Polars, DuckDB, RAPIDS) outperform clusters for many tasks.

Session goals:

- Understand the **single-node** processing paradigm
- Compare it directly with cluster processing
- Learn the modern tools: **Polars, DuckDB, NVIDIA RAPIDS**

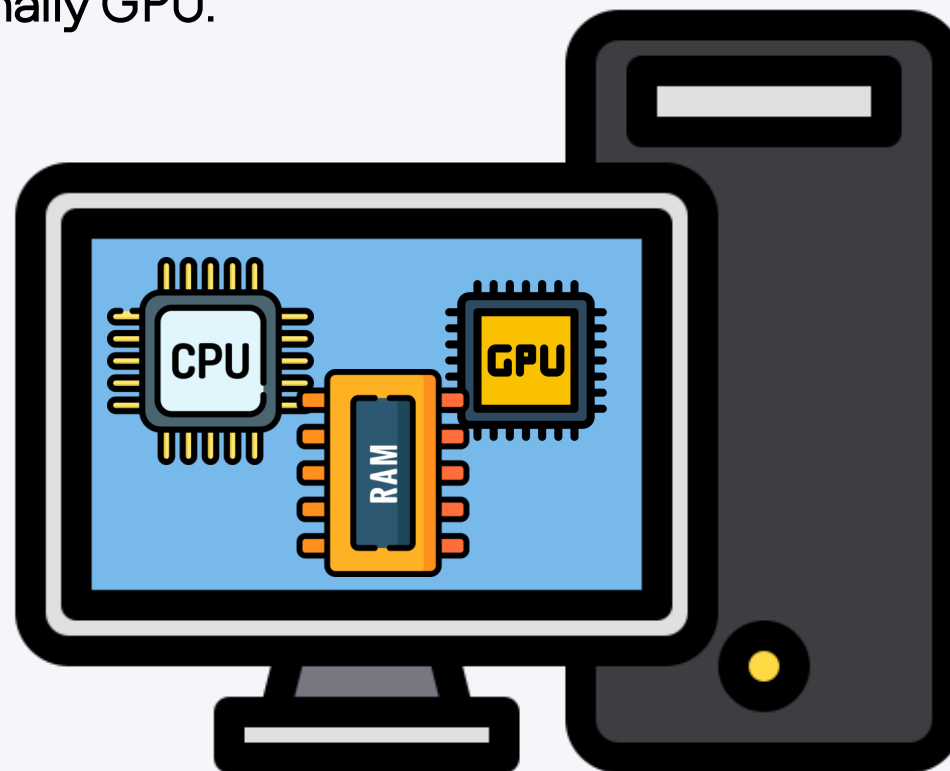
Data Engineering Traditional ETL vs. Modern Data Engineering Approaches

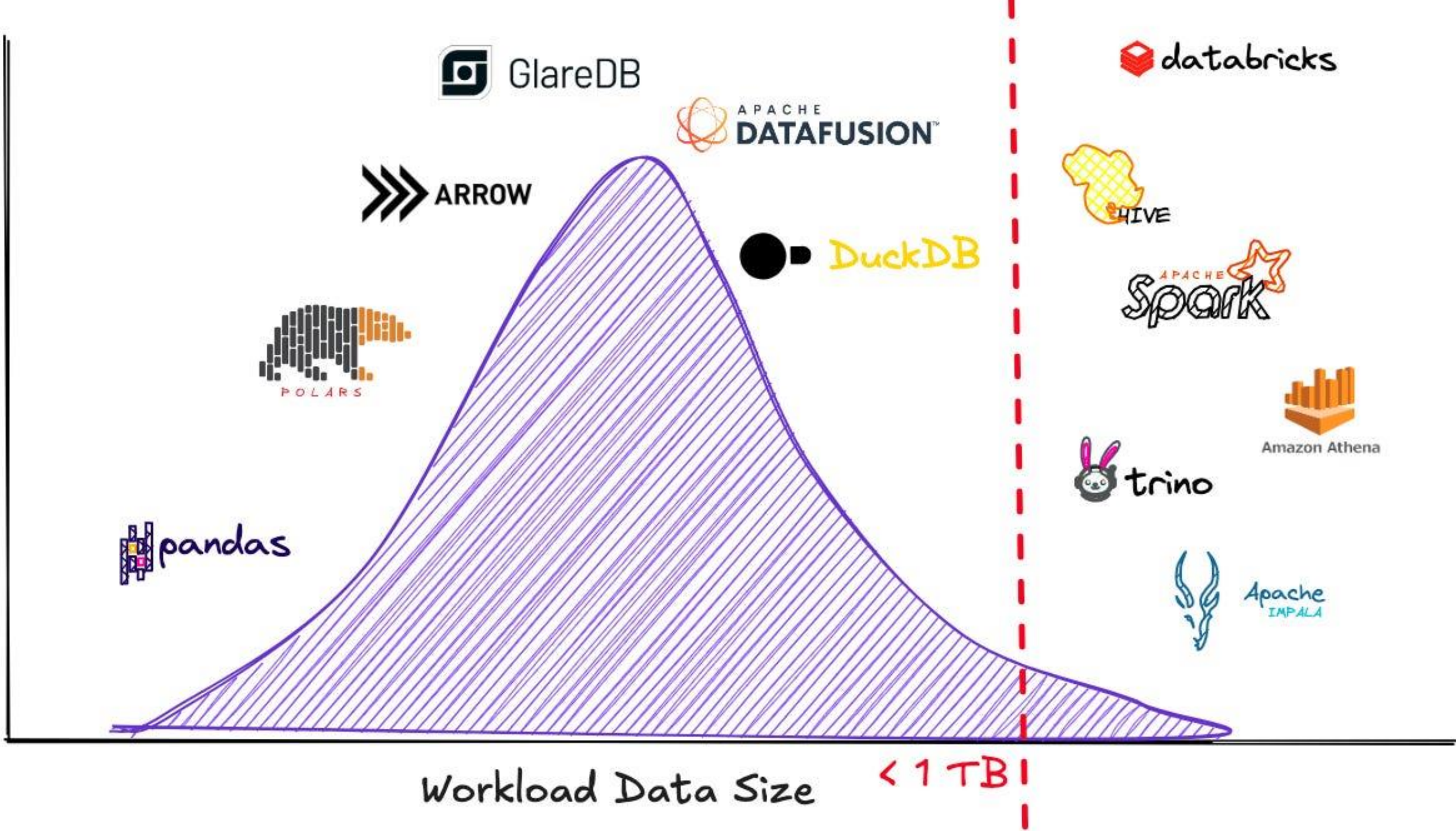
Feature	Traditional ETL	Modern Data Engineering
Data Source	Primarily structured	Flexible (structured, semi-structured, unstructured)
Processing Mode	Batch Processing	Batch, Micro-batch, Real-time Streaming
Data Transformation	Separate transformation stage	Transformation can occur during or after loading
Tools	Specialized ETL tools	Open-source frameworks, cloud-based tools
Scalability	Limited scalability	Highly scalable
Data Latency	High latency due to batch processing	Lower latency with real-time processing options

What Is Single-Node Processing?

“Running data workloads entirely on one machine, leveraging its CPU, RAM, NVMe and optionally GPU.”

- Local memory (RAM)
- Local compute (CPU core)
- Local storage (NVMe, high IOPS)
- Optional GPU (RAPIDS)



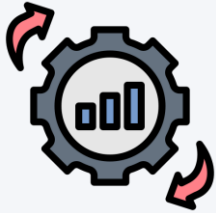


Real-World Use Cases



- Data Exploration

Single-node is unbeatable for exploratory analysis.



- ETL / Transformations

Cluster overhead only pays off when the dataset is genuinely large.



- ML Prototyping

You don't want to wait for a cluster to spin up while experimenting.

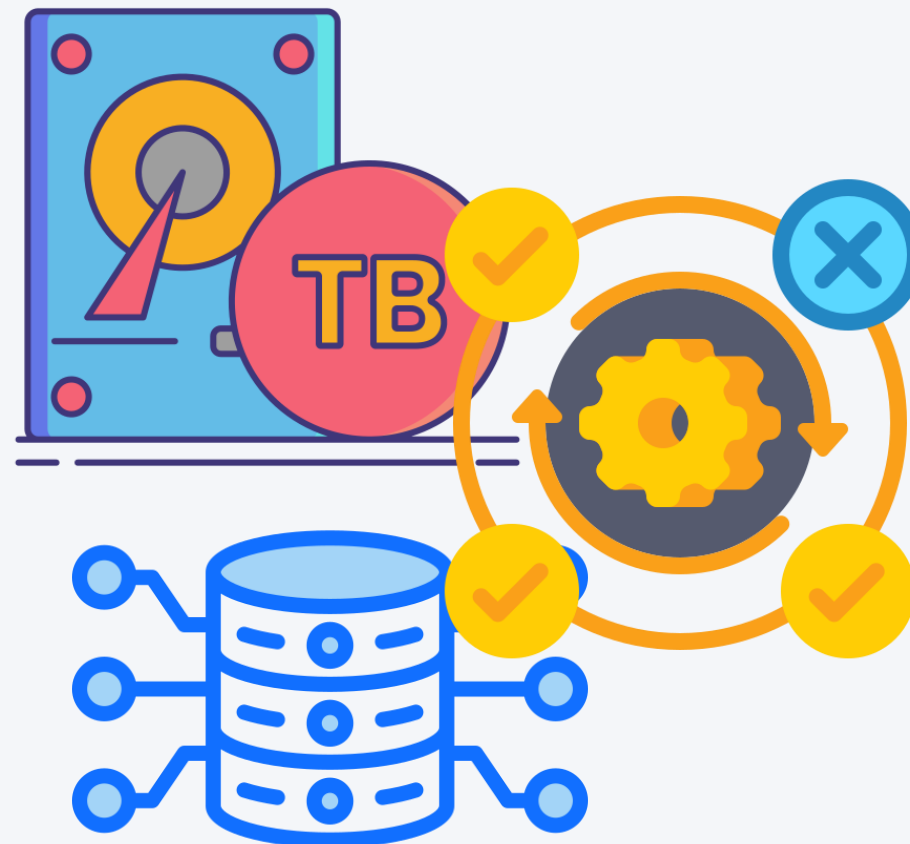


- Embedded Analytics

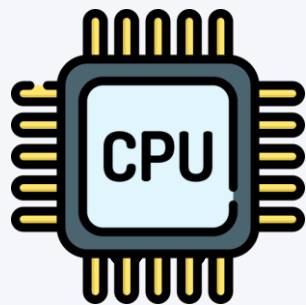
Embedded analytics = instant, portable, zero-infrastructure analytics.

When NOT to Use Single-Node

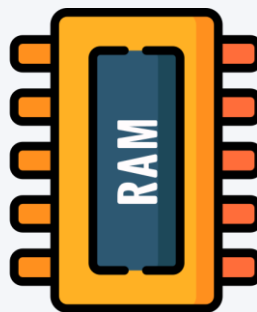
- Datasets far beyond RAM / NVMe
Tens of billions of rows, TB-scale joins
- Workloads requiring fault tolerance
Critical pipelines, long-running jobs
- High-throughput or real-time streaming
Kafka → Flink/Spark Structured Streaming



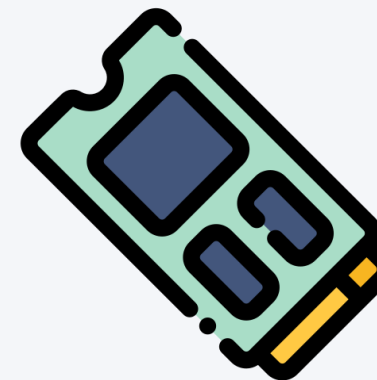
Physical Limits of Single-Node Processing



- Controls how fast calculations happen
- More cores = more parallel work
- Modern CPUs are very powerful



- Limits how much data fits in memory
- Columnar formats help use less RAM
- Sometimes the speed of RAM matters more than size



- Very fast disk (much faster than old SSDs)
- Lets you work with data bigger than your RAM
- Great for streaming large datasets

Advantages of Single-Node Processing

Very fast start-up
(no cluster spin-up, no scheduler overhead)



Lower cost
(one machine vs. a whole cluster)



Reproducible
(easy to run the same code locally or in CI)



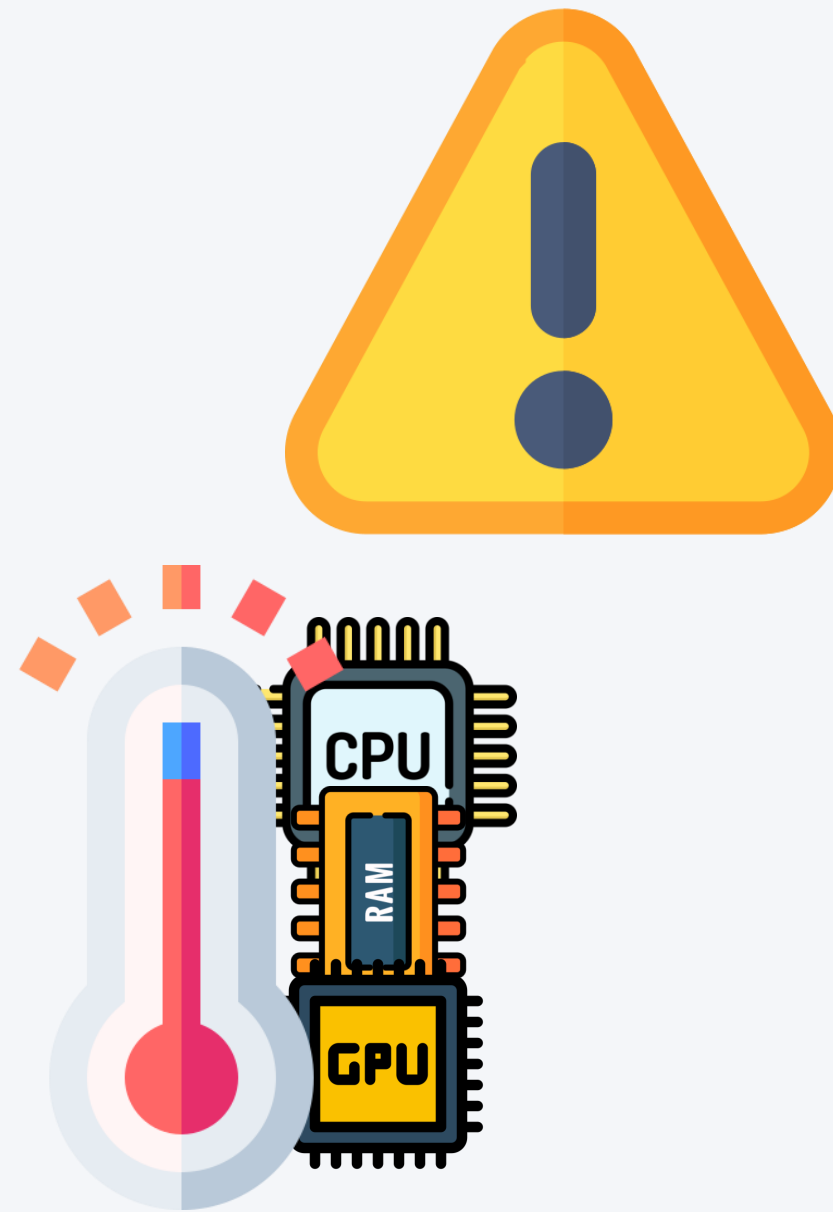
Simpler to develop
(no distributed debugging, no cluster config)



Low latency
(ideal for exploration, prototyping, small-mid ETL)

Limitations of Single-Node Processing

- **Limited by hardware**
(CPU, RAM, NVMe — you can only scale so far)
- **No fault tolerance**
(if the machine dies, the job dies)
- **Huge datasets don't fit**
(terabytes/petabytes → cannot run on one node)
- **Not ideal for massive throughput**
(e.g., 24/7 pipelines processing billions of rows)

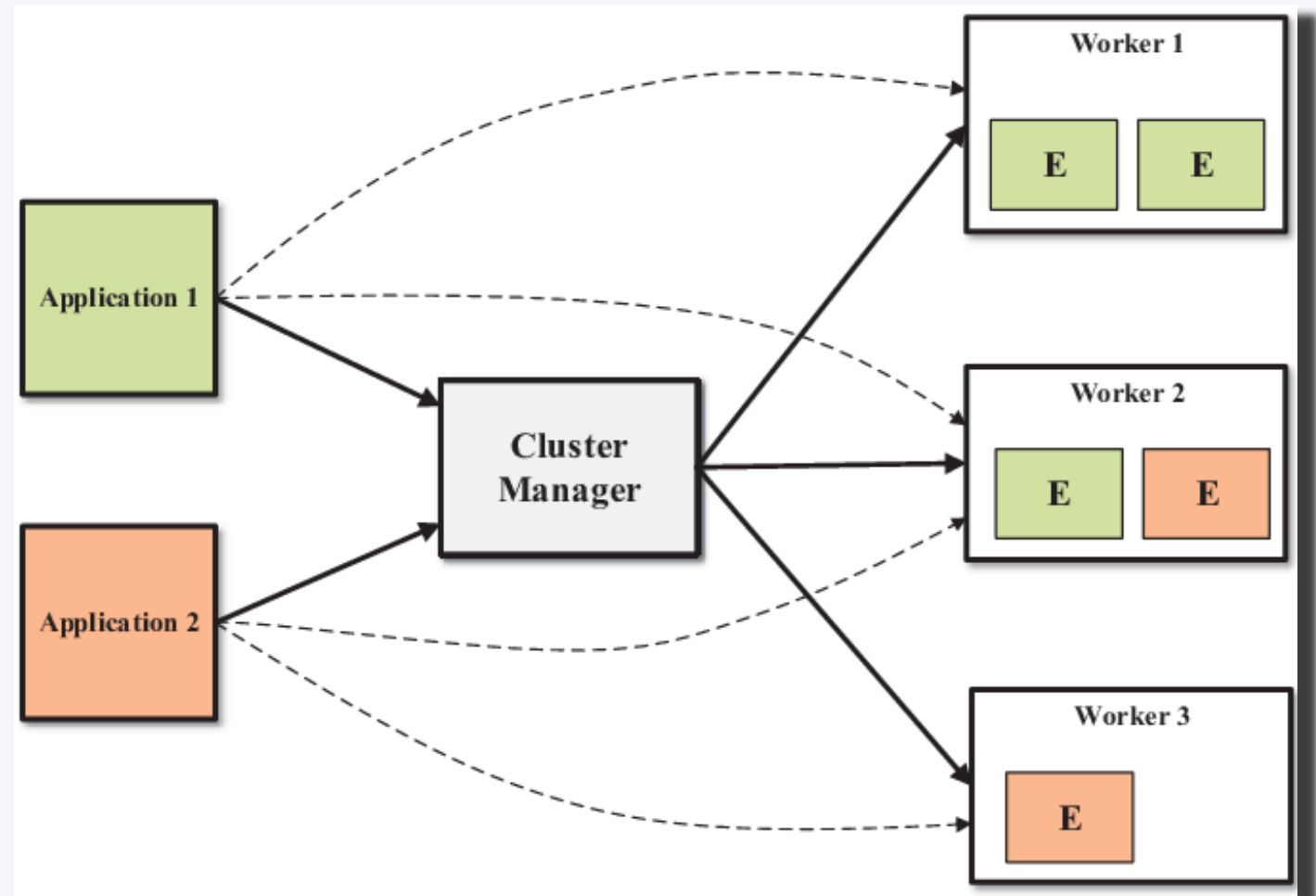


Single-Node Processing on Cloud



What Is Cluster Processing?

- Multiple machines working together
(distributed CPU, RAM, and storage)
- Coordinated by a scheduler
(driver → scheduler → workers)
- Designed for large-scale data
(huge datasets, massive throughput)



Typical Cluster Technologies

Most widely used distributed engines



Typical Cluster Technologies: *Distributed Frameworks*

These are the main distributed computing frameworks you'll see in real-world data engineering.



- **Strength:** General-purpose distributed processing
- **Key trait:** Excellent for SQL + batch at large scale
- **Limitation:** High startup overhead (slow for interactive work)



- **Strength:** Real-time, low-latency streaming
- **Key trait:** Best-in-class *stateful* stream processing
- **Limitation:** More complex to operate and tune

Typical Cluster Technologies: *Distributed Frameworks*

These are the main distributed computing frameworks you'll see in real-world data engineering.



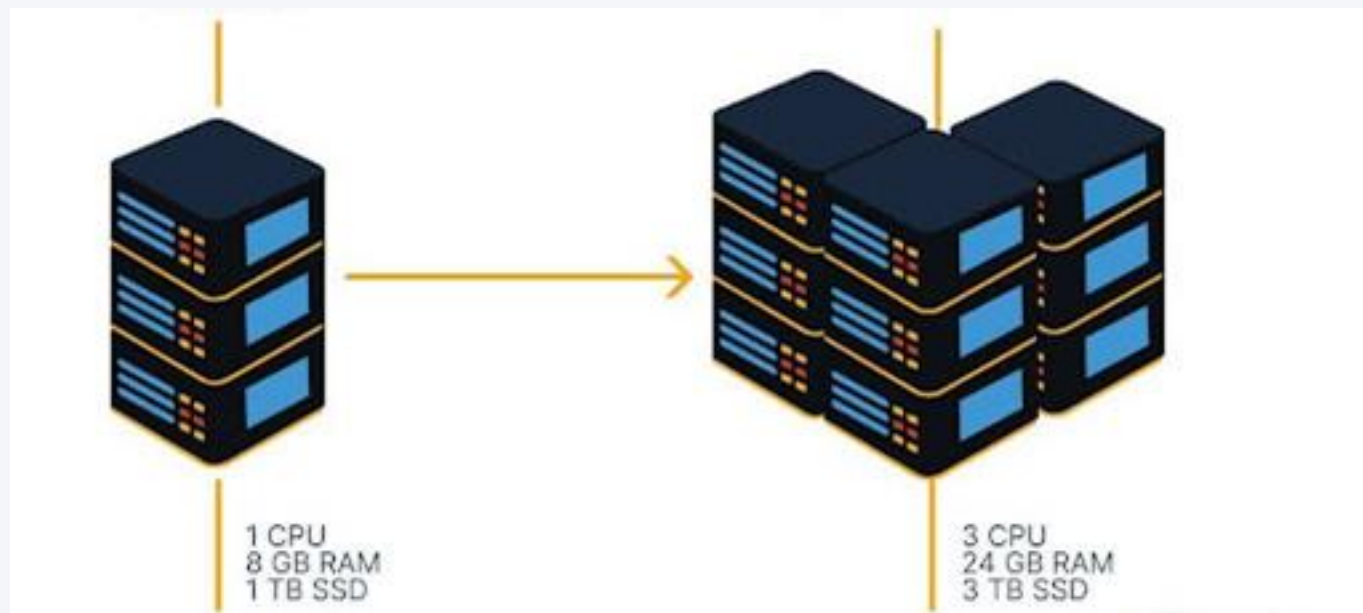
- **Strength:** Scales Python code easily
- **Key trait:** Lightweight, works naturally with NumPy/Pandas
- **Limitation:** Less robust for massive production clusters



- **Strength:** SQL interface for huge datasets on Hadoop
- **Key trait:** Widely used in enterprise environments for big data analytics
- **Limitation:** High latency; designed for batch, not real-time

When Do You Really Need a Cluster?

- Your data is too big for one machine
(hundreds of GB → TB → PB)
- You need fault tolerance
(jobs must survive machine failures)
- You require massive throughput
(24/7 pipelines, billions of rows/day)
- Real-time distributed streaming
(low-latency event processing → Flink, Kafka Streams)



Computing vs Data Loading — Bottleneck

When the bottleneck is Computing

- CPU is busy doing calculations
- Operations like joins, aggregations, sorting

Best choice: Single-Node tools (Polars, DuckDB)

→ They are optimized for raw speed

When the bottleneck is Data Loading

- Machine is waiting for data to arrive
- Slow disk / slow network / too many small files

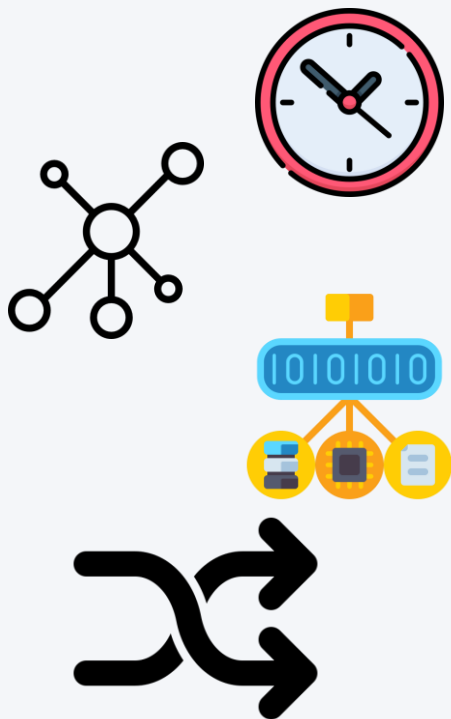
Best choice: Cluster systems (Spark, Flink, Hive, Dask)

→ They read data in parallel

The Hidden Cost of Clusters: Overhead

“Clusters are powerful — but you pay for coordination.”

Clusters introduce extra work that single-node systems don't:



- **Startup delay**
(clusters take time to spin up; single-node starts instantly)
- **Network communication**
(data moves between machines → slower)
- **Serialization / Deserialization**
(data must be packaged and unpacked many times)
- **Shuffle**
(expensive data movement during joins, groupbys, sorts)

Vertical vs Horizontal Scaling

Two Ways to Scale Your Workloads



Vertical Scaling (Scale Up)

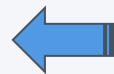


Upgrade one machine

- More CPU
- More RAM
- Faster NVMe
- Add a GPU

Best for:

- Single-node engines (Polars, DuckDB, RAPIDS)
- Medium to large datasets
- Low-latency workloads



Horizontal Scaling (Scale Out)



Add more machines

- More workers
- More parallel reads
- More distributed compute

Best for:

- Spark, Flink, Hive, Dask
- Very large datasets (TB–PB)
- High-throughput pipelines

How to Choose: Practical Decision Cases

Case 1 — Cleaning a 5–20 GB dataset

- Local file or Parquet dataset
- CPU-heavy transforms

Case 3 — Fast SQL analytics in a notebook

- Ad hoc queries
- Immediate results needed

Case 2 — Joining 300+ GB of data

- Input too big for RAM
- Heavy shuffles required

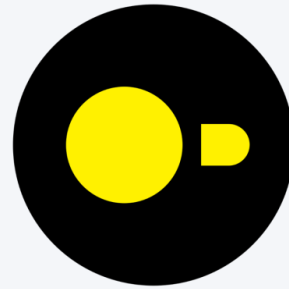
Case 4 — Real-time event processing

- Low latency
- Millions of events per minute

The Three Tools We Will Learn Today



Polaris



DuckDB



RAPIDS

Session 2 Tool: Polars



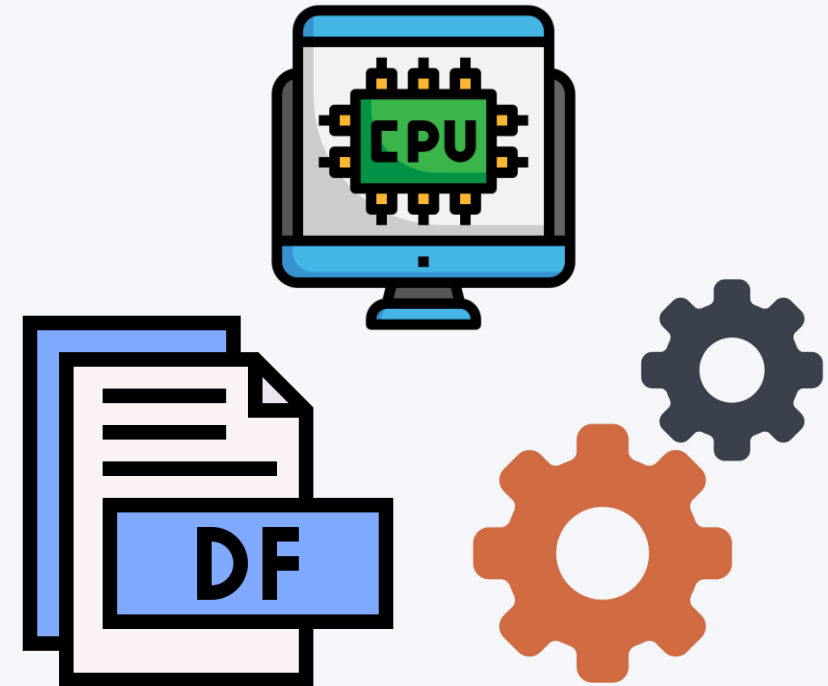
Polars

What is Polars?



Fast. Efficient. Modern.

- A modern, high-performance DataFrame engine
- Written in Rust
- Columnar and vectorized execution
- Automatically multithreaded
- Python API with Rust backend
- Ideal for medium-scale data processing



Why Polars? (Motivation & Pain Points)

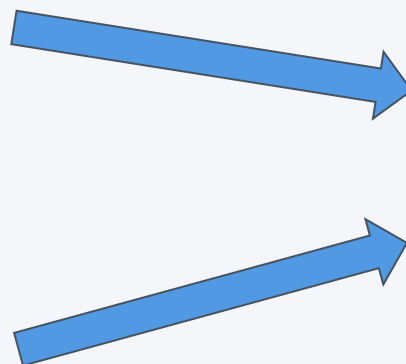
A response to the limitations of pandas and the overhead of Spark.

Pandas limitations:

- Single-threaded
- Slow on medium-sized datasets
- High memory usage
- Not built for modern hardware

Spark overhead:

- Heavy startup time
- Requires cluster infrastructure
- Overkill for many daily workloads

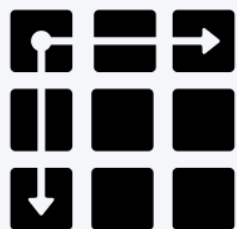


Polars solves this gap:

- Fast, efficient, single-node processing
- Designed for performance from the ground up
- Scales vertically with your machine

Polars Architecture (High-Level Overview)

Modern design built for modern hardware.



1. Columnar memory model

- Efficient data access
- Vectorized operations



3. Automatic multithreading

- Uses all available CPU cores
- Parallel execution by default



2. Rust engine

- Compiled, safe, and very fast
- Minimal overhead compared to Python



4. Lazy query optimizer

- Builds an optimized execution plan
- Pushdowns, predicate pruning, projection pruning

Lazy vs Eager Execution

Polars lets you choose the right mode for each task.



Eager Mode

- Executes operations immediately
- Similar to pandas
- Good for quick exploration



Lazy Mode

- Builds a query plan without executing
- Optimized before execution
- Uses projection pruning, predicate pushdown, and operator fusion
- Best for complex pipelines and large datasets

Expressions: The Core of Polars

Expressions are the language Polars uses to optimize and parallelize your transformations.

What are Expressions?

- Declarative transformations
- Represent “what” to compute, not “how”
- Fully vectorized and optimized

Why they matter

- Allow aggressive optimizations
- Enable lazy query planning
- Avoid intermediate DataFrames
- Make complex transformations concise and efficient



First Steps with Polars DataFrames

Theory becomes clearer once we see Polars in action.

- Create a DataFrame from a Python dict or list
- Inspect the data (shape, schema, head)
- Select one or more columns
- Apply a simple filter

```
import polars as pl

df = pl.DataFrame({
    "id": [1, 2, 3, 4],
    "value": [10, 20, 15, 5]
})

df.head()
df.select("id", "value")
df.filter(pl.col("value") > 10)
```

Basic Column Operations in Polars

Theory becomes clearer once we see Polars in action.

- Select one or more columns
- Filter rows based on conditions
- Create new columns with expressions
- All operations are vectorized and fast

```
df = pl.DataFrame({
    "id": [1, 2, 3, 4],
    "value": [10, 20, 15, 5]
})

df.select("value")
df.filter(pl.col("value") > 10)
df.with_columns((pl.col("value") * 2).alias("value_x2"))
```

Sorting and Describing Data

Theory becomes clearer once we see Polars in action.

- Sort by one or multiple columns
- Get unique values quickly
- Describe basic statistics

Useful for data exploration before building pipelines

```
df.sort("value")  
df.unique("id")  
df.describe()
```

GroupBy Basics

Theory becomes clearer once we see Polars in action.

- Group rows by one or more columns
- Apply aggregations (mean, sum, count, min, max, etc.)
- Fully vectorized and parallelized

Key operation for analytics and ETL

```
df.groupby("category").agg([
    pl.col("value").mean(),
    pl.col("value").sum()
])
```

Combining Operations (Pipelines)

Theory becomes clearer once we see Polars in action.

- Polars encourages chaining operations
- Creates clear, readable transformation steps
- Optimized when using lazy execution
- No intermediate DataFrames required

```
df = (  
    pl.DataFrame(data)  
    .filter(pl.col("value") > 10)  
    .with_columns((pl.col("value") * 2).alias("value_x2"))  
    .sort("value_x2")  
)
```

Reading Data: read_ vs scan_

Theory becomes clearer once we see Polars in action.

read_* functions (eager)

- Load the entire file into memory
- Operations run immediately
- Good for small to medium files
- Useful for exploration

```
df = pl.read_csv("data.csv")
```

scan_* functions (lazy)

- Do not read the file immediately
- Create a lazy plan pointing to the file
- Enable predicate pushdown + projection pruning
- Ideal for large files and complex pipelines

```
lazy_df = pl.scan_csv("data.csv")
```

Moving to Practice: The Polars Notebook

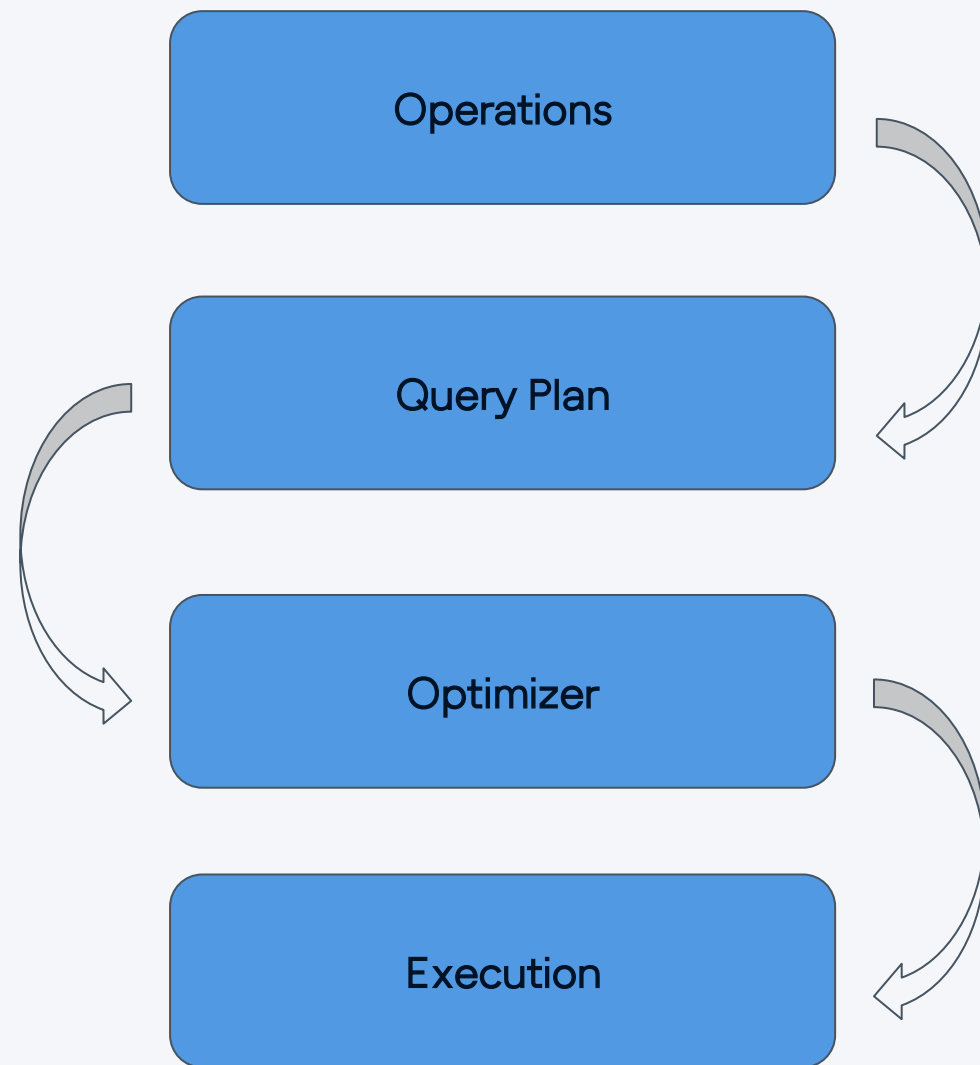
Polars Notebook: Hands-On Practice



Lazy Execution: Why It Matters

Lazy mode = faster pipelines, less memory, smarter execution.

- Builds a full query plan before execution
- Allows global optimizations
- Reduces I/O and memory usage
- Enables efficient processing of large datasets
- Only executes when `.collect()` is called



Lazy Execution: Why It Matters

```
lazy_df = (  
    pl.scan_csv("data.csv")  
        .filter(pl.col("value") > 10)  
        .with_columns((pl.col("value") * 2).alias("v2"))  
)  
  
result = lazy_df.collect()
```

Lazy Optimizations: Pushdown, Pruning & Fusion

Polars does less work, and finishes faster, by optimizing your entire pipeline before execution.

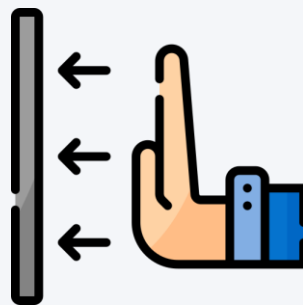
Projection Pruning

- Reads only the columns you actually use
- Avoids loading unnecessary data



Predicate Pushdown

- Applies filters *before* reading data
- Reduces the number of rows early



Operator Fusion

- Combines operations into a single pass
- Minimizes intermediate steps and memory usage



Lazy Optimizations: Pushdown, Pruning & Fusion

Projection Pruning

```
pl.scan_csv("big.csv").select("value")
```

Operator Fusion

```
(  
    pl.scan_csv("big.csv")  
        .with_columns(pl.col("a") * 2)  
        .with_columns((pl.col("a") + pl.col("b")).alias("sum"))  
        .filter(pl.col("sum") > 10)  
)
```

Predicate Pushdown

```
pl.scan_csv("big.csv")  
    .filter(pl.col("value") > 100)  
    .select("value")
```

Advanced Expressions in Polars

Polars does less work, and finishes faster, by optimizing your entire pipeline before execution.

Advanced expressions allow you to:

- Apply conditional logic to columns
- Transform string data efficiently
- Extract and manipulate date/time components
- Perform vectorized operations without Python loops
- Combine multiple expressions into optimized lazy pipelines



Advanced Expressions in Polars

Conditional expression

```
pl.when(pl.col("value") > 10)
    .then("high")
    .otherwise("low")
```

Date extraction

```
pl.col("date").dt.month()
```

String transformation

```
pl.col("name").str.to_uppercase()
```

Joins in Polars

Polars does less work, and finishes faster, by optimizing your entire pipeline before execution.

- **Inner join**
Returns rows that have matching keys in both tables.
- **Left join**
Returns all rows from the left table and matching rows from the right.
- **Outer join**
Returns all rows from both tables; unmatched values are filled with nulls.
- **Semi join**
Returns rows from the left table *only if they have a match* in the right table.
- **Anti join**
Returns rows from the left table *that do NOT have a match* in the right table.
- **Cross join**
Returns the cartesian product: all combinations of rows from both tables.

Joins in Polars

Inner join

```
df_left.join(df_right, on="id", how="inner")
```

Cross join

```
df_left.join(df_right, how="cross")
```


Window Functions in Polars

A window aggregates “across” rows while keeping the original row structure.

Window functions let you:

- Compute values *over groups or partitions* without collapsing rows
- Create running totals, rankings, moving averages
- Access previous/next values (lag/lead)
- Perform complex analytics while preserving row granularity
- Similar to SQL window functions (**OVER PARTITION BY**)



Window Functions in Polars

Running total per group

```
pl.col("value").cumsum().over("category")
```

Lag value

```
pl.col("value").shift(1).over("group")
```

Row number inside each group

```
pl.int_range(0, pl.count()).over("group")
```

Performance Tips & Best Practices in Polars

Polars is fastest when you let the optimizer do the work.

To get maximum performance from Polars:

- Prefer **lazy mode** for any non-trivial pipeline
 - Use **expressions**, never Python loops
 - Avoid **.apply()** unless absolutely necessary
 - Select only the columns you need
- Filter early — Polars pushes filters down automatically
 - Prefer **scan_***() over **read_***() for large files
 - Keep transformations in a single chained pipeline
 - Avoid converting back to pandas unless required

Performance Tips & Best Practices in Polars

Good — lazy optimized pipeline

```
(  
    pl.scan_parquet("data.parquet")  
        .filter(pl.col("value") > 10)  
        .with_columns((pl.col("value") * 2).alias("v2"))  
        .select(["id", "v2"])  
        .collect()  
)
```

Bad — eager + Python loops

```
df = pl.read_csv("data.csv")  
  
df["v2"] = df["value"].apply(lambda x: x * 2) # slow, Python loop
```

Full Polars Pipeline (Conceptual Overview)

Polars encourages building one optimized, end-to-end pipeline.

```
(
    pl.scan_parquet("transactions.parquet")
        .filter(pl.col("amount") > 0)
        .with_columns([
            pl.col("amount").log().alias("log_amount"),
            pl.col("timestamp").dt.date().alias("date")
        ])
        .join(products_lazy, on="product_id", how="left")
        .groupby("date")
        .agg(pl.col("log_amount").mean().alias("daily_mean"))
        .collect()
)
```

Moving to Practice: The Polars Notebook

Polars Notebook: Hands-On Practice



Case Study: Polars vs Pandas (Single-Node)

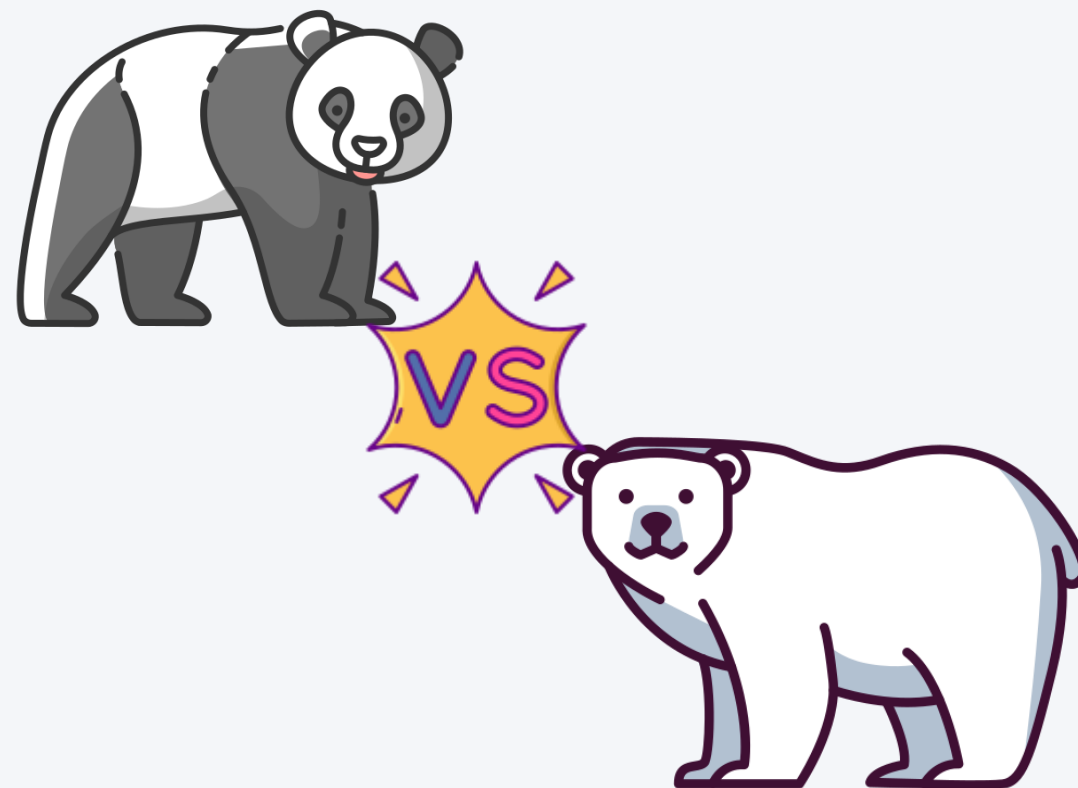
Polars scales on a single machine where pandas starts slowing down.

Polars outperforms pandas in:

- Multi-core execution (pandas is single-threaded)
- Large datasets (1M+ rows)
- Heavy transformations and groupby
- End-to-end pipelines with lazy execution
- Optimized file reading (Parquet/CSV)

Why?

- Rust engine (SIMD + parallelism)
- Columnar memory layout
- Query optimizer (lazy mode)



Case Study: Polars vs Spark (Single Node Context)

Spark wins at scale-out. Polars wins at scale-up.

Polars surpasses Spark in single-machine workloads when:

- The dataset fits in memory or NVMe
- You need low latency and fast iteration
- Startup overhead matters
- The job is mostly columnar analytics
- You don't need distributed fault tolerance

Spark strengths (not replaced):

- Multi-node scaling
- Fault tolerance
- Large distributed ETL
- Streaming pipelines (Structured Streaming)



Polars Summary (Section Wrap-Up)

Polars is what pandas would be if it were redesigned for modern hardware.

Polars gives you:

- A modern, fast DataFrame engine
- Columnar execution based on Arrow
- Native multithreading (Rust backend)
- Lazy query optimization
- Powerful expressions (no Python loops)
- Window functions, joins, and pipelines
- Excellent single-node performance
- Interoperability with Arrow, Parquet, DuckDB and more



Session 2 Tool: DuckDb



DuckDB

A Quick Recap: What is SQL and Why It Matters

SQL is the universal language of analytics — DuckDB speaks it natively.

Core concepts:

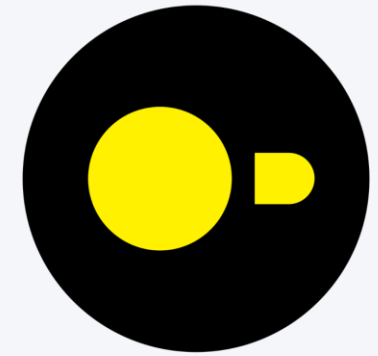
- SQL = *Structured Query Language*
- The standard language for data analysis
- Used to **filter**, **transform**, **join**, and **aggregate** data
- Declarative: *you say what you want, not how to do it*
- Every major analytics system uses SQL (BigQuery, Snowflake, Redshift, Spark SQL)

```
SELECT category, AVG(value)
FROM table
WHERE value > 10
GROUP BY category;
```

What is DuckDB?

DuckDB = a blazing-fast analytical SQL engine for single-machine workloads.

- Designed for **fast analytical queries**
- Columnar, **vectorized**, and highly **optimized**
- **SQL-first interface** (full SQL support)
- “**SQLite for Analytics**”: lightweight but extremely powerful
- Runs inside Python, R, CLI, or any application

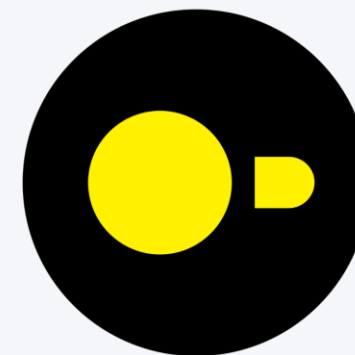


DuckDB

Why DuckDB Matters Today

Modern analytics don't always need a cluster.

- Zero configuration: no server, no cluster
- Extremely **fast analytics** on a single machine
- Reads Parquet/CSV at **high speed**
- Perfect for notebooks, **data exploration**, and local workflows
- **SQL-first engine** that feels like a local BigQuery/Snowflake
- Great for ETL, BI, **prototyping**, and ad-hoc analysis
- **Ideal** complement to Polars



DuckDB

Notebooks ▾

+

 DuckDB UI basics

Attached databases ▾

+

 memory

 main

 trains

 trains

381k rows







123 service_id



-

 date



-

T type

17

-

123 train_number



-

T station_code

573

-

T station_name

657

-

 departure_time



11%

 arrival_time



11%

 Run

 memory ▾

1

/**

2

Exploring Column Diagnostics

3

4

When running a query, your results will appear below the editor

5

and column diagnostics will appear on the right panel, providing an

6

overview of your data without requiring additional queries.

7

8

Run this query, then click on column names in the right panel to view detailed statistics.

9

*/

10

11

FROM trains;

Query results



service_id	date	type	train_number	station_code	station_name	departure_time
11,196,117	2023-05-15	Intercity	1,410	RTD	Rotterdam Centraal	2023-05-15 00:00:00
11,196,117	2023-05-15	Intercity	1,410	DT	Delft	2023-05-15 00:13:00
11,196,117	2023-05-15	Intercity	1,410	GV	Den Haag HS	2023-05-15 00:29:00
11,196,117	2023-05-15	Intercity	1,410	LEDN	Leiden Centraal	2023-05-15 00:45:00
11,196,117	2023-05-15	Intercity	1,410	SHL	Schiphol Airport	2023-05-15 01:03:00
11,196,117	2023-05-15	Intercity	1,410	ASD	Amsterdam Centraal	2023-05-15 01:19:00
11,196,117	2023-05-15	Intercity	1,410	UT	Utrecht Centraal	
11,196,118	2023-05-15	Nightjet	420	NURNB	Nürnberg Hbf	2023-05-15 00:01:00
11,196,118	2023-05-15	Nightjet	420	FFS	Frankfurt (Main) Süd	2023-05-15 01:47:00
11,196,118	2023-05-15	Nightjet	420	FFFM	Frankfurt (M) Hbf	
11,196,118	2023-05-15	Nightjet	420	FNAF	Frankfurt Flughafen Fernb	2023-05-15 01:59:00
11,196,118	2023-05-15	Nightjet	420	FMZ	Mainz Hbf	2023-05-15 02:18:00
11,196,118	2023-05-15	Nightjet	420	KKO	Koblenz Hbf	2023-05-15 03:13:00
11,196,118	2023-05-15	Nightjet	420	BONN	Bonn Hbf	2023-05-15 04:01:00
11,196,118	2023-05-15	Nightjet	420	KKW	Köln West	

Filter

380,959 Rows





▾

 Run

 memory ▾

1

/**

2

Working with Files

3

4

The DuckDB UI can read and write files just like in other clients.

5

This example demonstrates exporting data to a CSV file and then

6

querying that same file.

Current Cell ▾

380,959 Rows

8 Columns





Default ▾

123 service_id



-

 date



-

T type

17

-

Sprinter

167k

44%

Stoptrein

91.7k

24%

Intercity

83.3k

22%

Sneltrain

11.6k

3%

Stopbus i.p.v. trein

7.90k

2%

stoptrein

7.74k

2%

Intercity direct

4.71k

1%

Snelbus i.p.v. trein

3.78k

1.0%

Thalys

1.08k

0.3%

ICE International

915

0.2%

Other

1.17k

0.3%

123 train_number



-

T station_code

573

-

T station_name

657

-

 departure_time

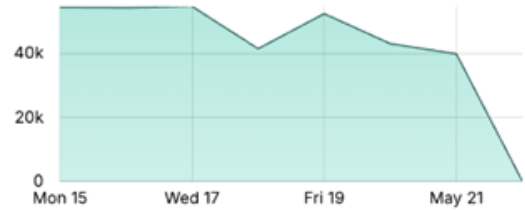


11%

 arrival_time



11%



about 7 days

bucketed by day

earliest

2023-05-15

00:13:00.000

latest

2023-05-22

07:14:00.000

low bucket

40

2023-05-22

00:00:00.000

high bucket

54.7k

2023-05-17

00:00:00.000

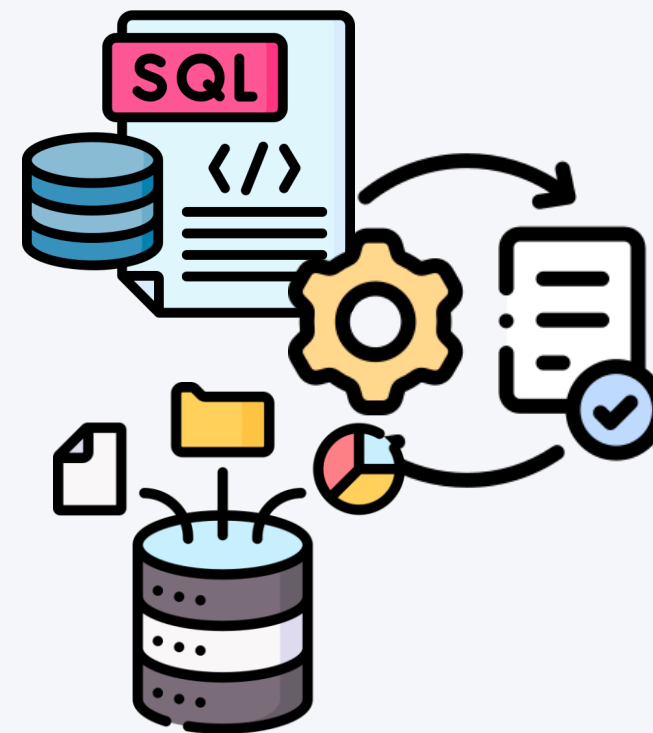
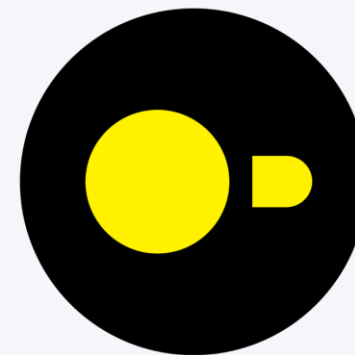
DuckDB Basics (SQL-First Analytics)

If you know SQL, you already know how to use

DuckDB.

Core ideas:

- DuckDB uses SQL as its primary interface
- You can query files directly
`SELECT * FROM 'file.parquet';`
- No need to load data into a DataFrame
- All transformations use familiar SQL
- Results can return as pandas, Polars, Arrow, CSV or Parquet
- Runs entirely **in-process** (inside Python, R, CLI, or apps)



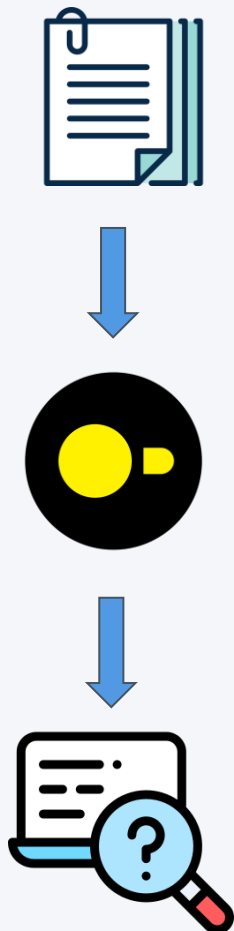
Moving to Practice: The DuckDB Notebook

Polars Notebook: Hands-On Practice



Reading Data: The DuckDB Superpower

DuckDB reads data where it is — without moving it.



Core points:

- Query files **directly**
`SELECT * FROM 'file.parquet';`
- No DataFrame loading → no memory pressure
- Extremely **fast scanning of Parquet** (and fast CSV)
- Reads only **required columns** (projection pushdown)
- **Applies filters at read time** (predicate pushdown)
- Can query **folders** of files as one dataset
- Ideal for large **local datasets** (GB-scale)

DuckDB + Python Integration

SQL on top of DataFrames — instantly.

```
import duckdb
import pandas as pd

df = pd.read_csv("data.csv")

duckdb.sql("""
    SELECT category, AVG(value)
    FROM df
    GROUP BY category
""").df()
```

- Query pandas DataFrames directly (**FROM df**)
- No copies → Arrow zero-copy conversion
- Works the same with Polars
- Returns DataFrames or files

SQL Expressions in DuckDB

Numeric & Conditional

- Arithmetic operations
- **CASE WHEN** logic
- Comparisons (**>**, **<**, **BETWEEN**)
- Type conversions (**CAST**, **TRY_CAST**)

```
LOWER(REPLACE(name, ' ', '_'))
```

```
CASE WHEN value > 100 THEN 'high' ELSE 'low' END
```

Date & Time Functions

- Extract fields: **YEAR**, **MONTH**, **DAY**, **WEEK**
- Conversions: **DATE()**, **TIMESTAMP()**, **CURRENT_DATE**
- Arithmetic: **date + INTERVAL '7 days'**

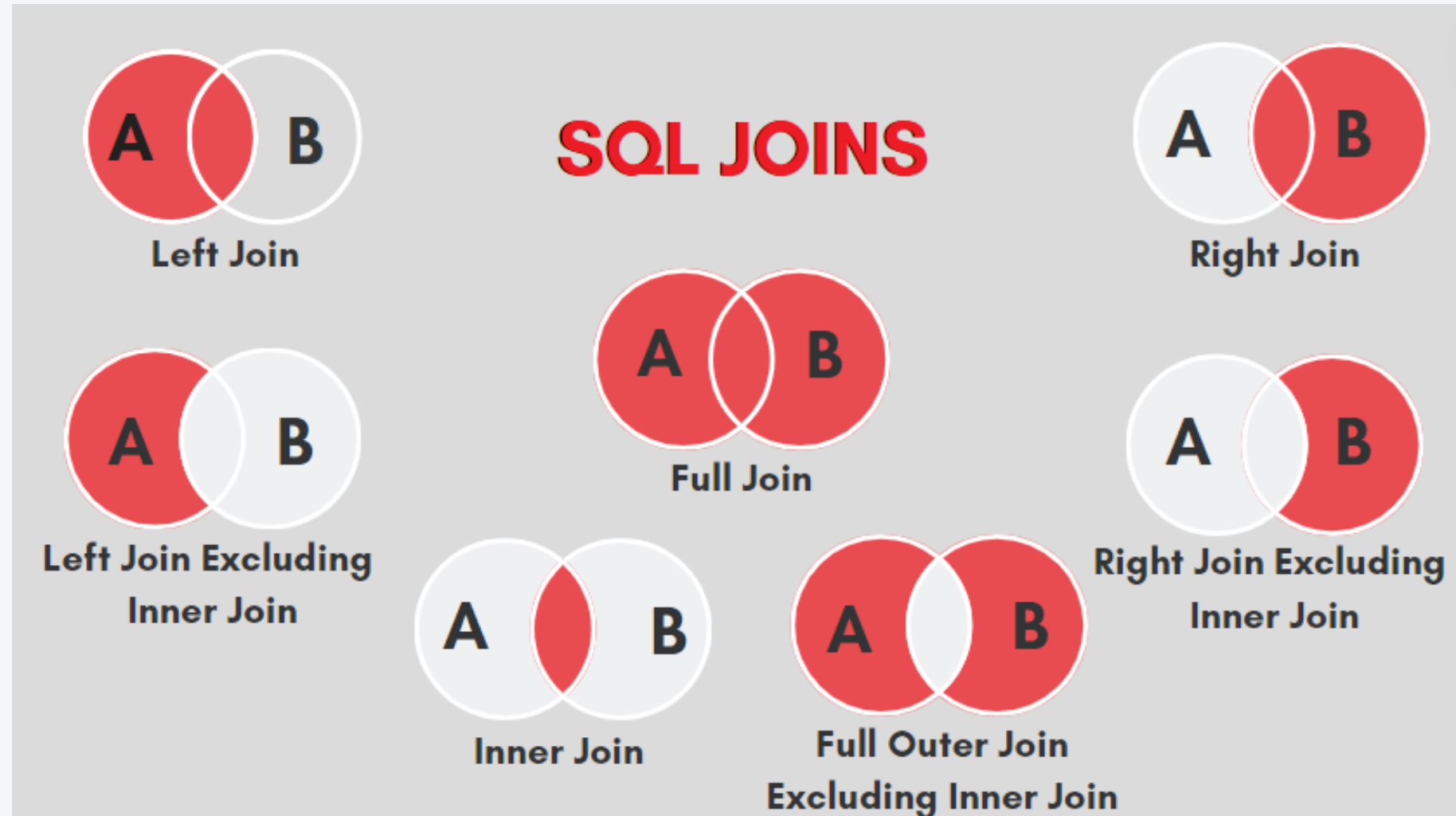
```
DATE(timestamp)
```

String Functions

- **LOWER**, **UPPER**
- **REPLACE**, **SUBSTR**, **CONCAT**
- **REGEXP_MATCHES** (regex support)
- Trimming and cleaning

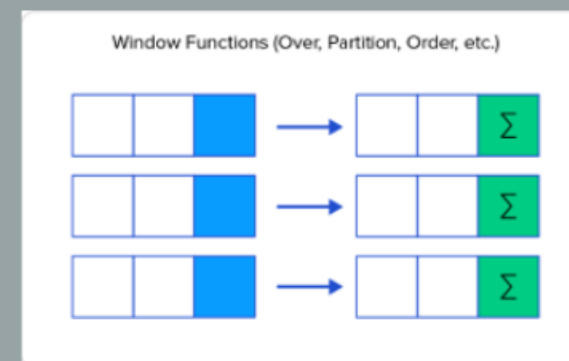
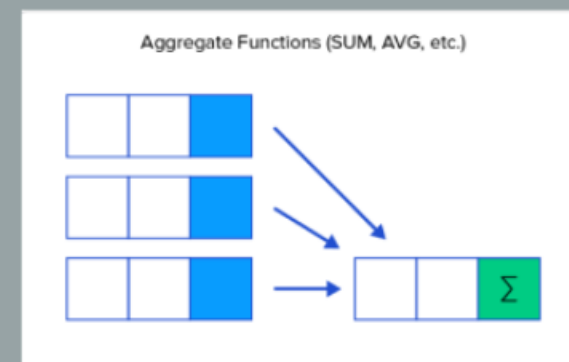
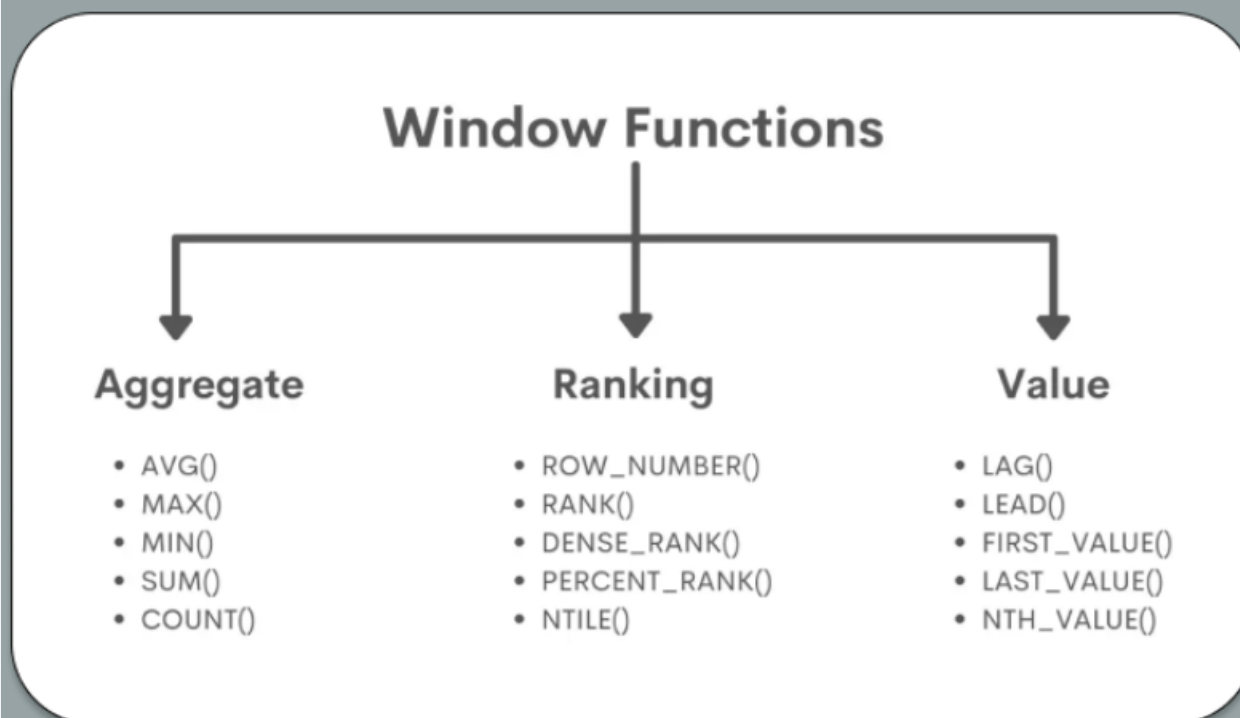
Joins in DuckDB

Supported joins
(same as Polars)



Window Functions in DuckDB

Everything You need to know about Window Functions in SQL



Why DuckDB Is Fast (Query Optimization)

Columnar execution



Vectorized processing



Smart query planning

Parquet-native scanning



Pushdown optimizations



Performance Tips for DuckDB

Small choices → massive performance differences.

Tip 1 — Prefer Parquet over CSV

Parquet is columnar → faster scans, smaller size, better pushdown.

Tip 2 — Query files directly

Avoid loading into pandas; let DuckDB scan the source.

Tip 3 — Use **LIMIT** when exploring

Reduce unnecessary work when profiling data.

Tip 4 — Create temporary views for multi-step analysis

Clearer SQL pipelines, re-usable transformations.

Tip 5 — Let DuckDB handle large joins

SQL join engine is highly optimized → avoid doing joins in pandas.

Tip 6 — Query folders, not just files

DuckDB can scan directory patterns (**folder/*.parquet**) efficiently.

Moving to Practice: The DuckDB Notebook

Polars Notebook: Hands-On Practice



DuckDB vs Spark (Single-Node Context)

Different Tools for Different Scales



DuckDB (single-node OLAP engine)

- **Instant** startup (no cluster, no JVM)
- Extremely **fast** for **local** analytics
- SQL-first, **low-latency** interactive work
- Perfect for **datasets** that fit in RAM/NVMe
- Ideal for **notebooks**, BI, **prototyping**, ETL on one machine



Spark (distributed compute engine)

- Scales **across many machines**
- Handles **terabytes** of data
- **Fault-tolerant**, resilient distributed execution
- Higher **latency**, larger **overhead**
- Ideal for **batch pipelines**, production ETL, ML at scale

DuckDB vs Polars (Complementary Tools)

Not Competitors, but Complements

Polars

- DataFrame engine (Python-first)
- Expression-based API
- Extremely fast transformations
- Great for pipeline-style ETL
- Uses Rust + Arrow under the hood

DuckDB

- SQL engine (SQL-first)
- Perfect for aggregations, joins, analytics
- Reads Parquet/CSV directly
- Ideal for complex SQL logic and BI tasks
- Embedded, zero-configuration OLAP engine

How they fit together

- Use **Polars** for fast in-memory transformations
- Use **DuckDB** for SQL-heavy analytics or multi-join queries
- Both share columnar execution concepts
- Both thrive in **single-node** scenarios

Full SQL Pipeline Overview

1. Scan

- Read Parquet/CSV
- Query folders/partitions
- Early filters & column pruning

2. Transform

- SQL expressions
- Clean & normalize fields
- Create derived columns

3. Join

- Lookup/dimension tables
- Combine datasets
- Semi/Anti joins for data quality

6. Output

- Export to Parquet/CSV
- Return as pandas/Polars
- Save tables or views

5. Aggregate

- Group-level metrics
- KPIs & trends
- Consolidate business logic

4. Windows

- Cumulative metrics
- Lag/Lead comparisons
- Time-based windows

DuckDB Summary

Key Takeaways

What DuckDB gives you

- SQL-first analytics on a single machine
- Extremely fast Parquet/CSV scanning
- Powerful joins, windows, and complex queries
- Vectorized, columnar execution (like BigQuery/Snowflake)
- Perfect for notebooks, prototyping, BI, and local ETL
- Seamless integration with **pandas**, **Polars**, **Arrow**

When to use it

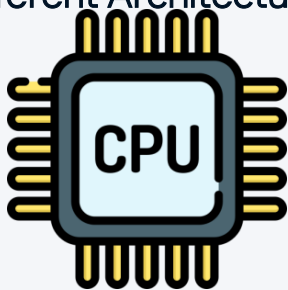
- You want fast analytics **without a cluster**
- You have up to a few GBs—tens of GBs locally
- Your workflow is SQL-heavy
- You iterate quickly and need low latency

Session 2 Tool: Nvidia Rapids (GPU Processing)



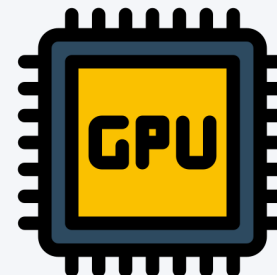
CPU vs GPU: How They Really Differ

CPU vs GPU: Different Architectures, Different Strengths



CPU (Central Processing Unit)

- Few, powerful cores
- Great for complex logic & branching
- Designed for general-purpose workloads



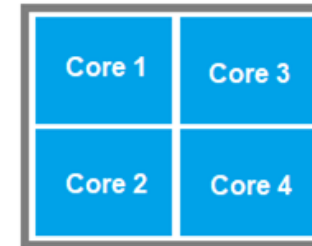
GPU (Graphics Processing Unit)

- Thousands of lightweight cores
- Massive parallel execution
- Designed for vectorized, repetitive workloads

What Is GPU Computing?

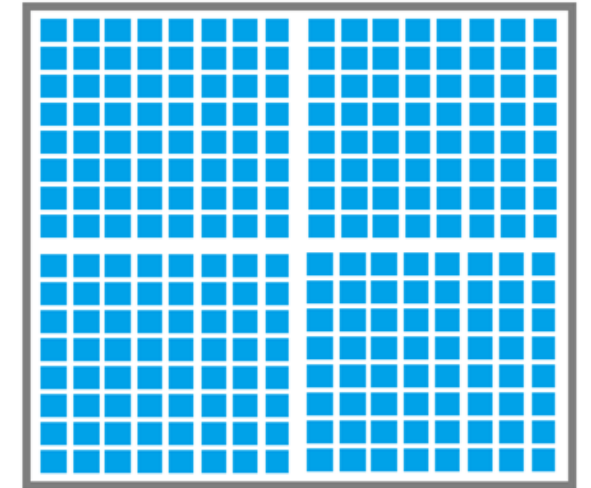
“GPUs = extreme parallelism, not general-purpose logic.”

- GPUs have thousands of lightweight cores
- Designed for massive parallel execution
- Excellent for **vectorized**, columnar workloads
- Optimized for arithmetic, not control-flow
- RAPIDS uses **CUDA** to accelerate DataFrames



(Fewer strong cores)

CPU



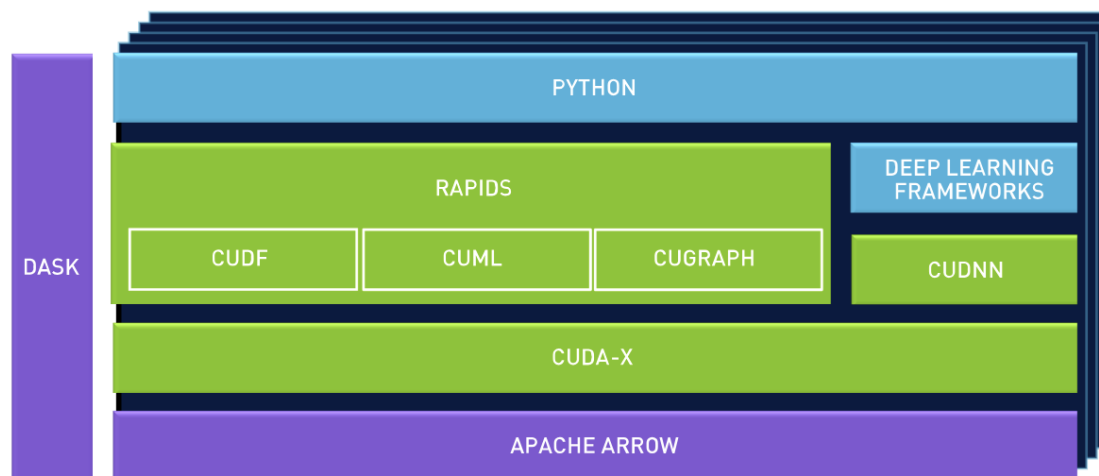
(Thousands weaker cores)

GPU

RAPIDS Ecosystem Overview

The RAPIDS Ecosystem

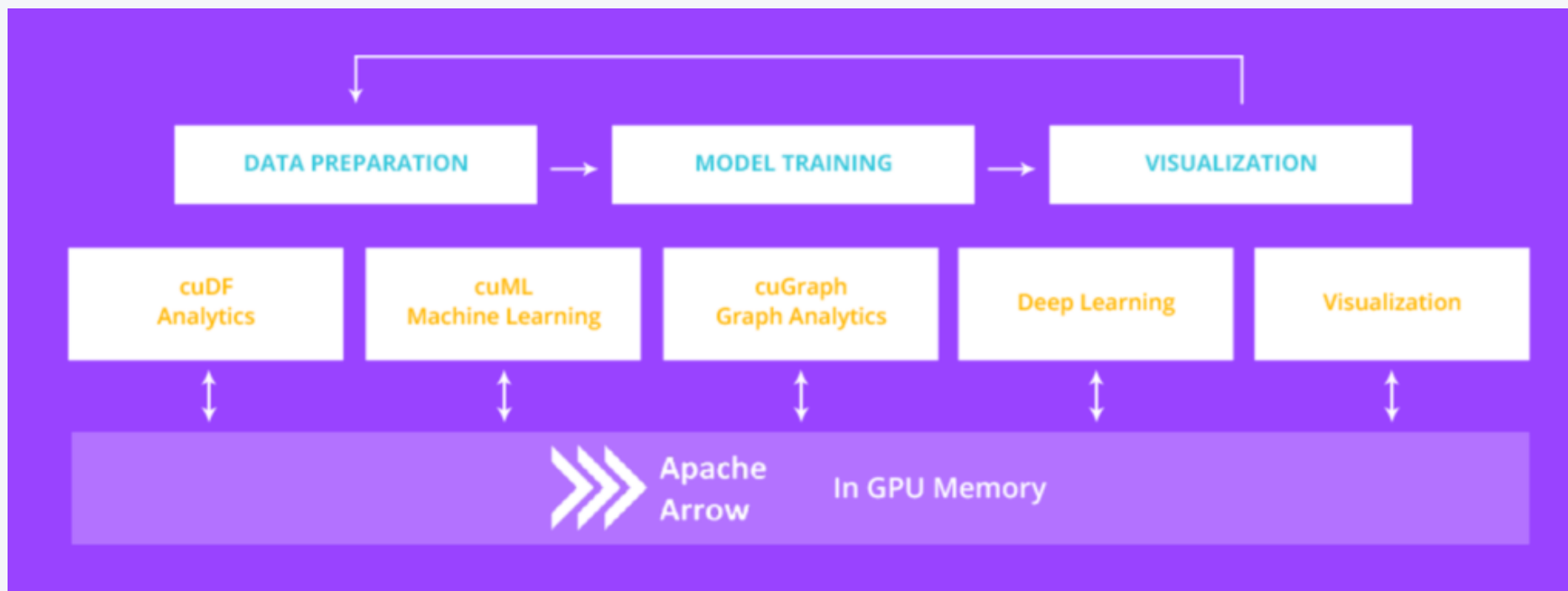
- **cuDF**: GPU DataFrames (pandas-like)
- **CuPy**: NumPy accelerated on GPU
- **cuML**: Machine learning algorithms on GPU
- **cuGraph**: Graph analytics on GPU
- **RMM**: GPU memory manager
- **Dask-cuDF**: Distributed multi-GPU DataFrames



How RAPIDS Works

The RAPIDS Ecosystem

- Stores data in a **columnar format** (like Parquet/Arrow)
- Uses **massive parallelism** for filters, joins, and groupbys
- Runs operations directly on the GPU
- Designed as a “GPU version of pandas”



What Is CUDA?

CUDA makes general-purpose GPU computing possible.

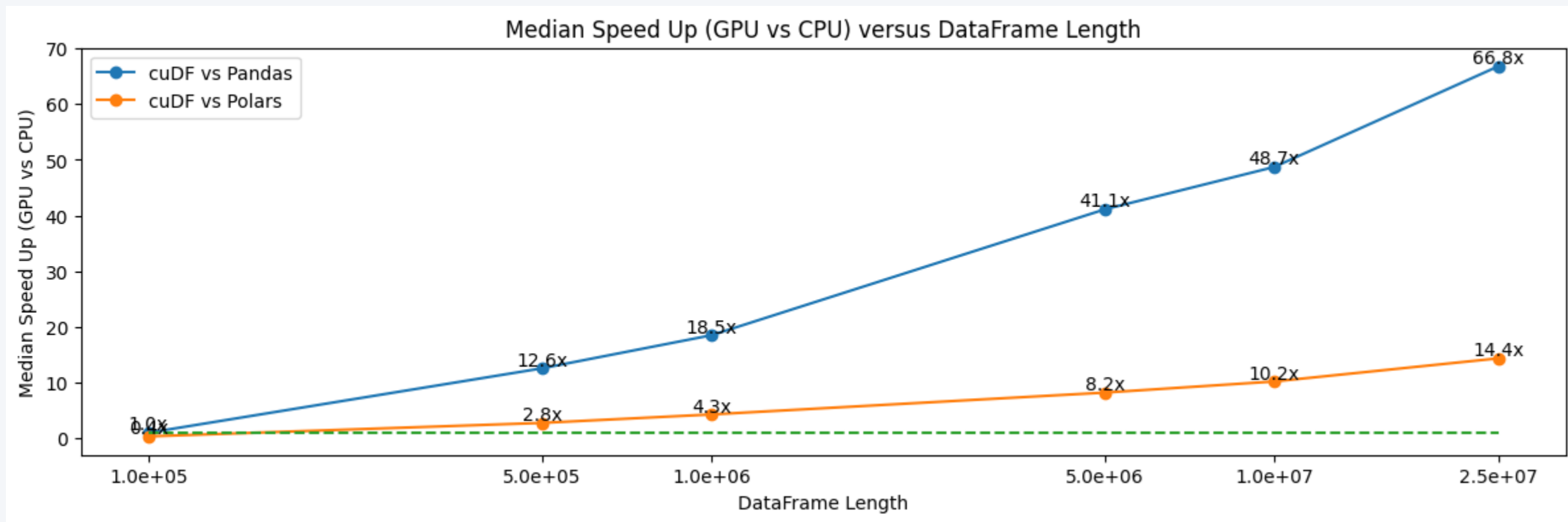
- NVIDIA's platform for running code on the GPU
- Allows thousands of operations to run in parallel
- Powers cuDF, cuML, cuGraph (RAPIDS)
- You don't write CUDA — RAPIDS does it for you



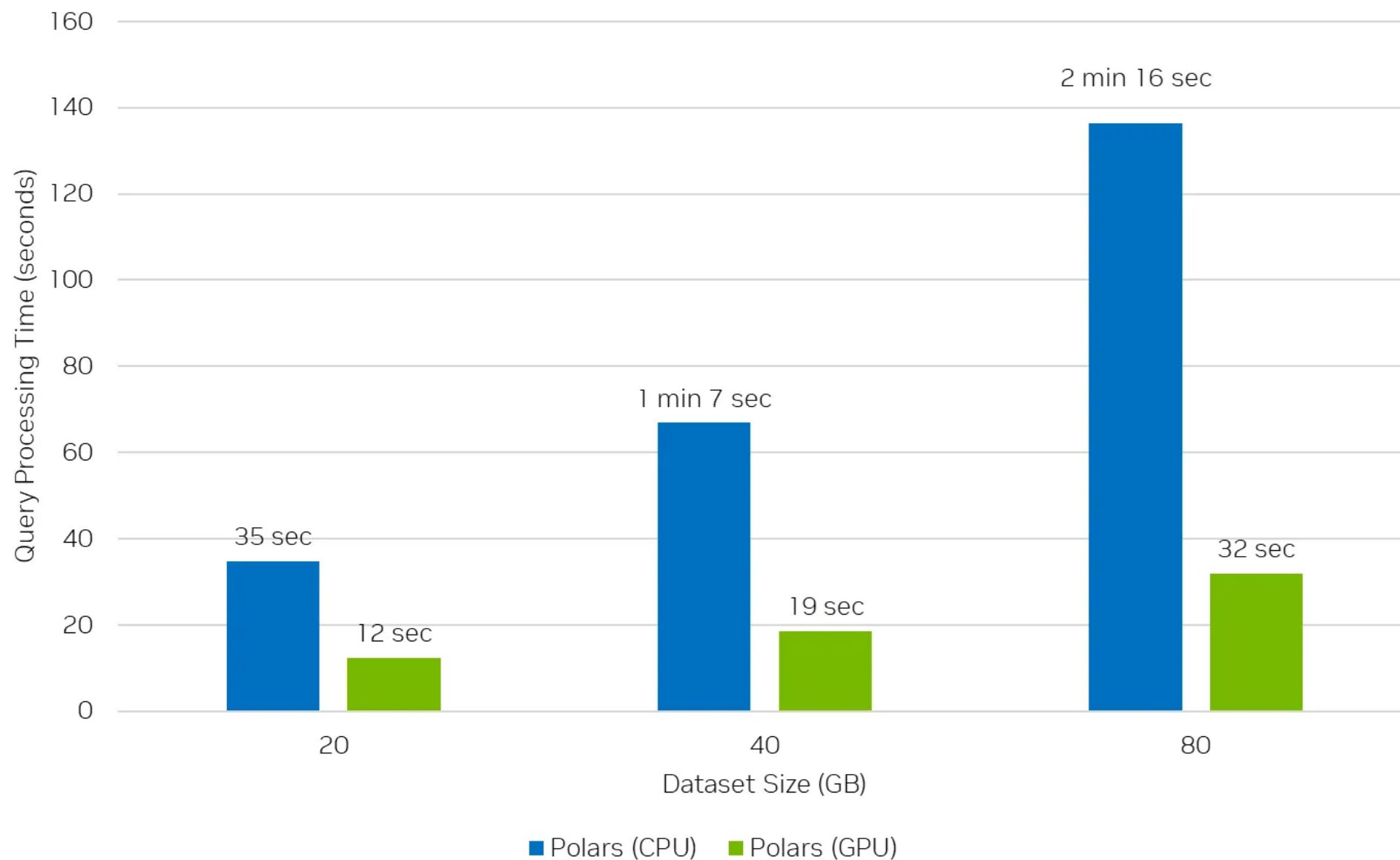
cuDF vs pandas vs Polars vs DuckDB

Task / Scenario	Best Tool
Small/medium ETL	Polars
Fast, SQL-style analytics	DuckDB
Heavy numerical workloads on GPU	cuDF (RAPIDS)
Feature engineering for ML	cuDF + cuML
Interactive prototyping	Polars / DuckDB
Large joins or groupbys on numeric data	cuDF

cuDF vs pandas vs Polars



Performance improves as data size grows



Advantages of GPU Processing

“GPUs shine when the workload is large and parallel.”

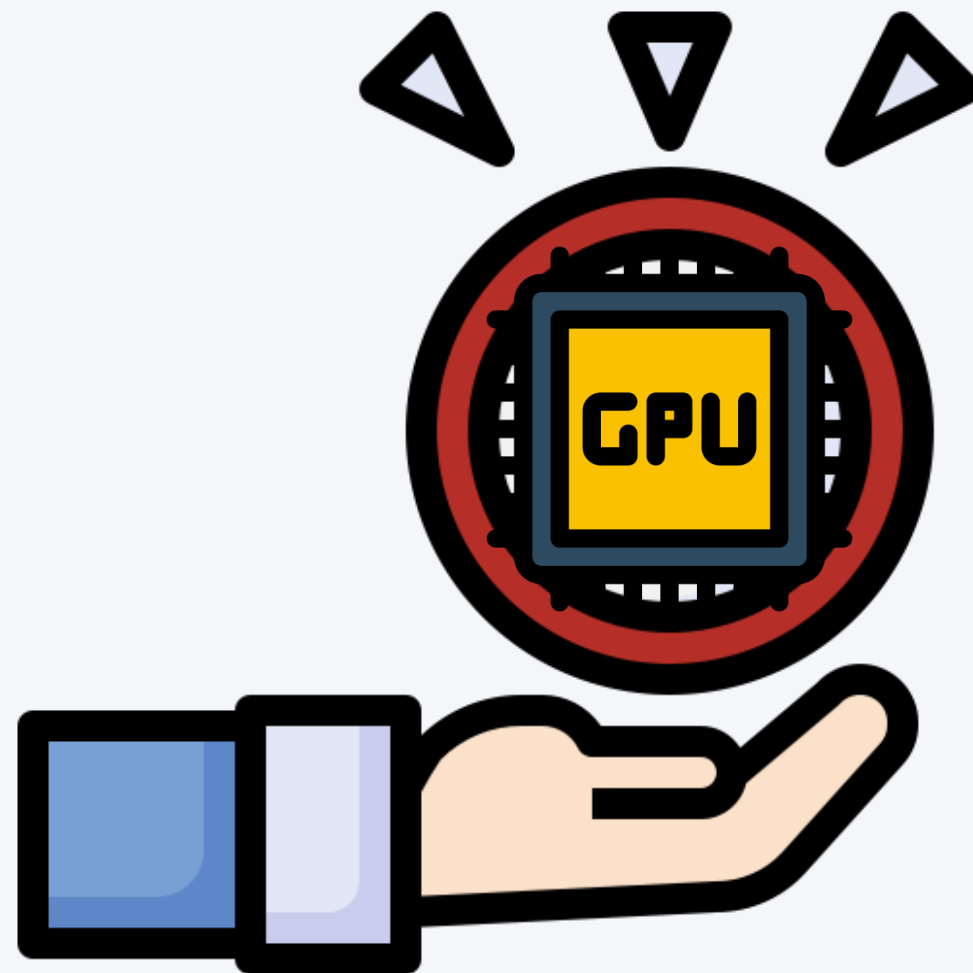
- Massive **parallelism** (thousands of cores)
- Extremely **fast** for **columnar** workloads
- Ideal for filters, joins, groupbys, ML features
- Great for **large** numeric datasets
- Can **outperform** CPU clusters on a single machine
- **Accelerates** end-to-end pipelines when all stays on GPU



Limitations of GPU Processing

When GPUs Are NOT the Best Choice

- GPU memory is **limited**
- Data **transfer** CPU ↔ GPU is **slow**
- **Not** good for Python **loops** or branching logic
- **Small datasets** do not benefit
- Requires numeric or columnar workloads
- CPU may be **faster** for complex, sequential tasks



What Makes RAPIDS Different

“Same code. New engine.”

- Same DataFrame workflow as pandas
- Executes all operations **on the GPU**
- Massive **parallelism** for large datasets
- Supports **full pipelines**: ETL + ML on GPU
- Accelerates workloads too large or slow for CPU
- Minimal learning curve



Same Code, Different Engine

“Same code. New engine.”

```
import pandas as pd

df = pd.read_csv("data.csv")
df["value_x2"] = df["value"] * 2
result = df.groupby("city")["value_x2"].mean()
```

```
import cudf

df = cudf.read_csv("data.csv")
df["value_x2"] = df["value"] * 2
result = df.groupby("city")["value_x2"].mean()
```

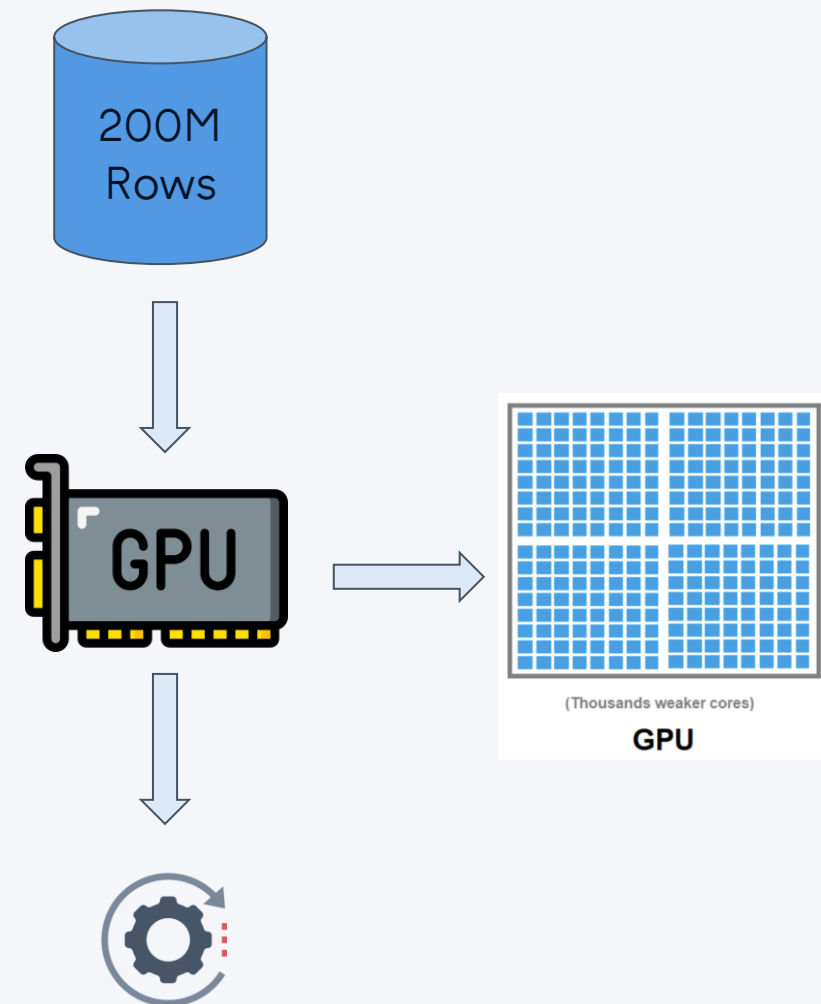
What GPUs Accelerate Best

If the work is “apply this to all rows”, the GPU is extremely fast.

Where GPUs Shine

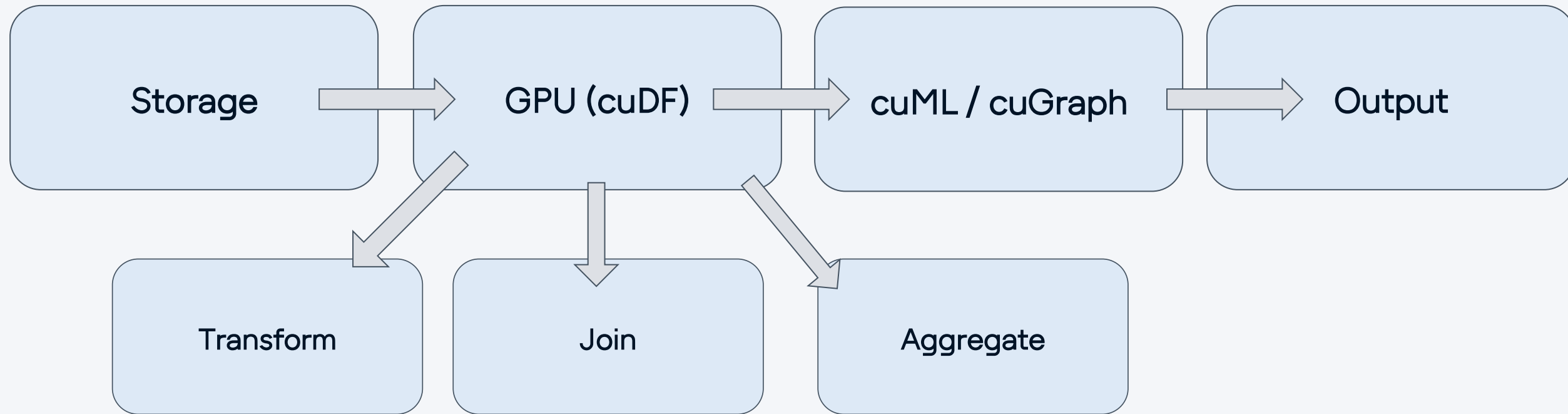
- Massive column operations (millions of rows)
- Numeric transformations at scale
- Large filters and masks
- Feature engineering
- Large groupbys & aggregations
- Rolling / cumulative windows
- Time-series analytics

“Think column-by-column, not row-by-row.”



End-to-End Pipeline on GPU

The real acceleration happens when you avoid CPU↔GPU transfers.



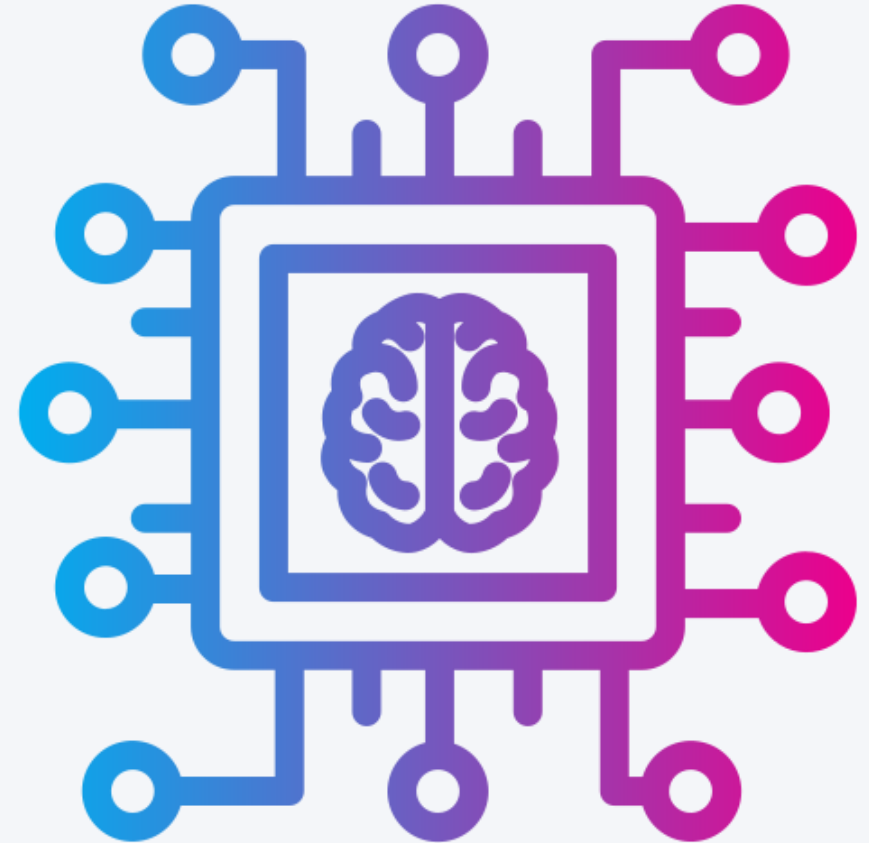
“Keep everything on the GPU. That's where the magic is.”

cuML Sneak Peek

ML becomes part of the same GPU pipeline.

cuML: ML on the GPU

- Same API as scikit-learn
- 10–50x faster on large datasets
- Works directly with cuDF
- Great for classical ML:
 - RandomForest
 - KMeans
 - Logistic Regression

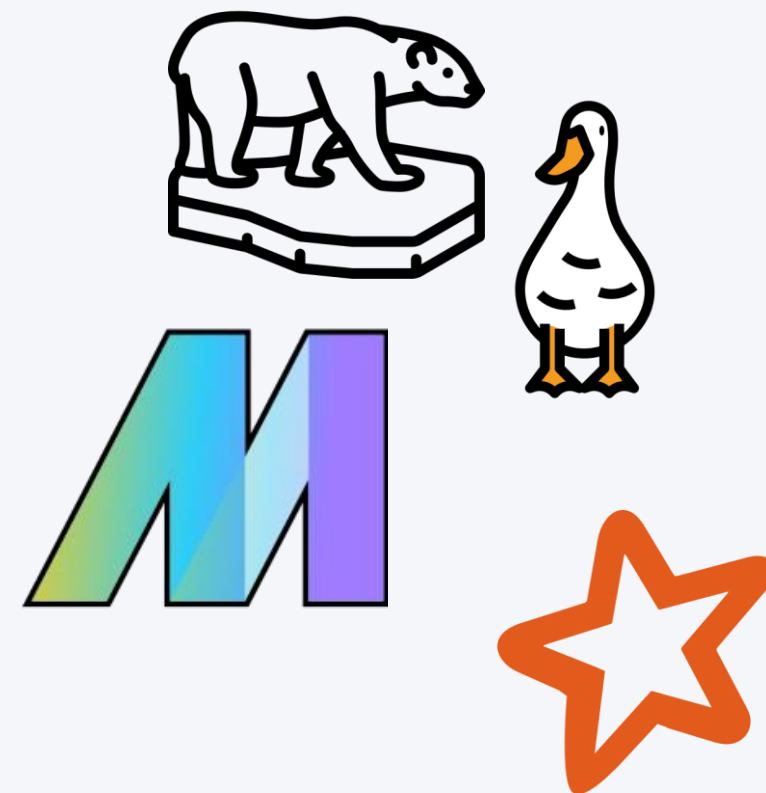


The Future of GPU Analytics

GPU analytics is moving from “specialized” to “default”.

Future of GPU Analytics

- Polars GPU engine (powered by cuDF)
- DuckDB exploring GPU backends
- Spark RAPIDS plugin (GPU acceleration for big clusters)
- Multi-GPU analytics becoming standard
- AI-assisted optimization of ETL pipelines



“The entire data stack is moving toward GPU acceleration.”

Moving to Practice: The GPU Notebook

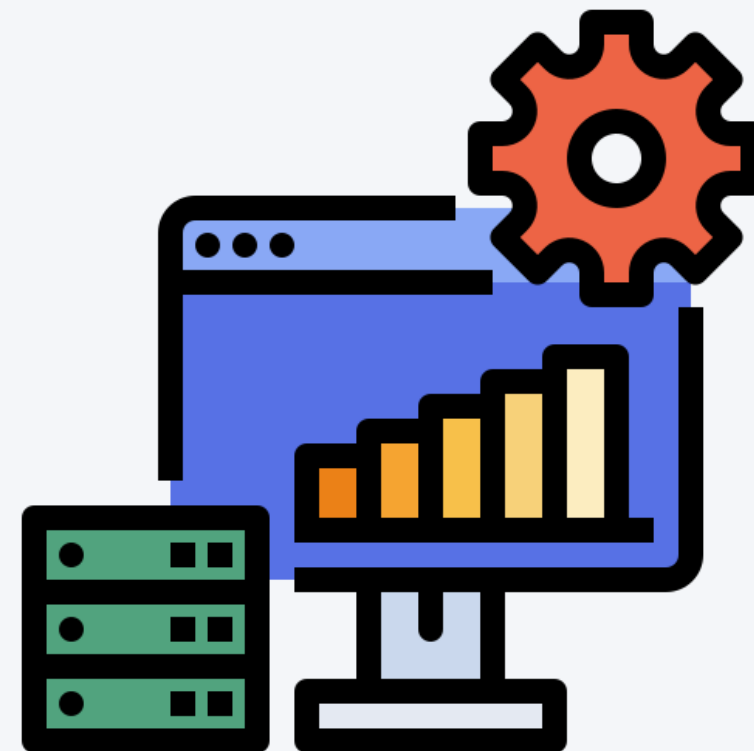
Polars Notebook: Hands-On Practice



Summary Slide

Takeaways

- GPU acceleration is ideal for large columnar workloads
- RAPIDS uses a pandas-like API
- End-to-end pipelines avoid costly transfers
- Huge speedups with minimal code changes
- The ecosystem is moving toward GPU-native processing



“The entire data stack is moving toward GPU acceleration.”

Tool Comparison — What to Use When

Different tools excel in different workloads.



Polars

Fast ETL on CPU, medium-sized datasets, expressive transformations



RAPIDS

Huge datasets, heavy numeric pipelines, GPU acceleration



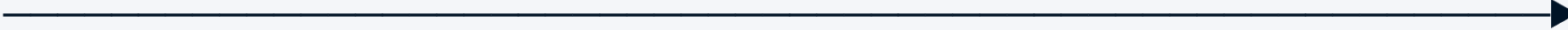
DuckDB

Local SQL analytics, joins & windows, BI-style workloads

“Choose the engine based on your workload, not your preference.”

Tool Comparison — How Each Engine Scales

How Each Engine Scales

Dataset Size 

- Polars 

| CPU fast, great until medium size

- DuckDB 

| SQL optimized, scales extremely well analytically

- RAPIDS 

| GPU massive-scale, excels for huge datasets

Real-World Tradeoffs in Industry

Cost

- CPU vs GPU cost
- Cluster vs single-machine cost
- Storage formats impact (CSV vs Parquet)

Maintainability

- Complexity of the stack
- Debugging difficulty
- Team skill requirements

Resources Needed

- Hardware availability (RAM, NVMe, GPU)
- Constraints in cloud/on-prem environments
- Operational overhead

Learning Curve

- Polars → very low
- DuckDB → very low
- RAPIDS → low (if you know pandas)
- Spark → higher

Polars + DuckDB as a “Mini Data Platform”



- Fast, cheap, simple
- Zero infrastructure
- High productivity

ETL / Processing



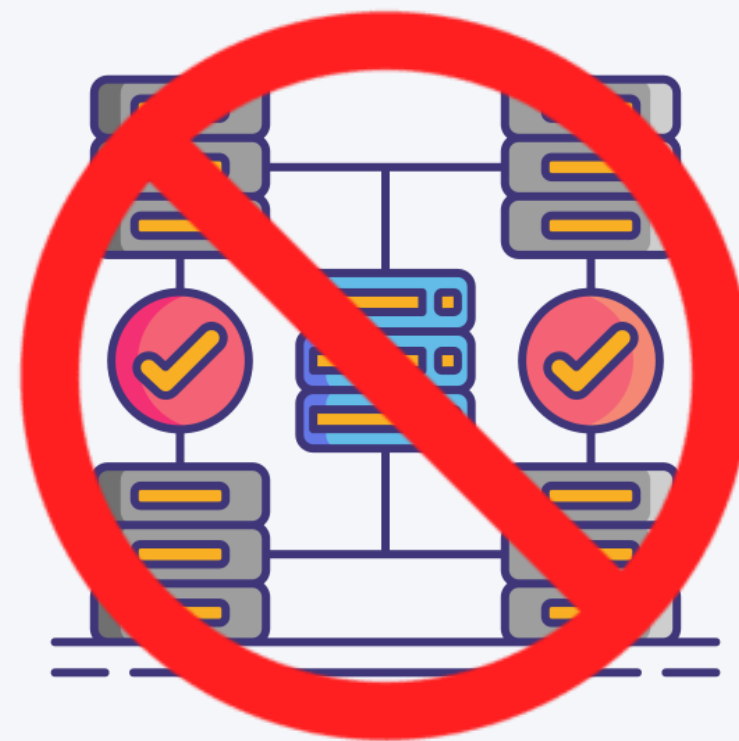
SQL / Analytics



Parquet / Csv / Apis

When NOT to Use a Cluster

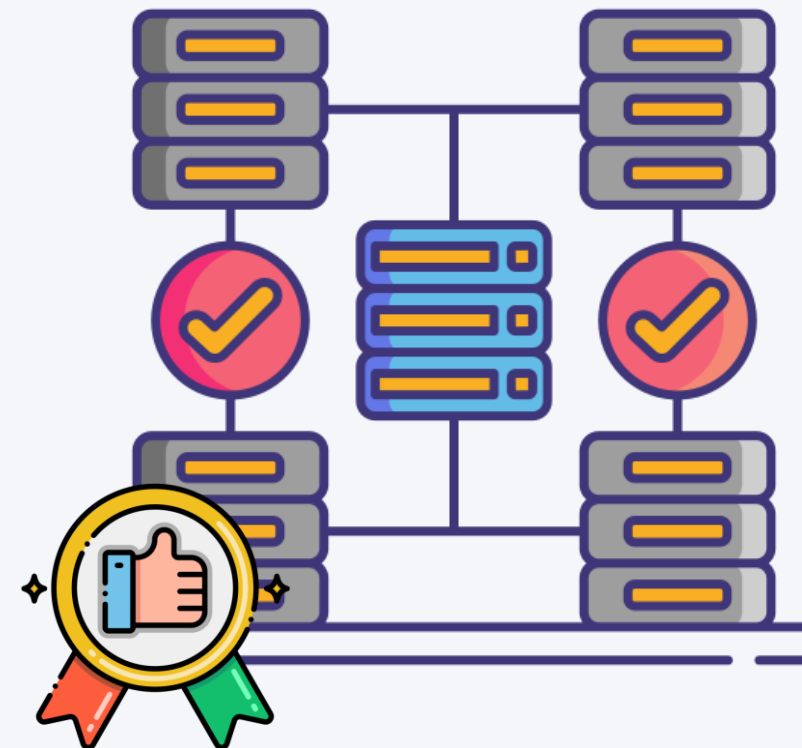
- Your data fits in a **single machine** (RAM + NVMe)
- You **don't need fault tolerance** or distributed retries
- You want **fast iteration** / interactive development
- The pipeline runs in **seconds** or minutes, not hours
- You're **exploring, prototyping, or debugging**
- **Scaling horizontally** adds more **overhead** than benefit



When You Actually Need Spark (Cluster)

"You need a cluster when the problem is fundamentally distributed."

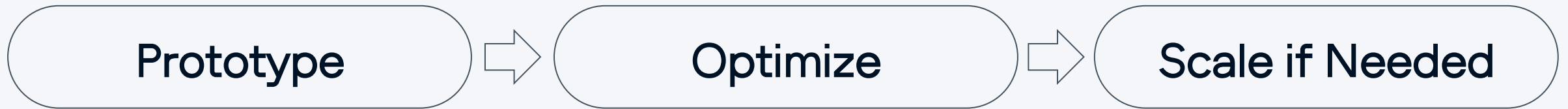
- Too much **data** for one machine
- **Long**-running jobs (hours/days)
- **Distributed** storage as **source**
- Enterprise environments require **Spark**
- **Real-time** pipelines at massive scale



“Spark is not obsolete — but it should be intentional.”

When You Actually Need Spark (Cluster)

"You need a cluster when the problem is fundamentally distributed."



“Don't start with Spark. End with
Spark

Documentation

POLARS

-  Docs → <https://pola.rs>
-  User Guide → <https://docs.pola.rs/user-guide/>
-  Expressions → <https://docs.pola.rs/user-guide/concepts/expressions/>
-  Cookbook → <https://docs.pola.rs/cookbook/>
-  GitHub → <https://github.com/pola-rs/polars>

DUCKDB

-  Docs → <https://duckdb.org/docs>
-  SQL Reference → <https://duckdb.org/docs/sql/introduction>
-  Python API → <https://duckdb.org/docs/api/python>
-  Working with Parquet → <https://duckdb.org/docs/data/parquet>
-  GitHub → <https://github.com/duckdb/duckdb>

RAPIDS (cuDF / cuML)

-  RAPIDS Docs → <https://docs.rapids.ai>
-  cuDF Docs → <https://docs.rapids.ai/api/cudf/stable/>
-  cuML Docs → <https://docs.rapids.ai/api/cuml/stable/>
-  Examples & Workflows → <https://docs.rapids.ai/maintainers/maintainers/>
-  GitHub → <https://github.com/rapidsai>

TALKS & LEARNING

-  DuckDB Design Talk → <https://www.youtube.com/watch?v=Z5zNPwMDYGg>
-  NVIDIA RAPIDS Talks → <https://www.youtube.com/@NVIDIADevelopers/search?query=rapids>
-  Modern Data Engines Talk (H2O.ai) → https://www.youtube.com/watch?v=B_9lswZlYck
-  Spark Architecture Refresher → <https://www.youtube.com/watch?v=7ooZ4S7Ay6Y>

Session 3: Processing with dbt

Carlos Ferrer

Agenda

Session 1: Data Ingestion & Spark SQL

- Data ingestion concepts & tools (Airbyte, Mage)
- Spark SQL overview & hands-on exercises

Session 2: Single-Node Processing

- Single-node vs. cluster processing
- Tools: Polars, NVIDIA Rapids, DuckDB

Session 3: Processing with dbt

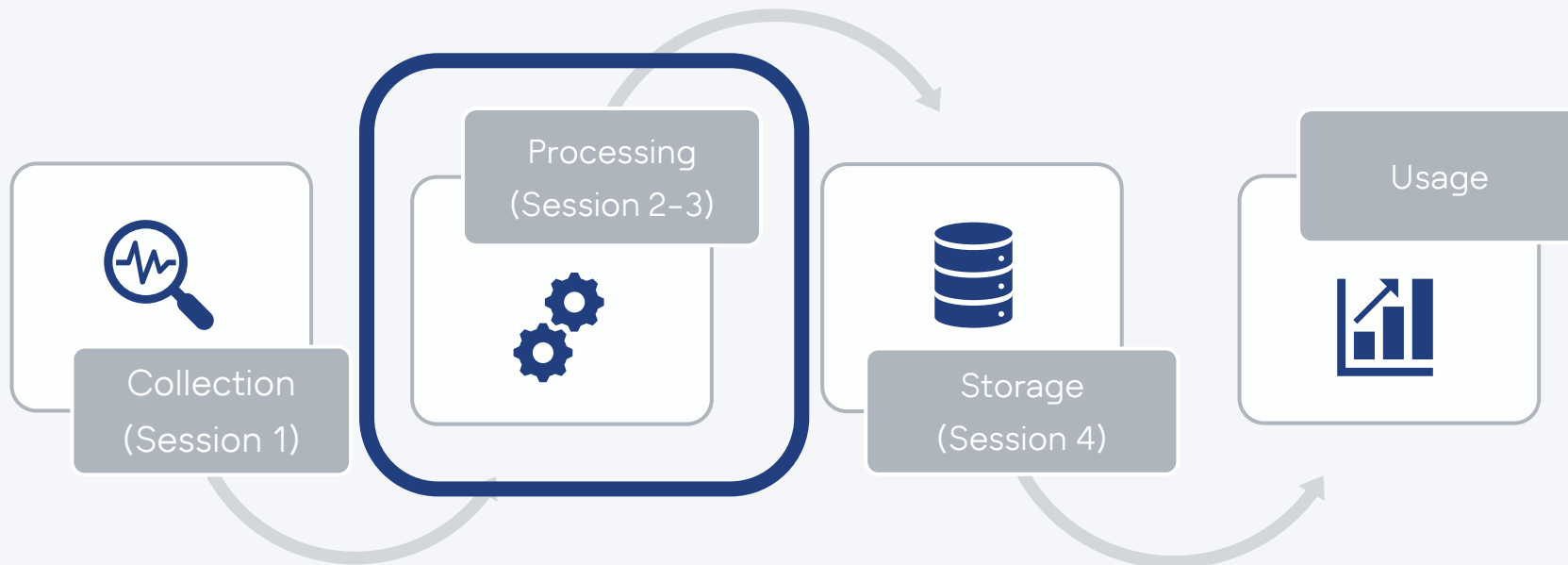
- dbt fundamentals (DuckDB-based)
- End-to-end data transformation exercise

Session 4: Vector Storage & Semantic Search

- Semantic search & vector databases
- End-to-end pipeline with Qdrant

Session 3: Processing with dbt

- dbt fundamentals (DuckDB-based)
- End-to-end data transformation exercise



Session 3: Session overview

What dbt is and why it exists

How dbt works internally

Core components of dbt Core

dbt in modern data architectures

Theory first, practice later



Where this session fits in the data lifecycle



Raw data is not analytics-ready

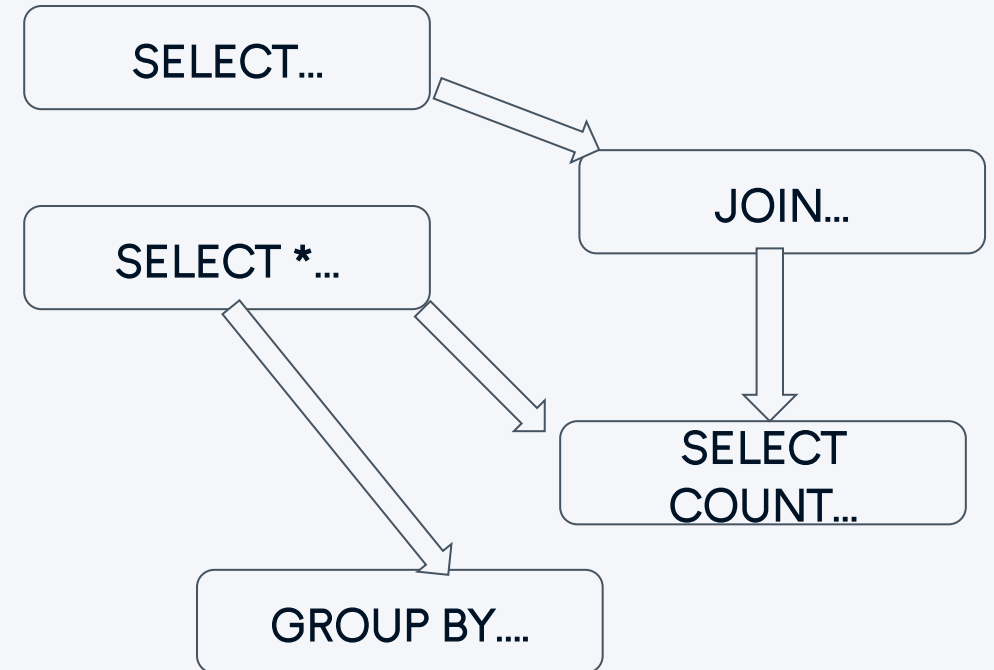
“Same data, different usability”

cust_id	ord_dt	amt	st_cd
10234	20251122	4599	A
10451	20251122	1299	B
10987	20251123	8999	A

customer_id	order_date	amount_eur	status
10234	2025-11-22	45.99	active
10451	2025-11-22	12.99	pending
10987	2025-11-23	89.99	active

The real complexity starts after ingestion

- SQL logic grows organically over time
- Queries get copied and pasted
- Business logic gets duplicated
- No single source of truth for transformations
- Ownership becomes unclear



SQL works... until it doesn't

- SQL is powerful and flexible
- But SQL alone doesn't provide structure
- Dependencies are implicit
- Changes become risky
- Maintenance cost grows with time

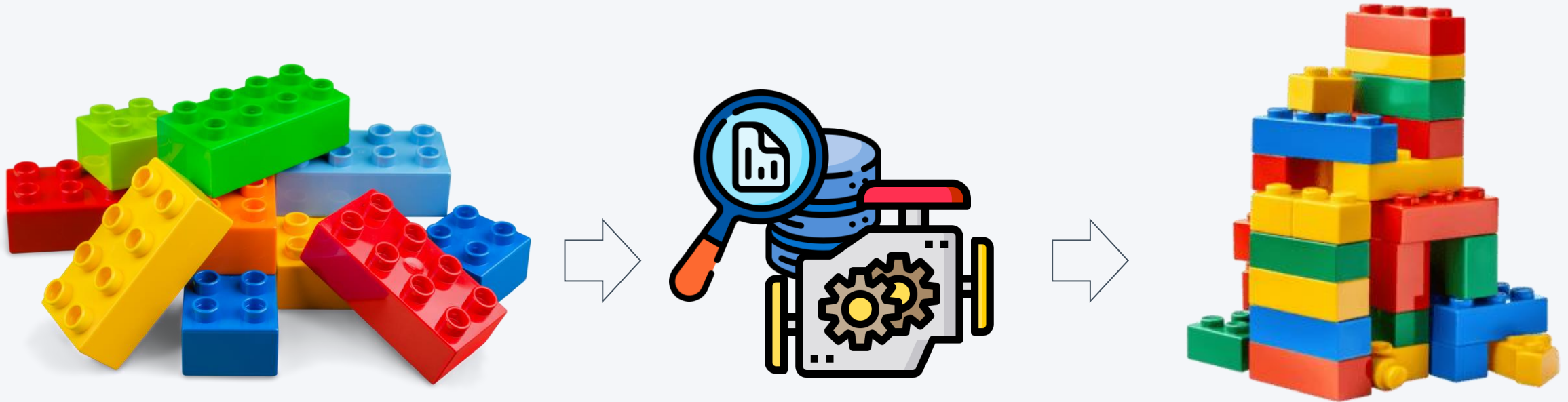


Fast, flexible



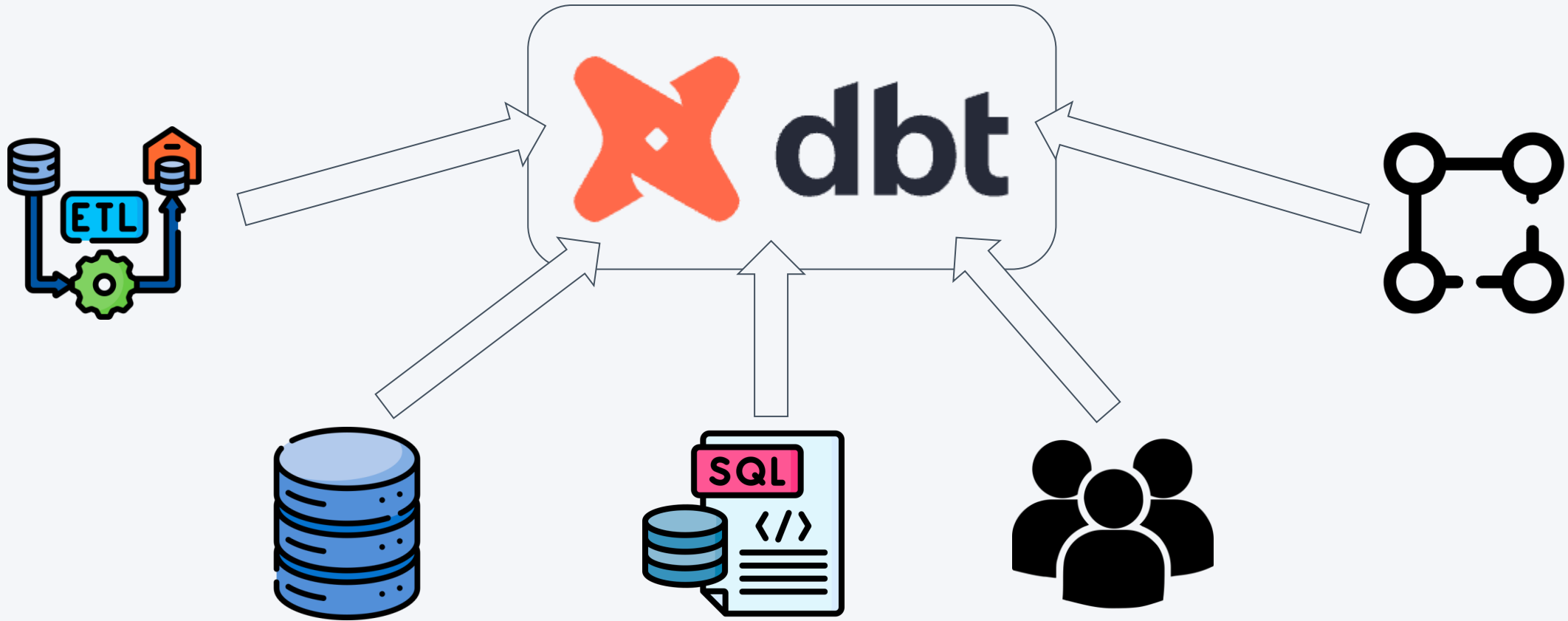
Hard to maintain at scale

Transformation needs engineering, not just queries

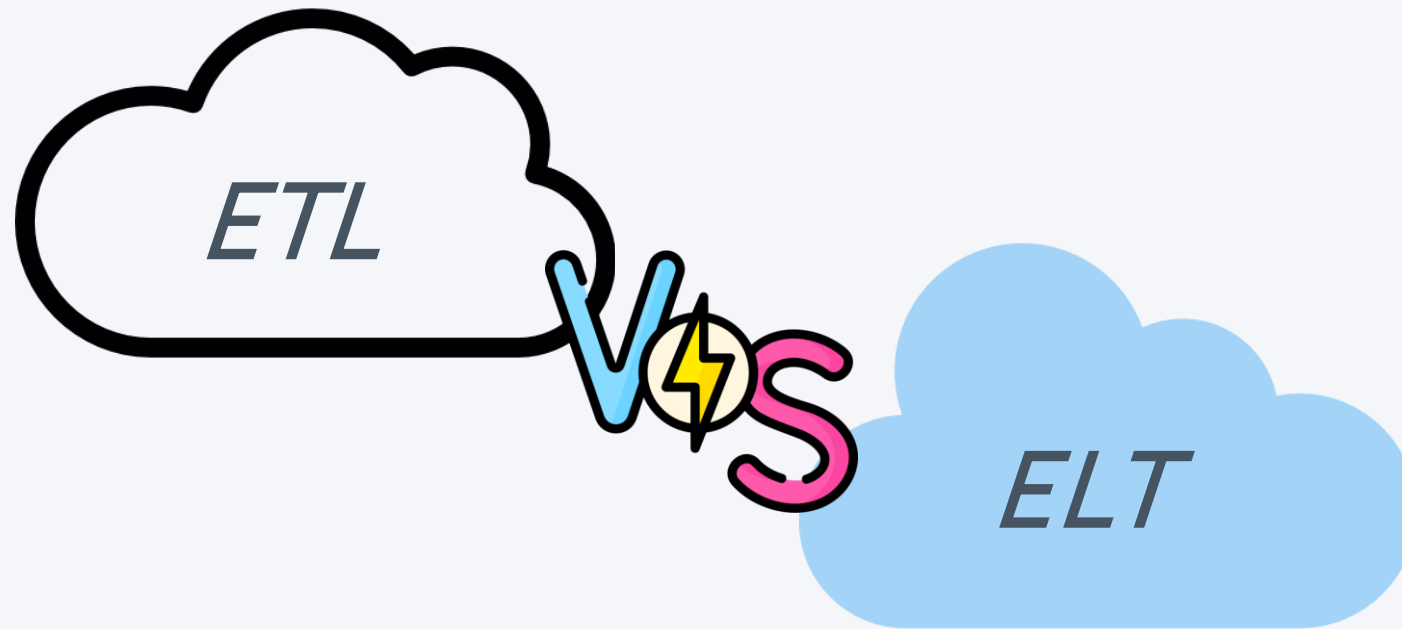


“Dependencies”, “Reuse”,
“Control”, “Structure”

This is the context where dbt appears



Why talk about ETL vs ELT?



Why ETL made sense (historically)

“Before cloud warehouses”

Warehouses were expensive



Compute inside the warehouse was limited

Transforming before loading reduced cost

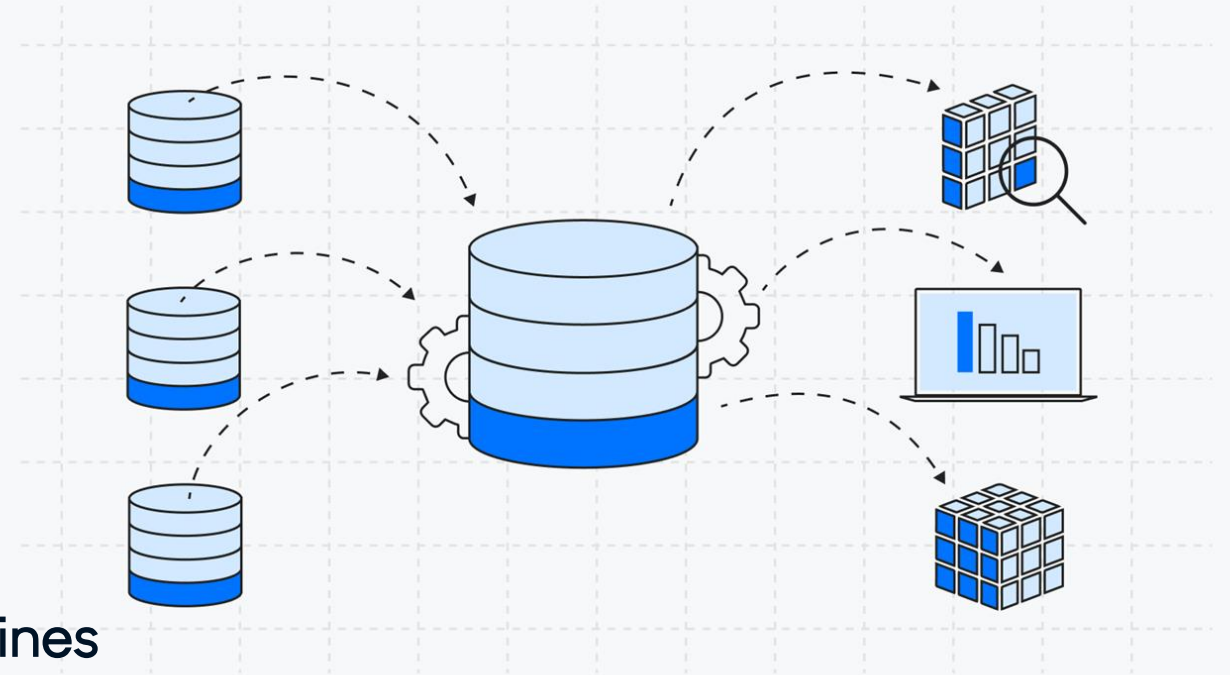


Centralized control of transformations

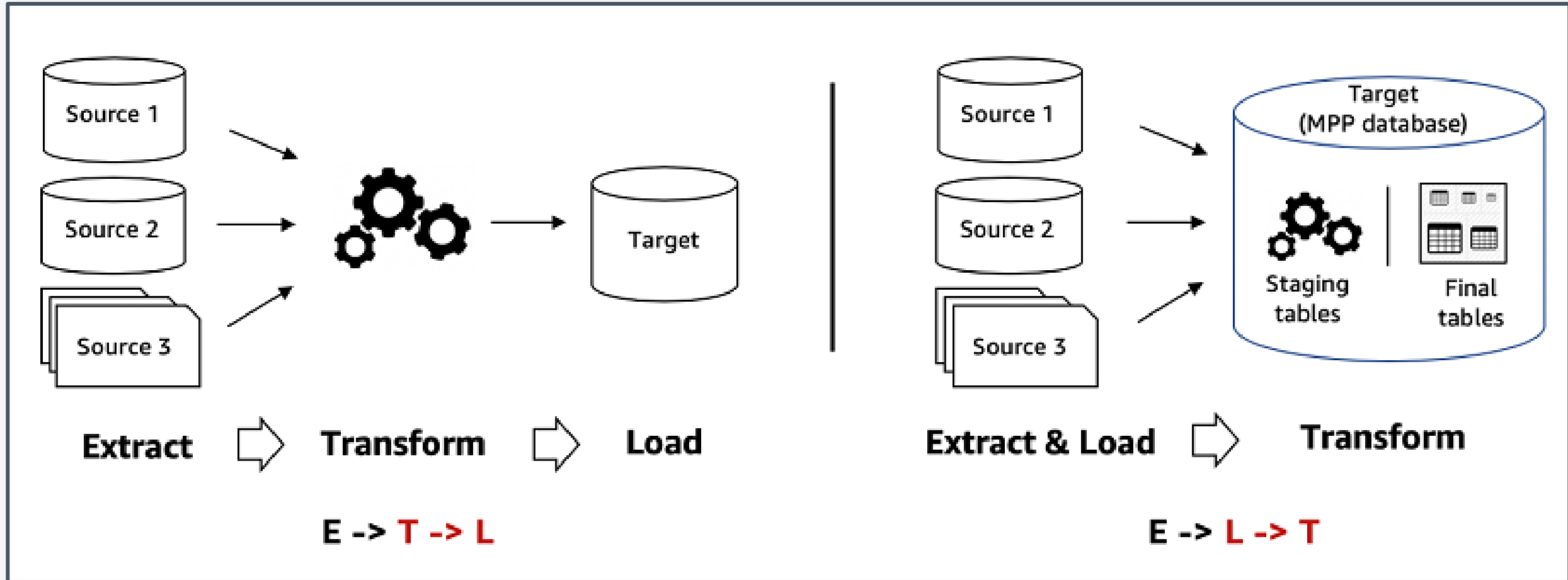
Limitations of ETL

“Before cloud warehouses”

- Transformations are **opaque**
- Logic is **hard to change**
- **Long development** cycles
- Tight **coupling** between **data** and **pipelines**



What is ELT?

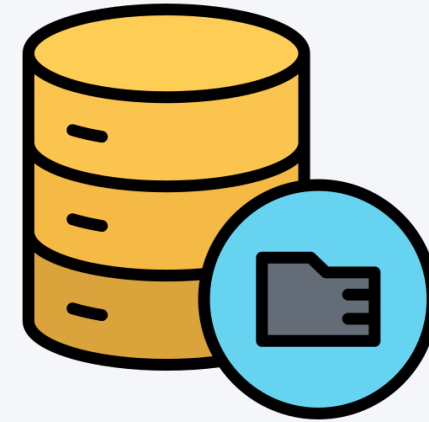


Why ELT works today



Cloud
warehouses are
scalable

Compute and storage
are decoupled



Transforming inside the
warehouse is cheap

New “problems” introduced by ELT



Raw data is always available

Transformations move to SQL

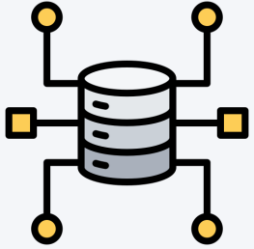


Many people write transformations

Risk of chaos without structure



This is where dbt fits perfectly



ELT needs structure

SQL needs engineering practices



Transformations need to be managed

dbt provides discipline to ELT



What is dbt (*Data Build Tool*) ?

“A transformation tool for analytics engineering”

❖ Key Features:

- **dbt** stands for *Data Build Tool*
- **dbt** is a transformation workflow tool
- **dbt** focuses on the T in ELT
- **dbt** works inside the data warehouse
- **dbt** is SQL-first



What problem does dbt solve?



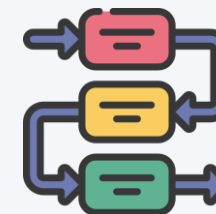
SQL transformations
grow without structure



Business logic is
duplicated



Teams struggle to scale
transformation work



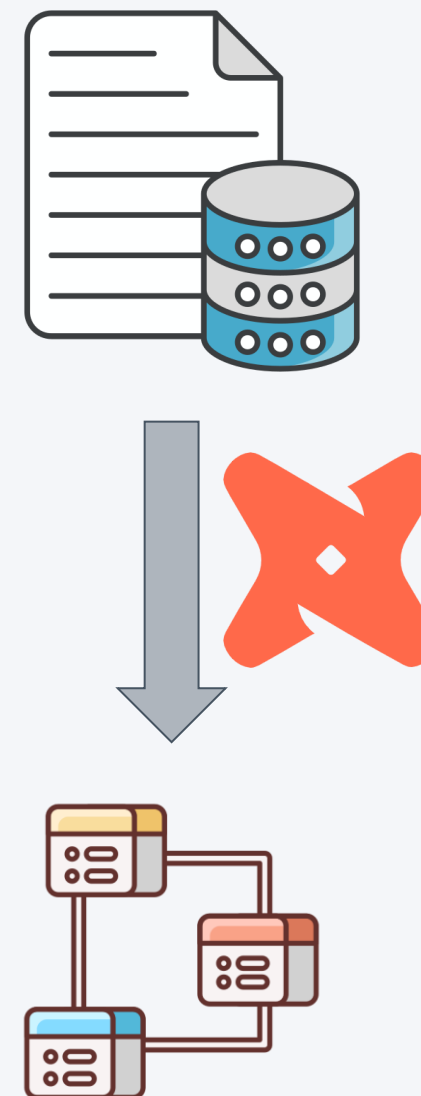
Dependencies are
implicit



Changes are risky

What dbt actually does

- ❖ Turns SQL files into **data models**
- ❖ **Manages dependencies** between models
- ❖ **Executes** transformations in the **right order**
- ❖ Applies **software engineering** practices to SQL
- ❖ Makes **transformations reproducible**



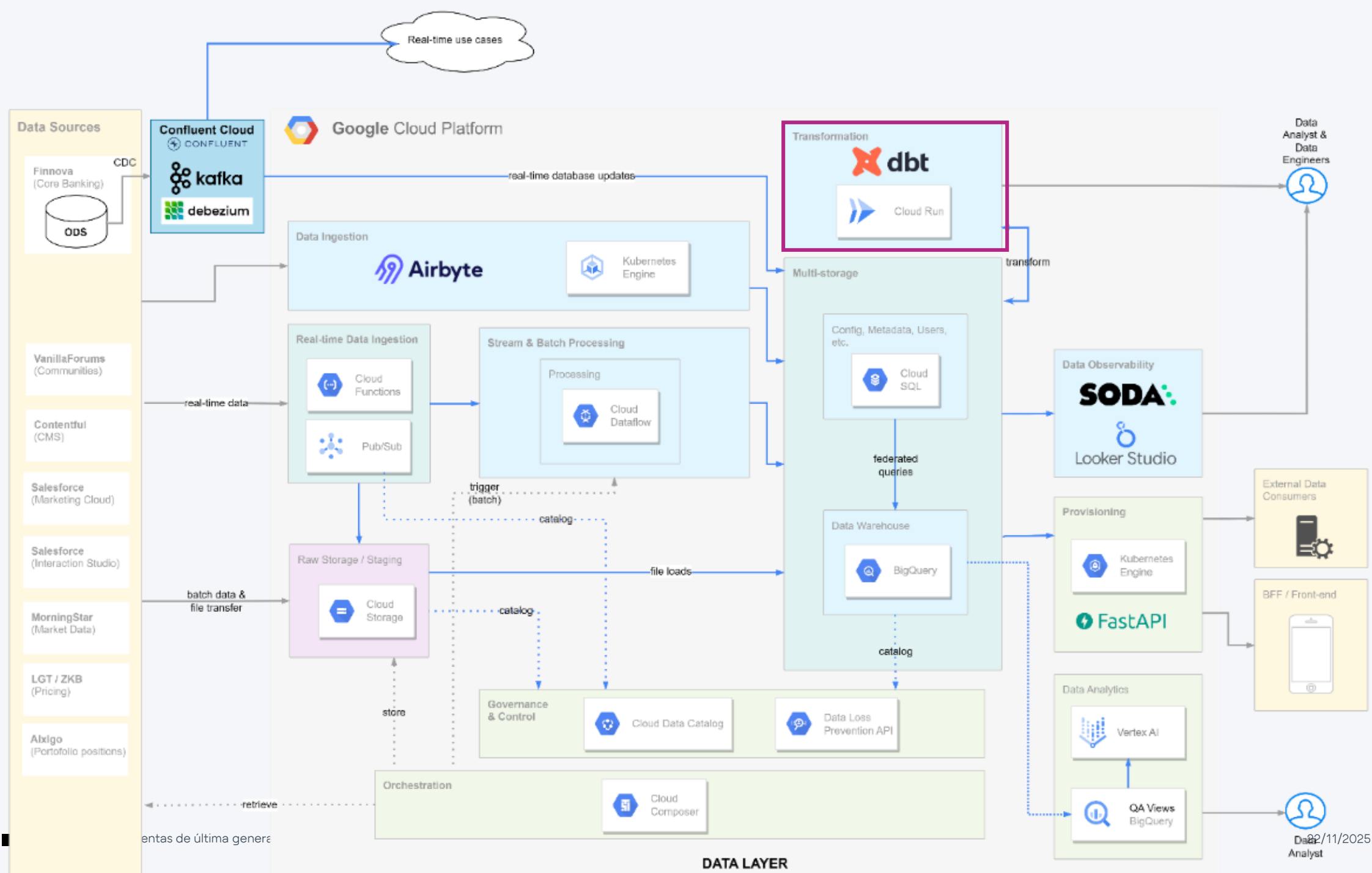
What dbt does NOT do

- ❖ dbt does not **extract** data
- ❖ dbt does not **load** data
- ❖ dbt does not **move** data between systems
- ❖ dbt does not **visualize** data
- ❖ dbt does not **replace** orchestration tools



dbt as the “T” in ELT





Why dbt is SQL-first

“No new language, no abstraction layer”

- SQL is the **language** of analytics
- SQL **runs** where the **data** lives
- SQL is **declarative** and **expressive**
- dbt **embraces** SQL instead of hiding it



Summary: what dbt is

"No new language, no abstraction layer"



A transformation workflow tool



SQL-first and warehouse-native

Focused on the T in ELT



Designed for scalable data teams



dbt comes in two flavors

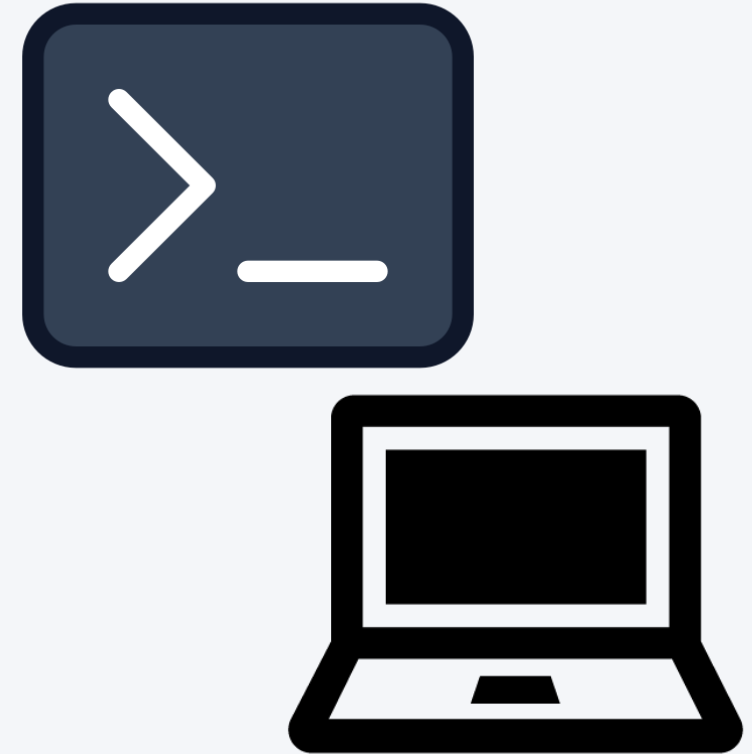
"Same dbt, different setup"



What is dbt Core?

"Same dbt, different setup"

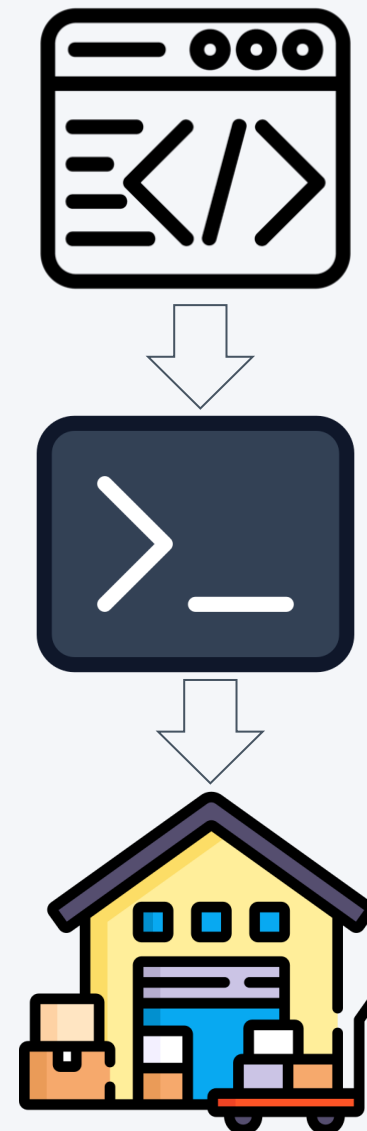
- Open-source command-line tool
- Runs locally or on your own infrastructure
- Fully controlled by the user
- No built-in UI or scheduler



What does using dbt Core feel like?

"Same dbt, different setup"

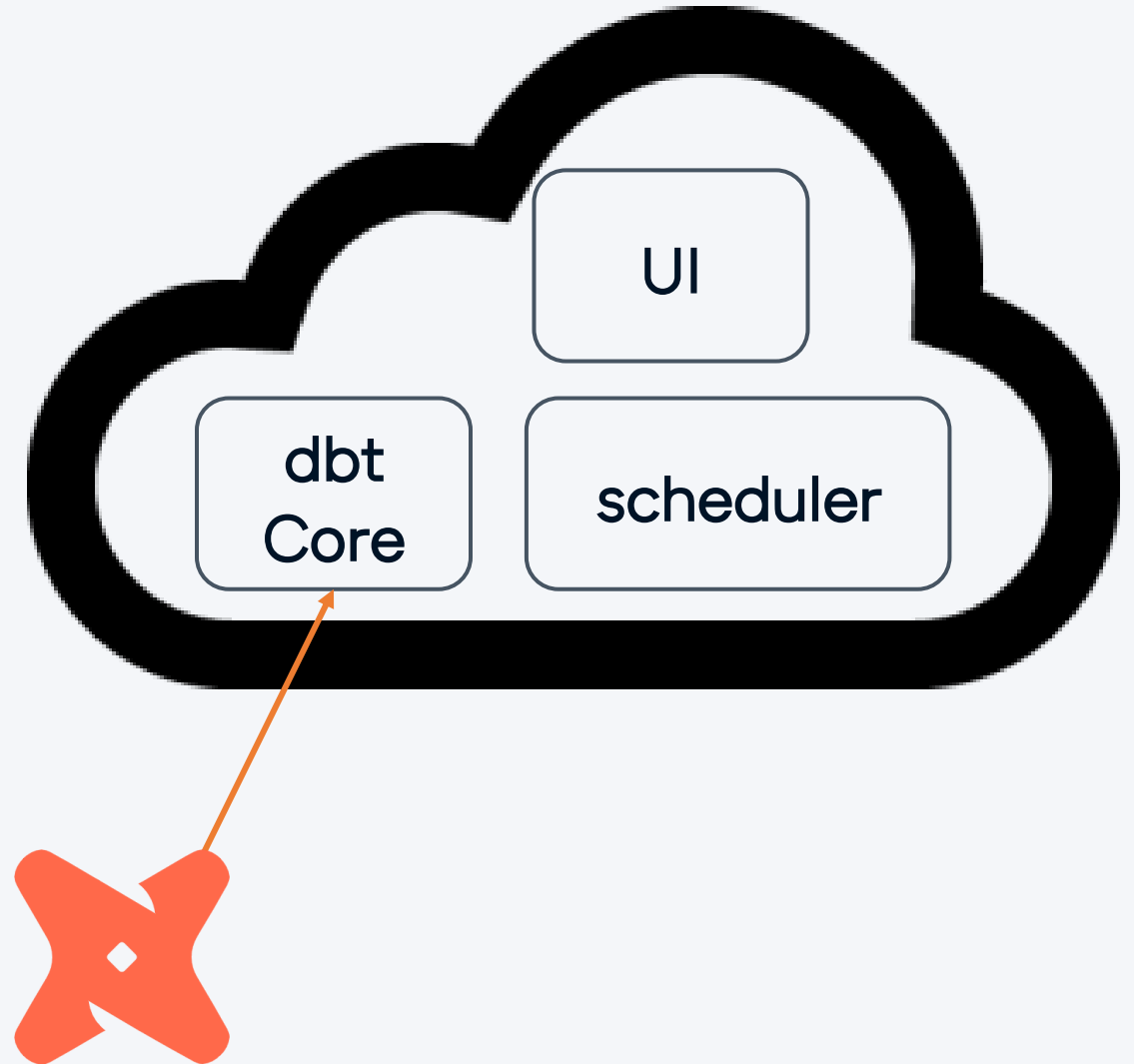
- You run dbt commands manually
- You manage environments yourself
- You handle credentials and secrets
- You decide how and when dbt runs



What is dbt Cloud?

"Same dbt, different setup"

- Managed SaaS platform
- Runs dbt Core under the hood
- Provides UI, scheduler, and environment management
- Simplifies team workflows
- Paid product



What dbt Cloud adds on top of dbt Core

"Same dbt, different setup"



IDE



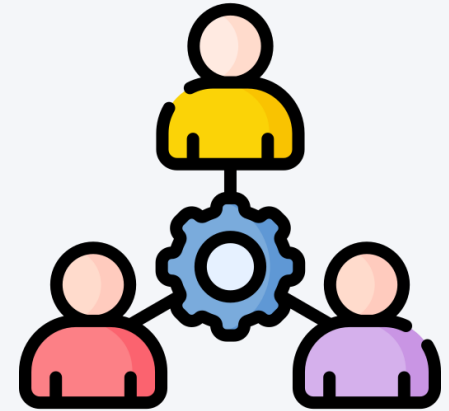
Scheduler



Environment
Manager



Logs and
Monitoring



Easier for
Teams

Core vs Cloud: conceptual differences

"Same dbt, different setup"

Conceptual
layer



Operational
layer



When dbt Core makes sense

"Same dbt, different setup"



Learning dbt

Full control

Custom
infrastructure

Small teams or
individual

Cost-sensitive
environments

Choosing Core or Cloud depends on context

"Same dbt, different setup"

Operational
complexity

Cost constraints

Team size

Governance needs

Execution frequency



Summary: dbt Core vs dbt Cloud

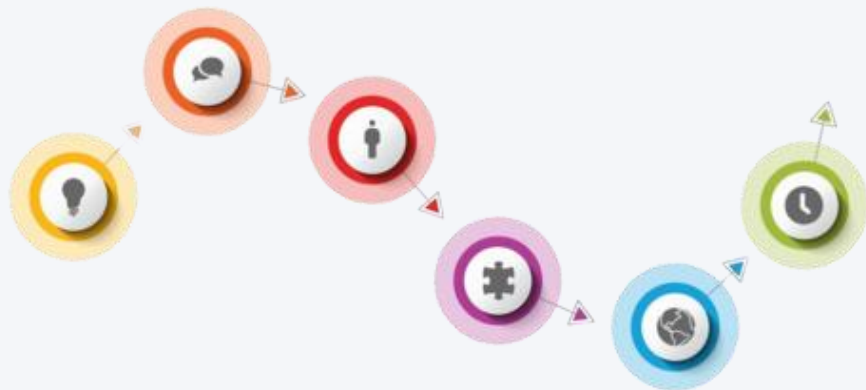
"Same dbt, different setup"

- ❖ Same dbt concepts
- ❖ Same transformation logic
- ❖ Different execution model
- ❖ Core = control
- ❖ Cloud = convenience



dbt is a declarative tool

- dbt is **declarative**, not imperative
- You describe *what you want*, not *how* to execute it

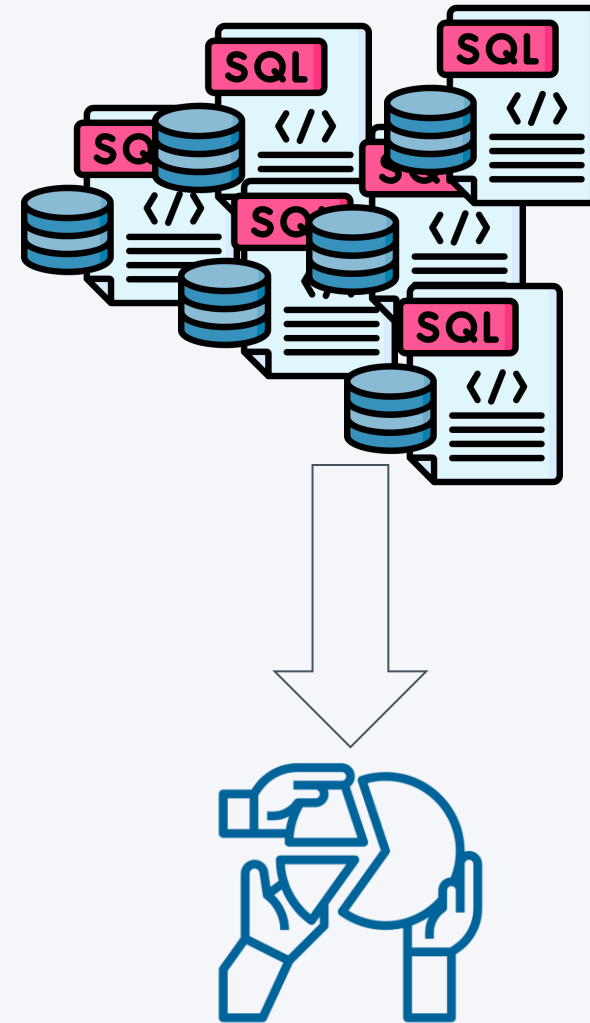


- Execution order is **inferred**, not hard-coded
- Dependencies define the **workflow**



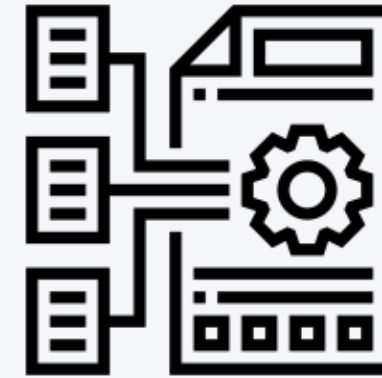
From SQL files to a dbt project

- ❖ dbt starts from SQL files
- ❖ Each file represents a model
- ❖ Models are part of a single project
- ❖ dbt reads the **entire project** before running



Compilation vs execution

- ❖ Compilation happens before execution
- ❖ SQL + refs + configs are resolved
- ❖ The DAG is built during compilation
- ❖ Execution happens after compilation



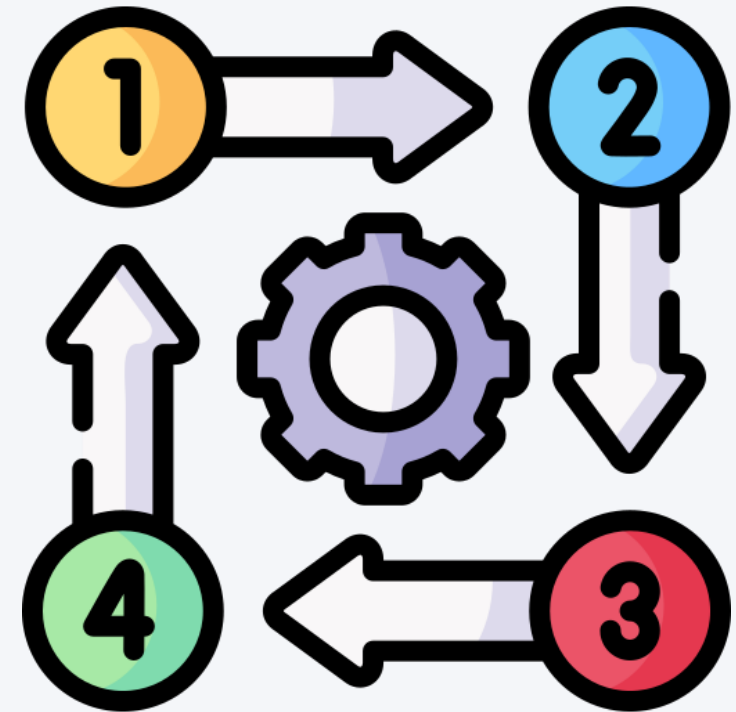
Errors: compilation vs execution

- ❖ Compilation errors stop dbt early
- ❖ Execution errors happen in the warehouse
- ❖ Different errors, different causes
- ❖ Understanding this saves debugging time



What happens during compilation

1. dbt reads all models
2. **ref()** calls are resolved
3. Dependencies are detected
4. Configurations are applied
5. The DAG is fully built

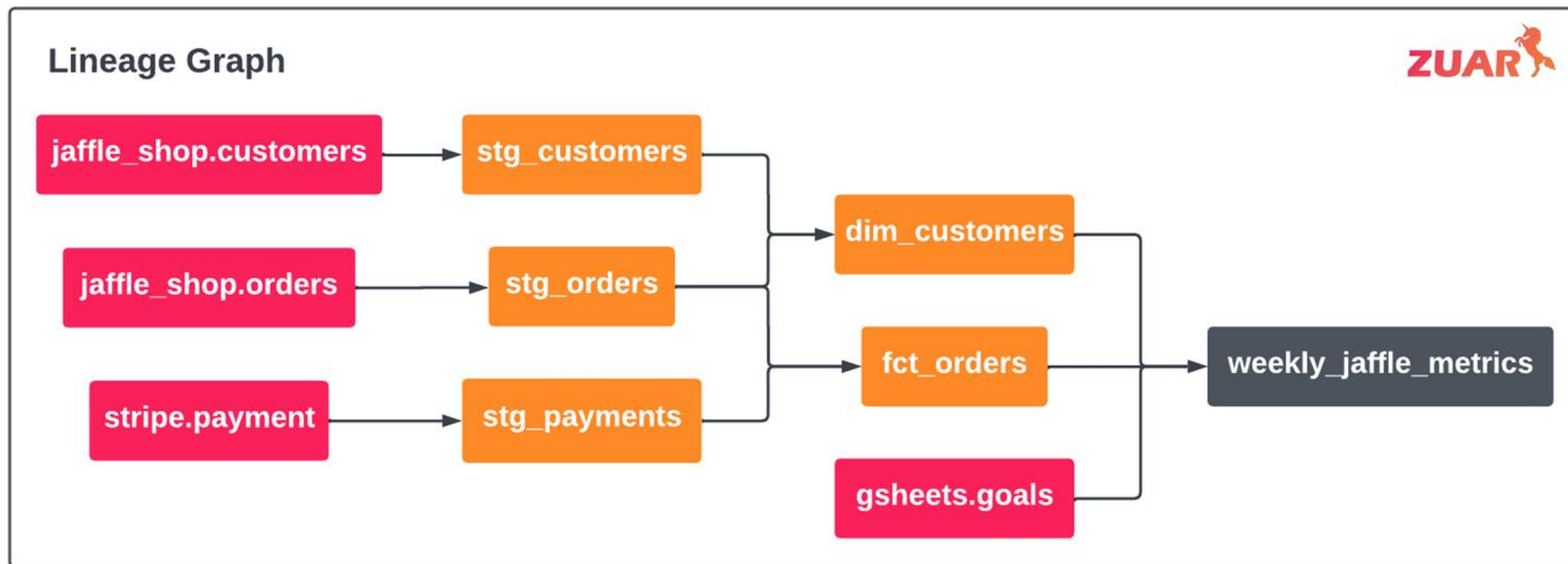


dbt as a compiler for SQL



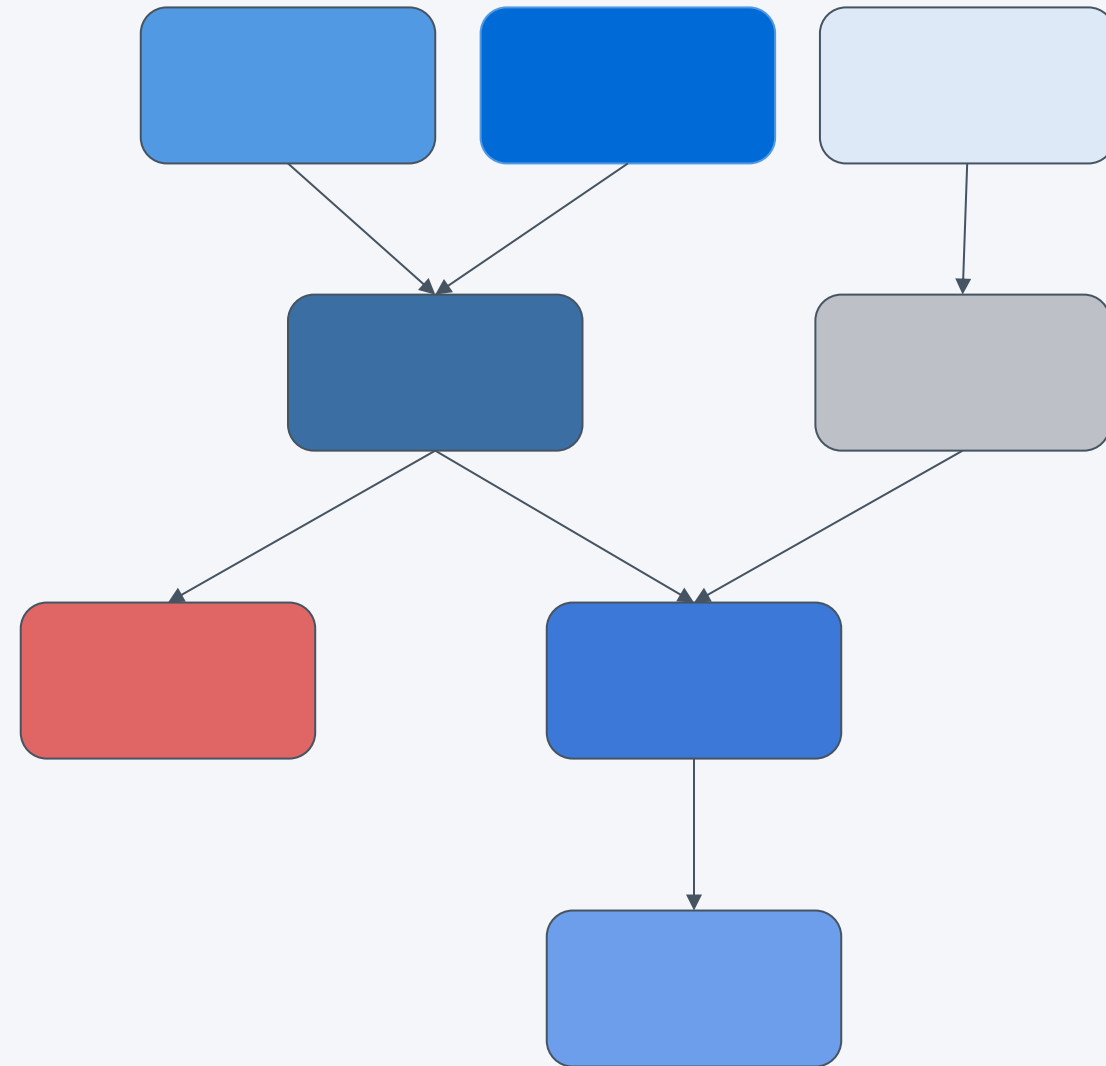
The DAG is the core of dbt

- ❖ dbt builds a Directed Acyclic Graph (DAG)
- ❖ Each model is a node
- ❖ Dependencies define edges
- ❖ The DAG defines execution order



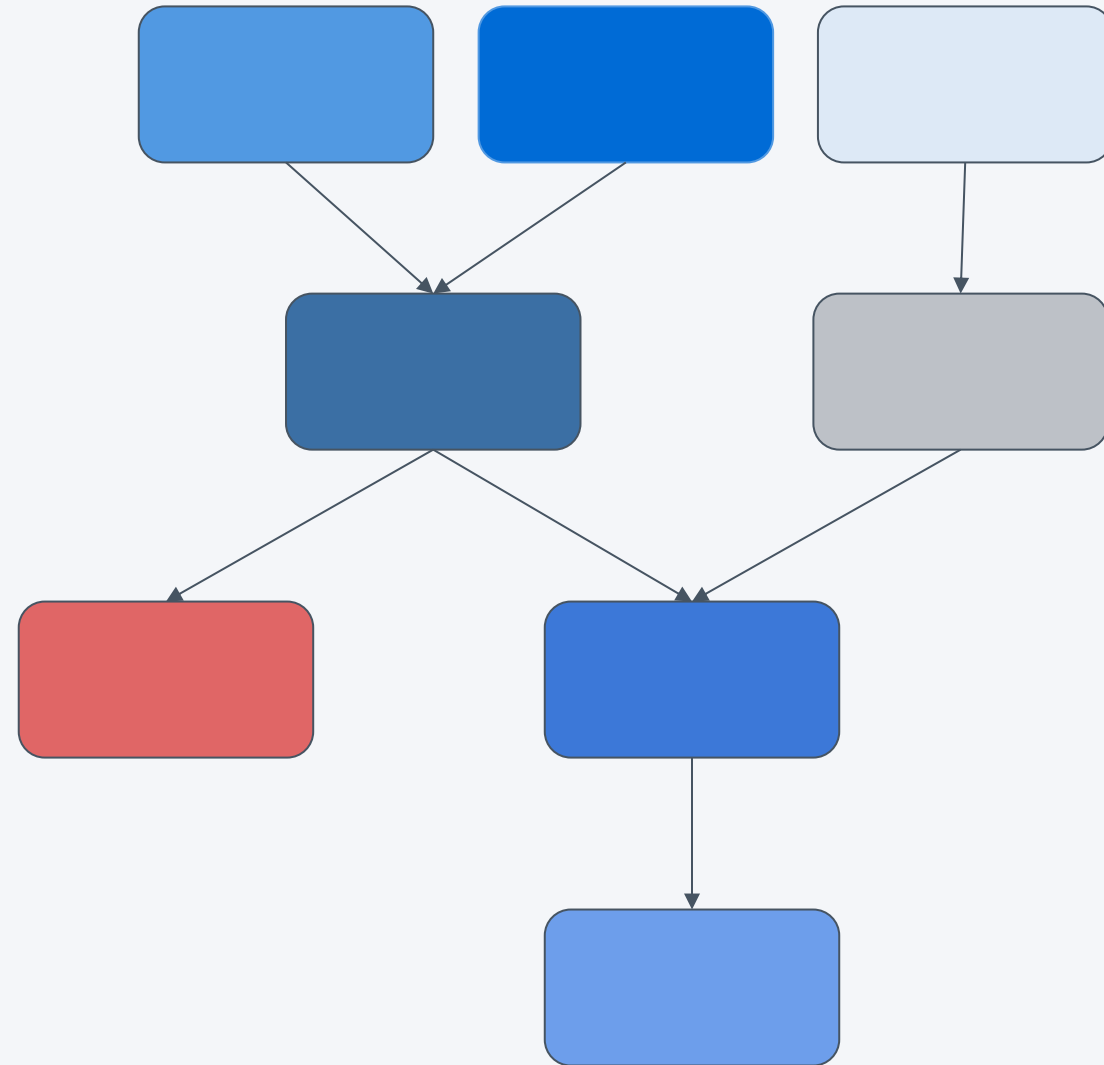
Execution follows the DAG

- ❖ Models are **executed** based on **dependencies**
- ❖ **Independent models** can run in **parallel**
- ❖ dbt never violates dependencies
- ❖ Execution order is **deterministic**



Why dbt does not need manual orchestration

- ❖ No need to define execution **steps**
- ❖ No need to manage task order **manually**
- ❖ Dependencies drive **execution**
- ❖ Logic and **orchestration** are **separated**



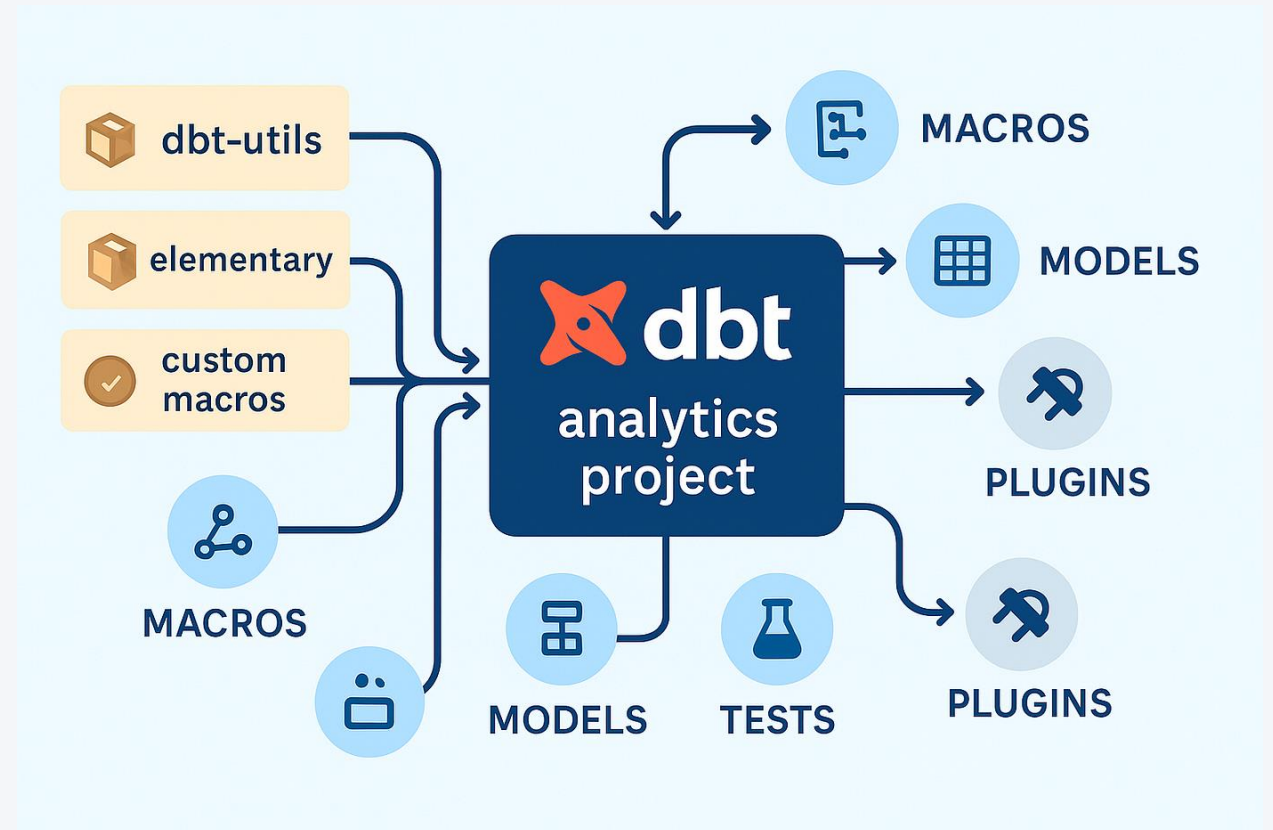
Mental model summary

- ❖ Write models declaratively
- ❖ dbt compiles the project
- ❖ A DAG is built
- ❖ Execution follows the DAG
- ❖ Same process every run

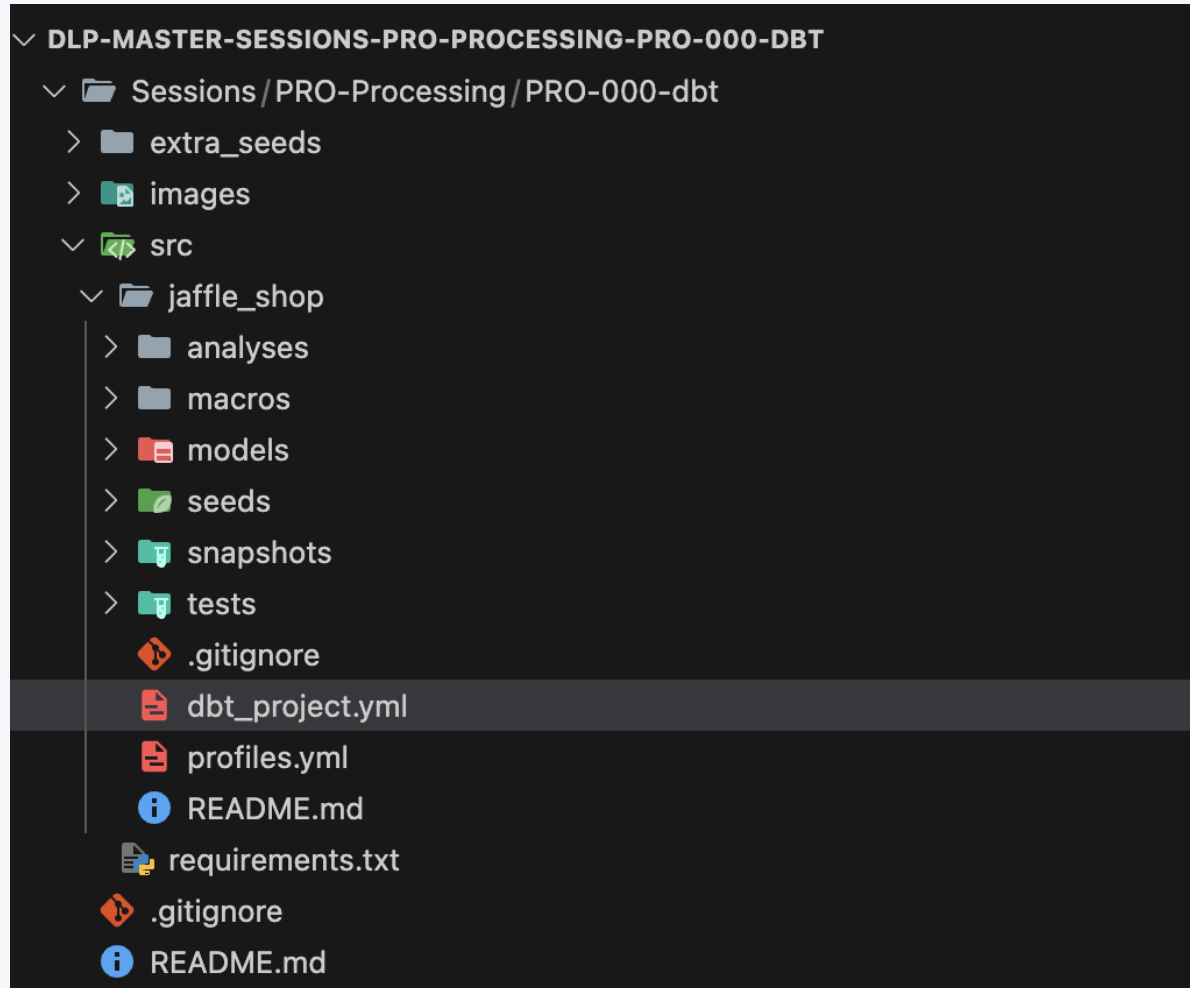


dbt project as a software project

- ❖ Version-controlled code
- ❖ Clear structure and conventions
- ❖ Separation of concerns
- ❖ Designed for collaboration



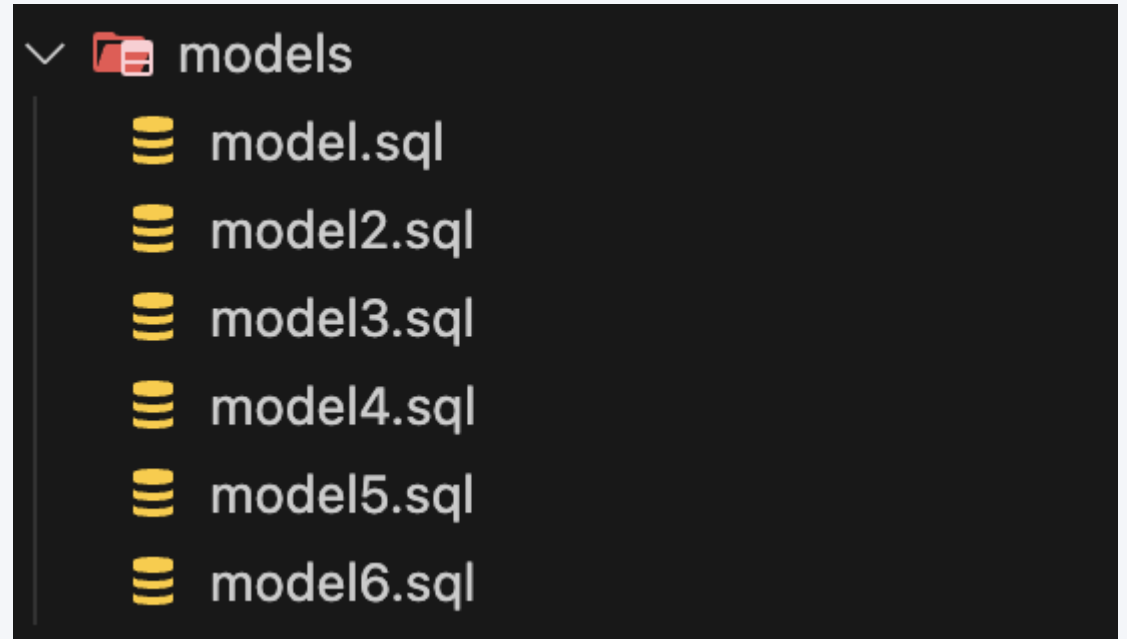
What is a dbt project?



- A dbt project is a codebase
- It contains models, configs, and metadata
- It defines how transformations are built
- dbt always operates at project level

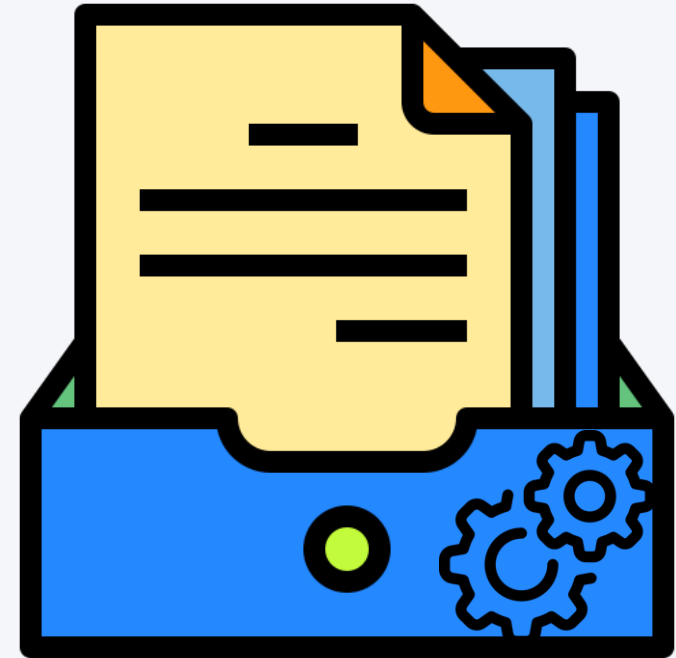
The **models/** directory

- ❖ Core of the dbt project
- ❖ Contains transformation logic
- ❖ One model per file
- ❖ Models define the DAG



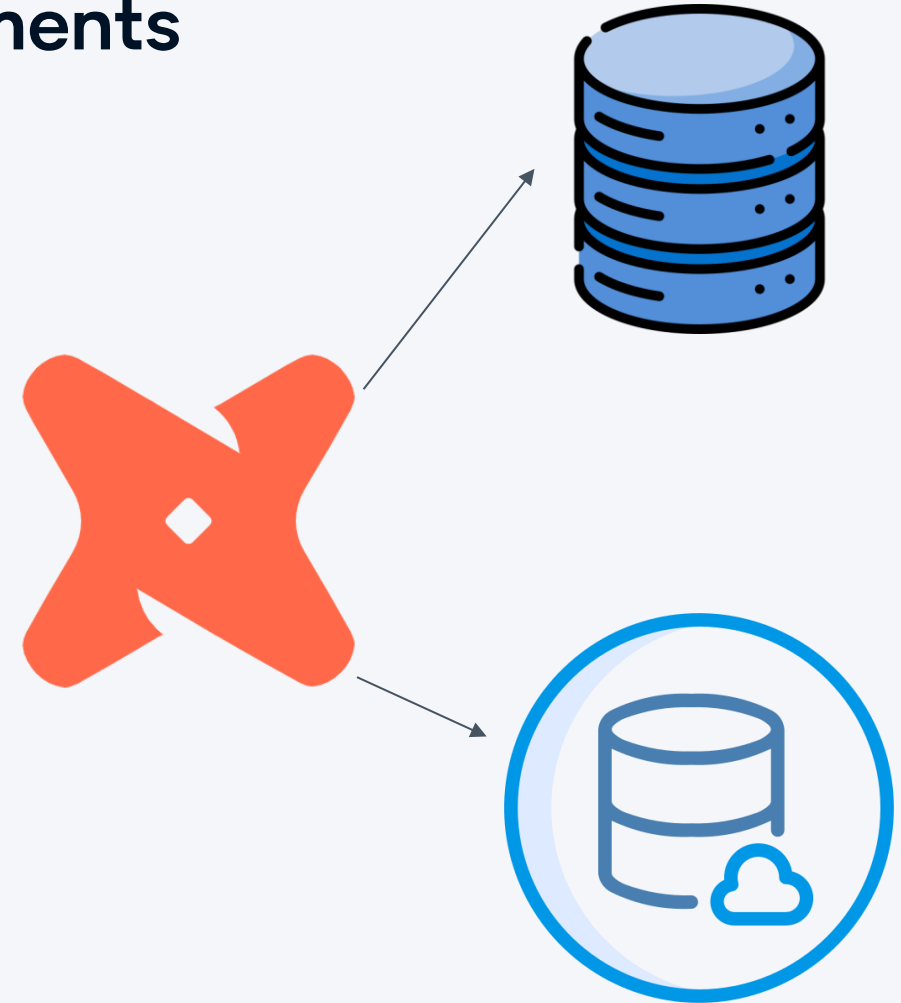
The role of `dbt_project.yml`

- Central configuration file
- Defines project-level settings
- Controls model behavior
- Sets defaults and conventions



profiles.yml: connections and environments

- Defines warehouse connections
- Contains credentials and targets
- Separate from the project code
- Environment-specific configuration



Why structure matters in dbt

To be Explicit

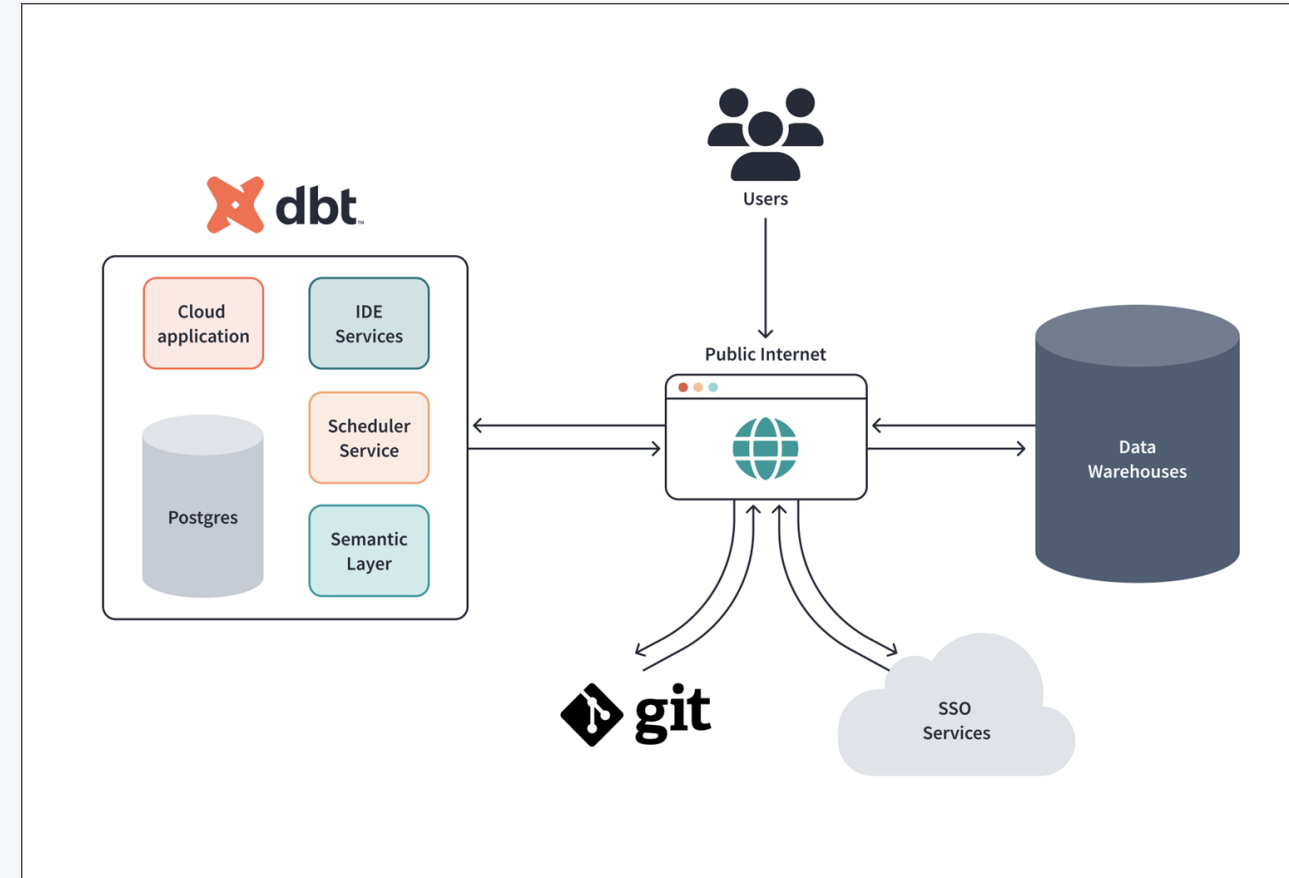
To be Easier

To be Reasonable

To be Scalable

dbt operates on the entire project

- dbt reads all models before running
- Dependencies are global
- Execution is project-wide
- Partial runs are still project-aware



Summary: the dbt project

- dbt works at **project level**
- **Structure** is intentional
- **Logic** and **configuration** are **separated**
- Projects enable **collaboration** and **scale**



What is a dbt model?

- ❖ A dbt model is a **SQL SELECT** statement
- ❖ One **model** = one **file**
- ❖ A **model** produces a **relation** in the **warehouse**
- ❖ **Models** are the **building** blocks of **dbt**

```
-- models/customers.sql  
SELECT  
    customer_id,  
    first_name,  
    last_name  
FROM raw_customers
```

Models are not ad-hoc queries

- ❖ Models are named **transformations**
- ❖ Models have a clear **purpose**
- ❖ Models are **reused** by other models
- ❖ Models are part of a **graph**, not isolated

```
-- ❌ Ad-hoc query  
SELECT *  
FROM raw_orders  
WHERE status = 'ACTIVE'
```

```
-- ✅ dbt model: stg_orders.sql  
SELECT  
    order_id,  
    customer_id,  
    order_date,  
    status  
FROM raw_orders  
WHERE status = 'ACTIVE'
```

One model, one responsibility

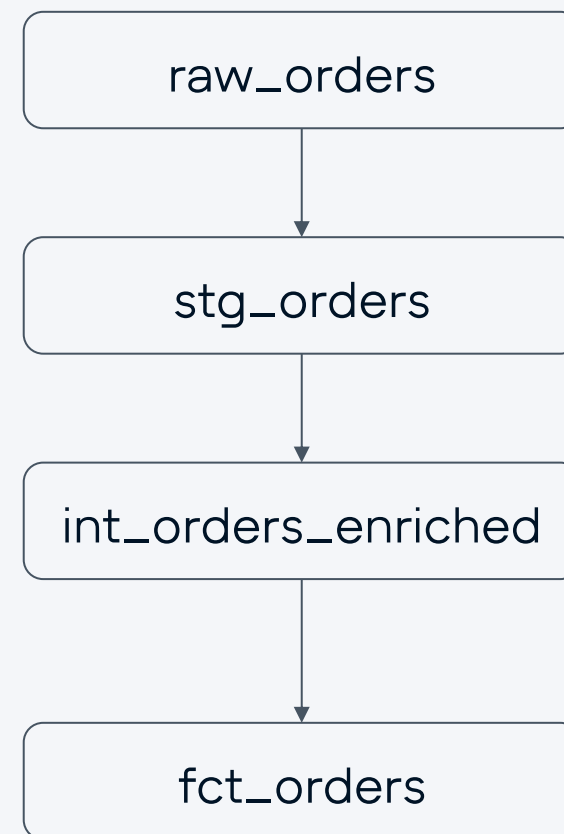
- ❖ Each model should do **one thing only**
- ❖ Avoid mixing cleaning, joins, and metrics
- ❖ Small models are easier to understand and change

❌ Too many responsibilities

```
SELECT
  c.customer_id,
  SUM(o.amount) AS total_revenue
FROM raw_customers c
JOIN raw_orders o ON c.customer_id = o.customer_id
WHERE o.status = 'COMPLETED'
GROUP BY c.customer_id
```

Models as building blocks

- ❖ Models are composed together
- ❖ Downstream models depend on upstream ones
- ❖ Complex logic emerges from composition



How models reference each other (ref())

- ❖ Models reference other models using `ref()`
- ❖ `ref()` declares a dependency
- ❖ dbt uses references to build the DAG

```
SELECT
    order_id,
    customer_id
FROM {{ ref('stg_orders') }}
```

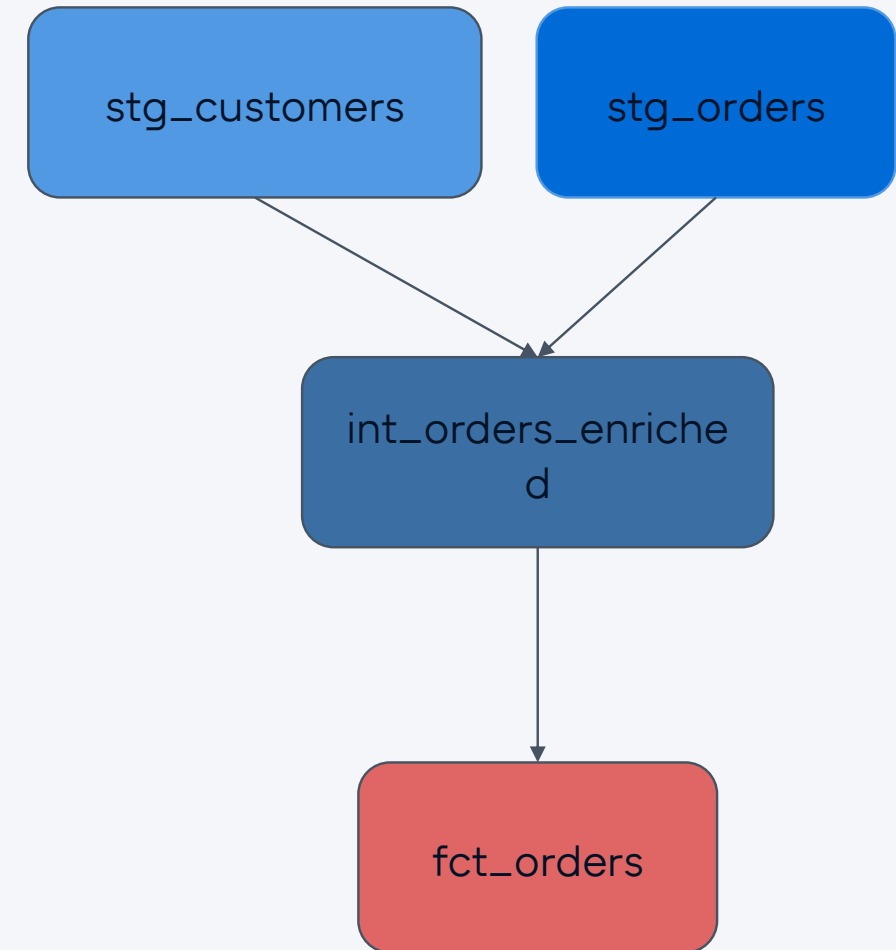

Why not reference tables directly?

- ❖ Hard-coded table names hide dependencies
- ❖ dbt cannot infer execution order
- ❖ Renaming becomes dangerous
- ❖ `ref()` keeps relationships explicit

```
SELECT
    order_id,
    customer_id
FROM {{ ref('stg_orders') }}
```

Models define the DAG

- ❖ Each model is a node
- ❖ Each `ref()` is an edge
- ❖ The DAG is built automatically



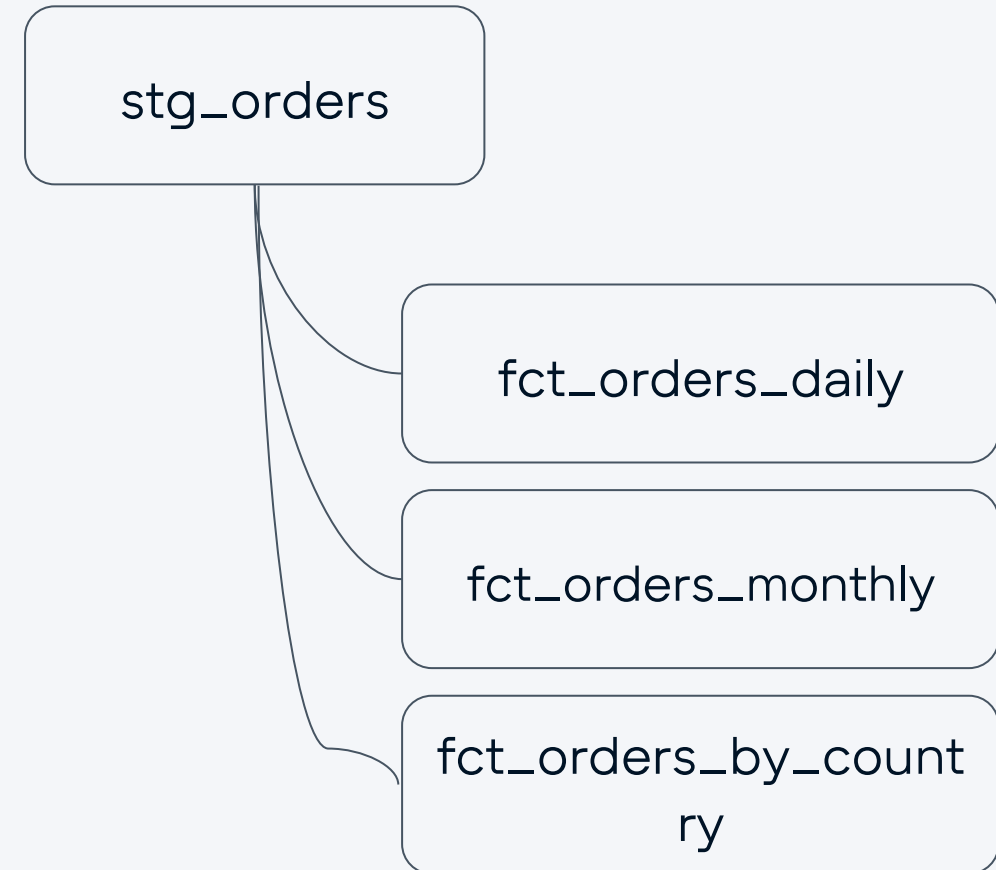
Naming models matters

- ❖ Names **communicate** intent
- ❖ Naming reflects **responsibility**
- ❖ Good names **reduce** cognitive load

- ✗ orders.sql
- ✓ stg_orders.sql
- ✓ int_orders_enriched.sql
- ✓ fct_orders_daily.sql

Models are meant to be reused

- Upstream models should be **stable**
- Downstream models **build on top**
- Reuse **reduces duplication** and risk



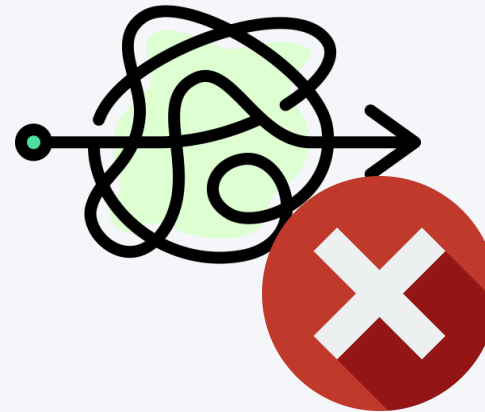
Summary: models in dbt

- ★ Models are SQL files
- ★ One responsibility per model
- ★ Models reference each other with `ref()`
- ★ Models compose into a DAG



Why layered modeling?

- Transformations grow in complexity
- Mixing concerns makes models hard to maintain
- Layers separate responsibilities
- Layered models scale better in teams



Layered modeling as separation of concerns

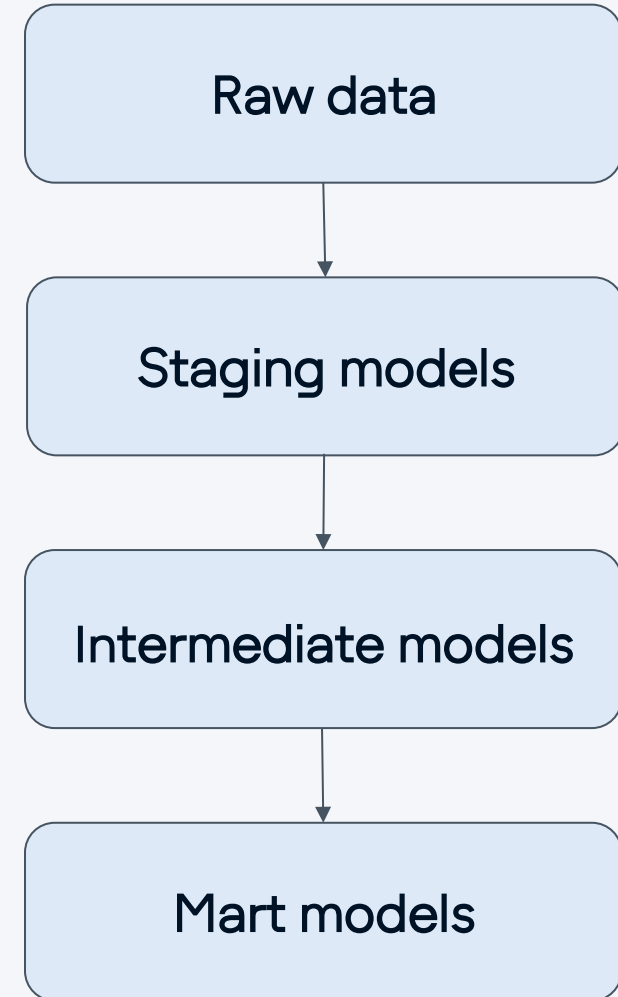
Is the data clean?

Is the data enriched?

Is the data business-ready?

Overview of dbt layers

- **Staging models:** clean and standardize raw data
- **Intermediate models:** join and enrich data
- **Mart models:** business-facing, analytics-ready data



Staging models: purpose

- First layer on top of raw data
- One staging model per source table
- Clean, rename, and type columns
- No business logic

```
-- stg_orders.sql
SELECT
    order_id,
    customer_id,
    CAST(order_date AS DATE) AS order_date,
    amount / 100.0 AS amount_eur,
    status
FROM raw_orders
```

What staging models should NOT do

- ❖ No joins across multiple sources
- ❖ No aggregations
- ❖ No business metrics
- ❖ No filtering for analysis



```
SELECT  
  customer_id,  
  SUM(amount)  
FROM raw_orders  
GROUP BY customer_id
```

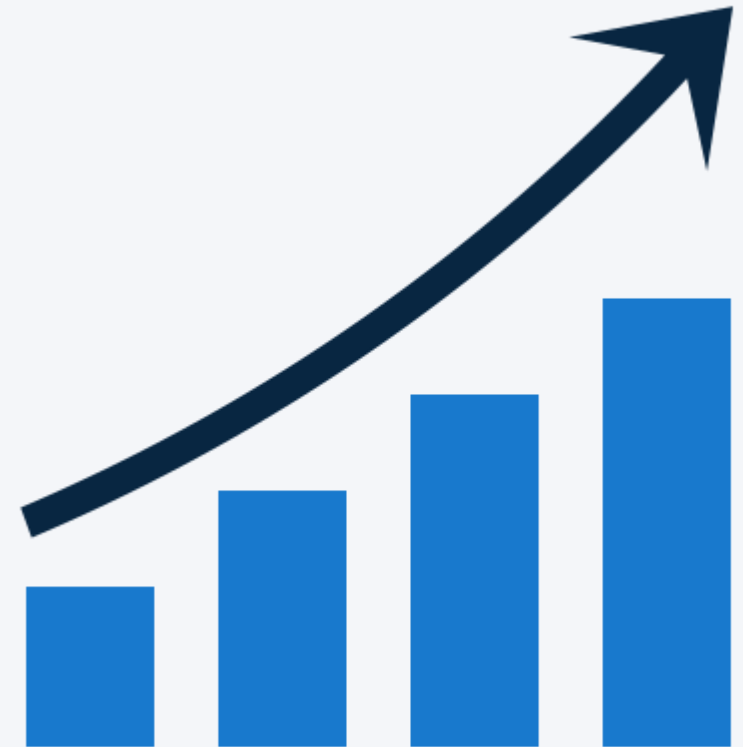
Intermediate models: why they exist

- ❖ Combine multiple staging models
- ❖ Apply joins and enrichments
- ❖ Prepare data for final consumption
- ❖ Reduce complexity in marts

```
-- int_orders_enriched.sql
SELECT
    o.order_id,
    o.order_date,
    o.amount_eur,
    c.country
FROM {{ ref('stg_orders') }} o
JOIN {{ ref('stg_customers') }} c
    ON o.customer_id = c.customer_id
```

When intermediate models are optional

- ❖ Small projects may skip this layer
- ❖ Simple pipelines may not need it
- ❖ Complexity is the deciding factor
- ❖ Introduce intermediate when models grow



Mart models: business-facing data

- ❖ Final layer for analytics and BI
- ❖ Aggregated and business-oriented
- ❖ Stable schemas and definitions
- ❖ One source of truth for metrics

```
-- fct_orders_daily.sql
SELECT
    order_date,
    SUM(amount_eur) AS total_revenue
FROM {{ ref('int_orders_enriched') }}
GROUP BY order_date
```

Why marts should stay simple

Easy to understand

Easy to validate

Easy to change safely

Built on top of clean
layers

Summary: layered modeling

- ★ Layers separate responsibilities
- ★ Each layer answers a different question
- ★ Layered models are easier to maintain
- ★ This pattern scales with team and data size



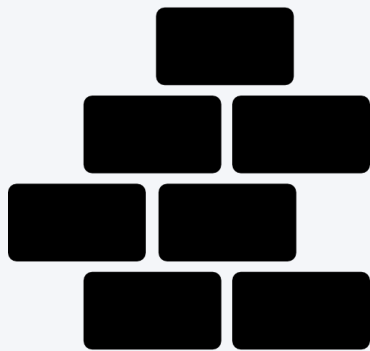
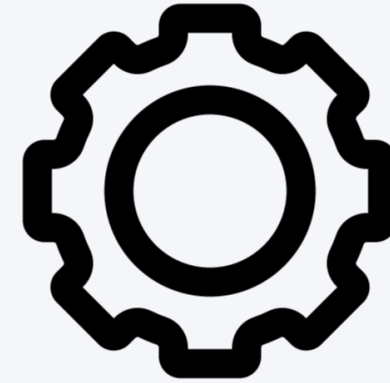
What is a materialization?

A materialization defines **when** a model is computed

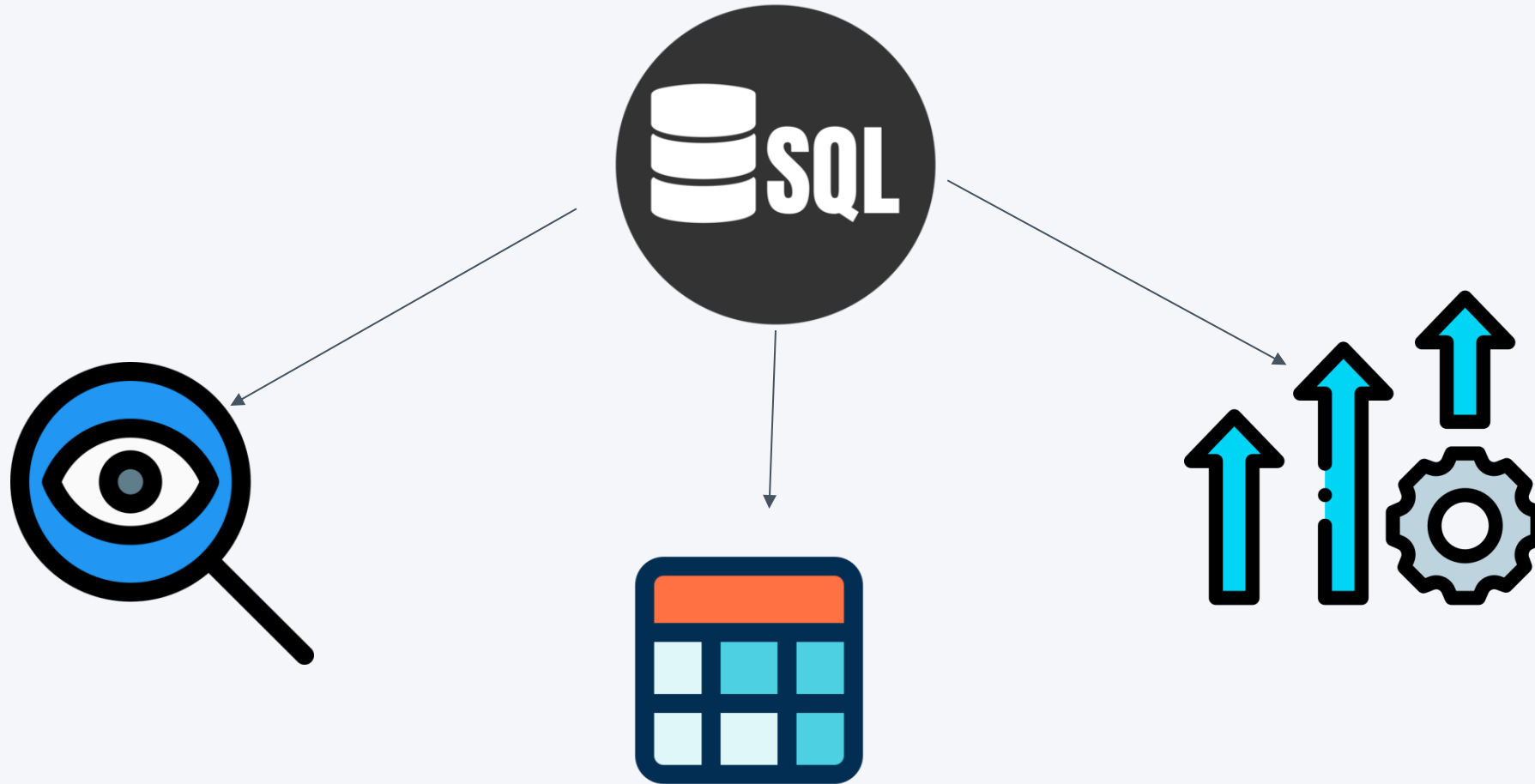
And **whether** the result is stored

The SQL logic does **not** change

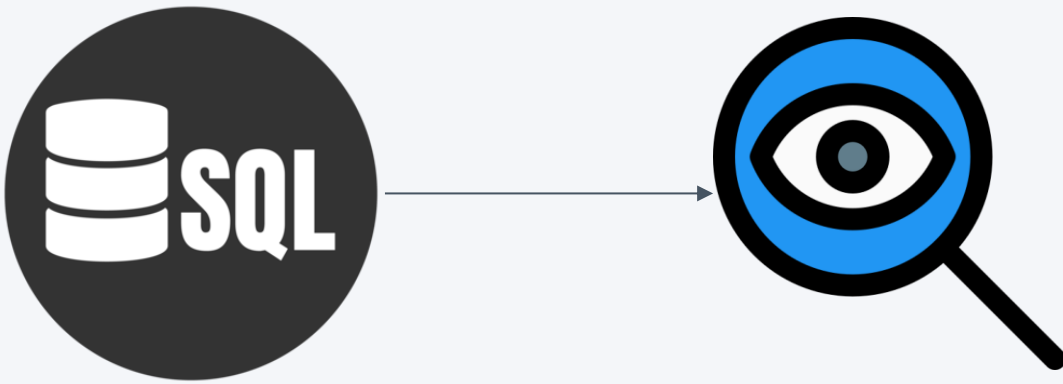
Only the physical behavior changes



Same SQL, different behaviors



View: compute on read



- ❖ Does not store data
- ❖ Stores the SQL logic
- ❖ Computed every time it is queried

Table: compute once, store result

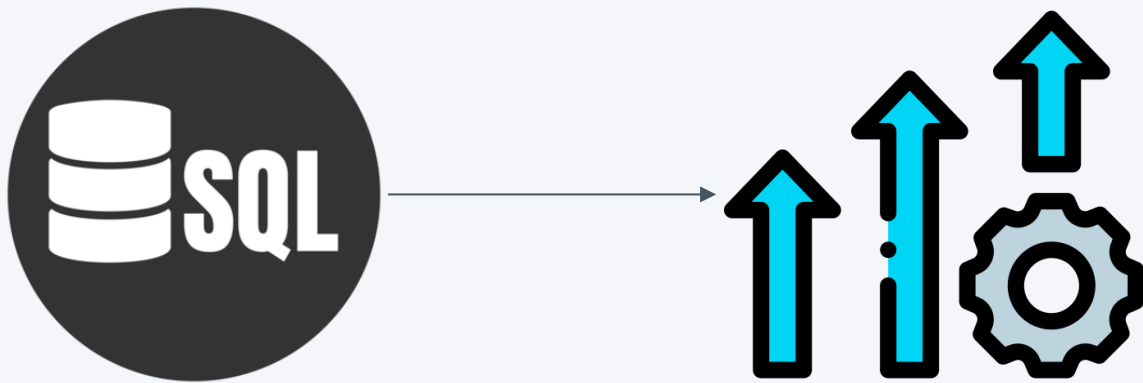


- ❖ SQL runs at build time
- ❖ Result is stored as a table
- ❖ Reads are fast

View vs Table (the core decision)

Question	View	Table
Stored?	No	Yes
When computed?	On read	On build
Good for	Early layers	Final data

Incremental: concept only



- ❖ Only processes new data
- ❖ Avoids full recomputation
- ❖ Used for very large tables

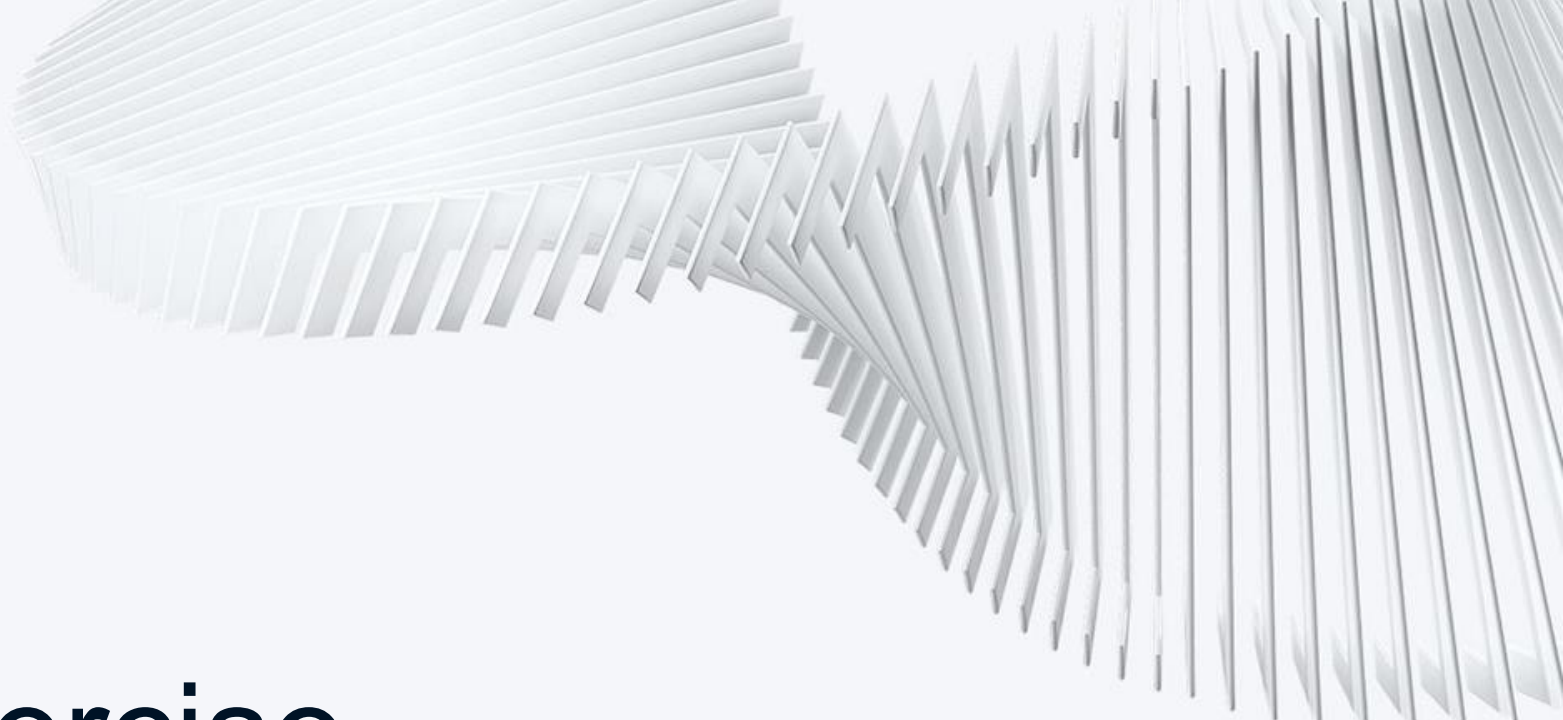
When incremental makes sense:

- Large fact tables
- Append-only data
- Stable logic

Summary: materializations

- ★ Materializations define physical behavior
- ★ View vs Table is the main decision
- ★ Incremental for scale
- ★ Key for performance and cost





Practical Exercise

Learn More About dbt

- Official dbt Documentation
 - [What is dbt? | dbt Developer Hub \(getdbt.com\)](https://getdbt.com/docs/what-is-dbt/)
- Models
 - [SQL models | dbt Developer Hub \(getdbt.com\)](https://getdbt.com/docs/sql-models/)
- Tests
 - [Add data tests to your DAG | dbt Developer Hub \(getdbt.com\)](https://getdbt.com/docs/data-tests/)
- Documentation
 - [Documentation | dbt Developer Hub \(getdbt.com\)](https://getdbt.com/docs/)
- dbt Adapters
 - [Trusted adapters | dbt Developer Hub \(getdbt.com\)](https://getdbt.com/docs/trusted-adapters/)
- Style Guide
 - [How we style our dbt models | dbt Developer Hub \(getdbt.com\)](https://getdbt.com/docs/style-guide/)

Session 4: Vector Storage & Semantic Search

Franzi Kröger

Agenda

Session 1: Data Ingestion & Spark SQL

- Data ingestion concepts & tools (Airbyte, Mage)
- Spark SQL overview & hands-on exercises

Session 2: Single-Node Processing

- Single-node vs. cluster processing
- Tools: Polars, NVIDIA Rapids, DuckDB

Session 3: Processing with dbt

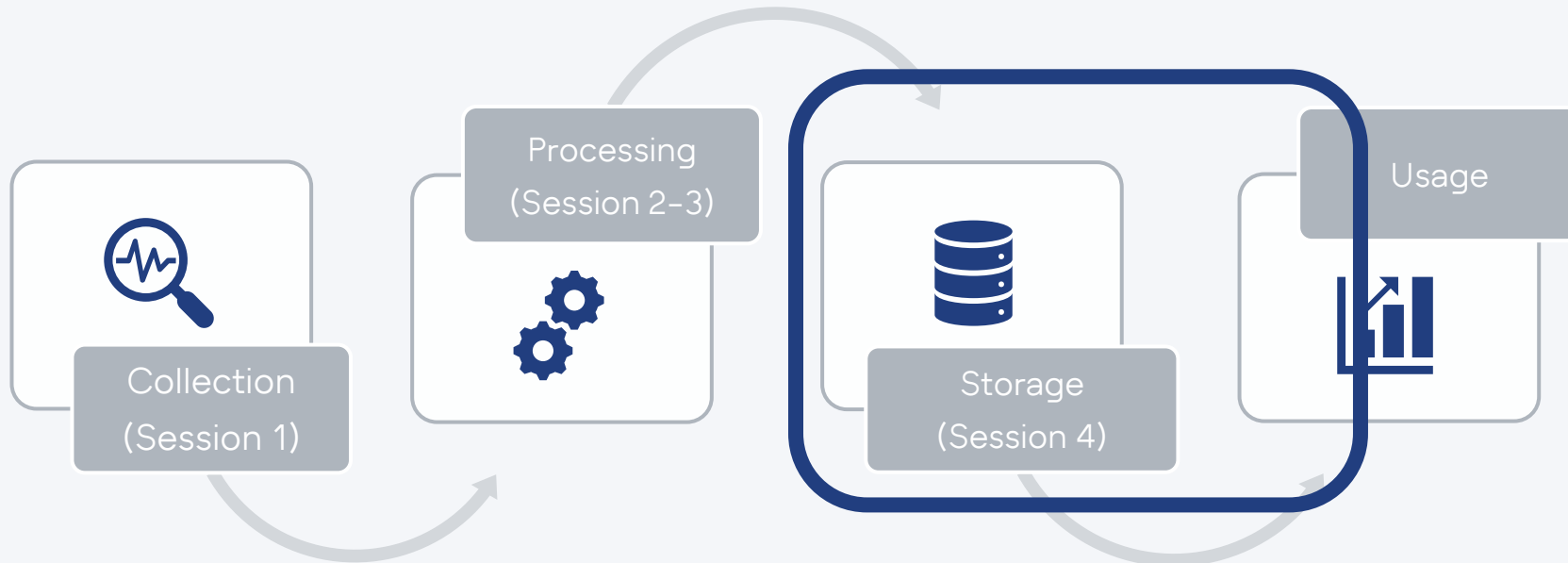
- dbt fundamentals (DuckDB-based)
- End-to-end data transformation exercise

Session 4: Vector Storage & Semantic Search

- Semantic search & vector databases
- End-to-end pipeline with Qdrant

Session 4: Vector Storage & Semantic Search

- Semantic search & vector databases
- End-to-end pipeline with Qdrant



Introduction

Vector stores

Introduction to NOSQL

A NOSQL database refers to any **non-relational database** that stores data in a more flexible, scalable way compared to traditional relational databases. While some say NOSQL stands for “non-SQL”, others interpret it as “not only SQL”.

They **don't rely on predefined schemas** and are designed to handle large volumes of unstructured or semi-structured data. They come with their own query languages, offering a more adaptable approach to data management than the rigid structure of SQL.

The **NOSQL types** available are the following:

- Files
- Key-Value
- Document
- Graph
- In-Memory
- Indexed
- Time Series
- Vector

Name	Dry/Wet Food	Good Boy (Y/N)
Fido	Dry	Y
Rex	Wet	N
Bubbles	Dry	Y
Cujo	Wet	N

Tag #	Height (in)	Weight (lbs)
1573	15	21
2684	9	7
3795	27	130
4806	6	5

Tag #	Name	Breed	Color	Age
1573	Fido	Beagle	Brown/White	1.5
2684	Rex	Pekingese	White	9
3795	Bubbles	Rottweiler	Black	5
4806	Cujo	Chihuahua	Gold	4

NOSQL

Key Value

Documents

Column Store

Introduction to NOSQL: Vector stores

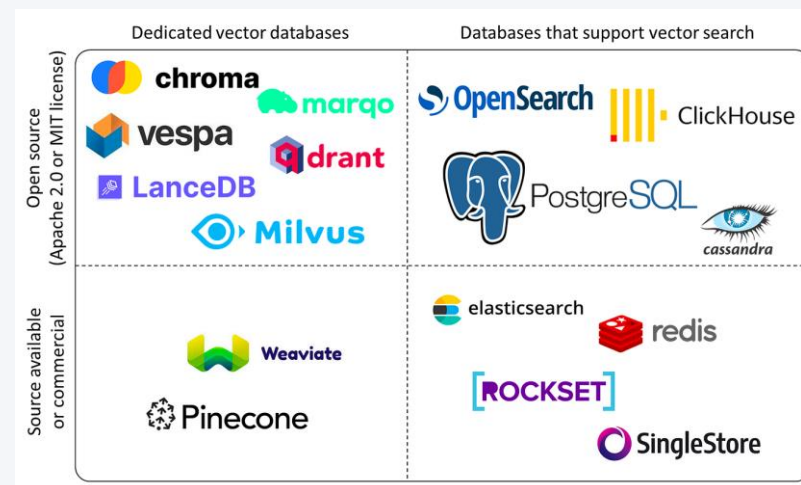
Description

A **Vector Database** is designed to store and manage high-dimensional data as vectors, produced and utilized by AI and Machine Learning models, particularly in **Generative AI**, where it **handles embeddings** — the vectorized representations of images, text, or other complex data.

Key aspects

- Enables fast searches based on vector similarity (semantic search)
- Used in applications like
 - image search and recognition
 - recommendation systems
 - chatbots

Technologies



Vector store use cases

Image and video recognition

Similarity search across visual embeddings to identify objects, faces, or scenes.



Natural language processing and text search

Semantic search and retrieval by comparing text embeddings based on meaning.



Recommendation systems

Find similar users or items to deliver personalized recommendations.



Emerging applications:

Examples: molecular structure analysis (healthcare), anomaly detection (finance fraud).

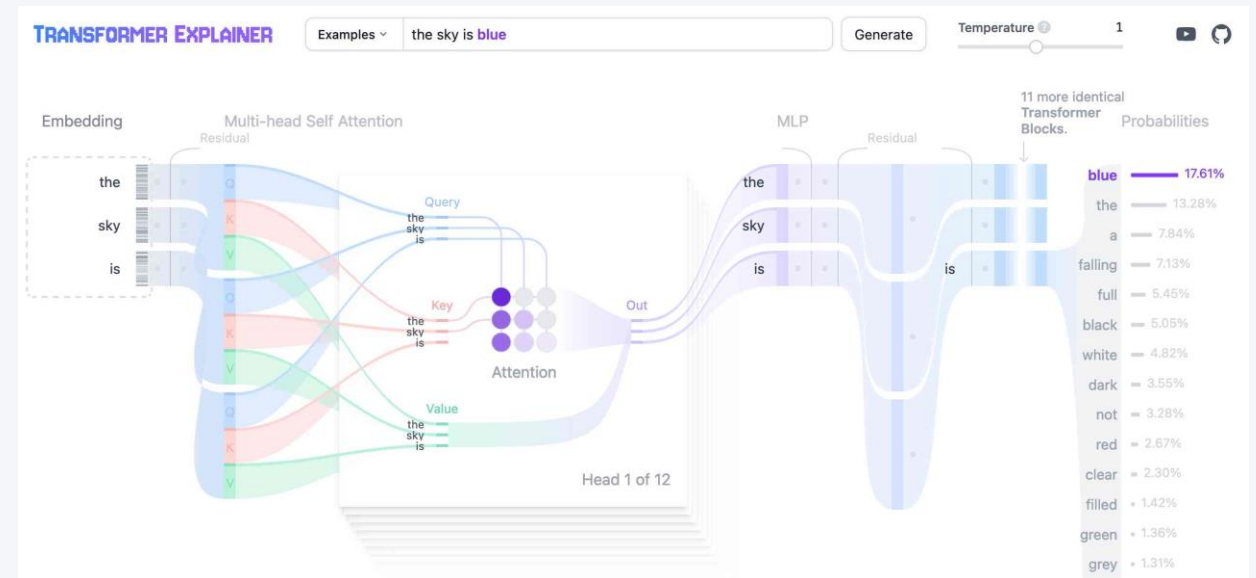
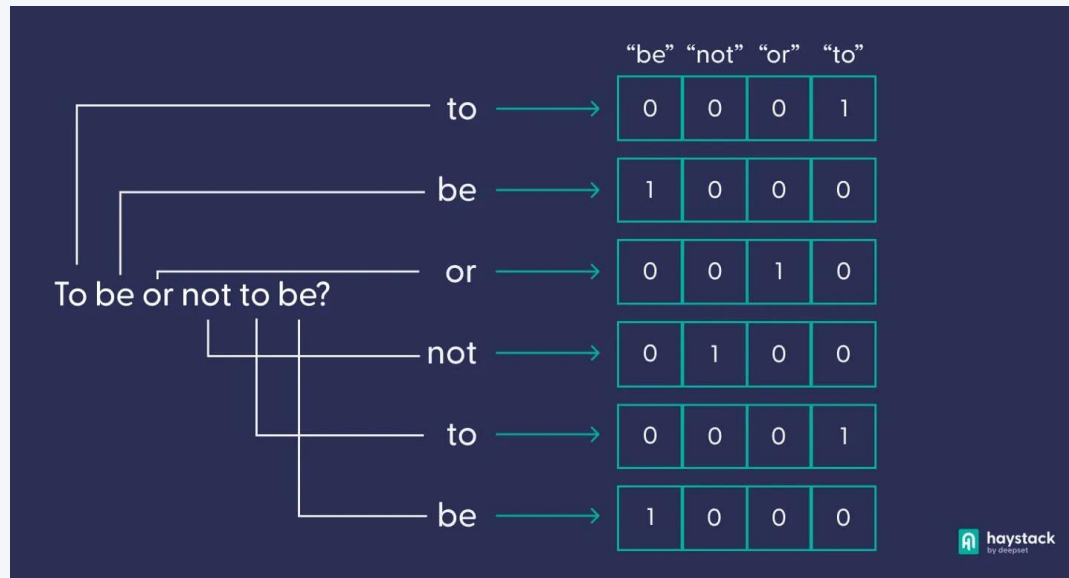


Introduction to embeddings

<https://poloclub.github.io/transformer-explainer/>

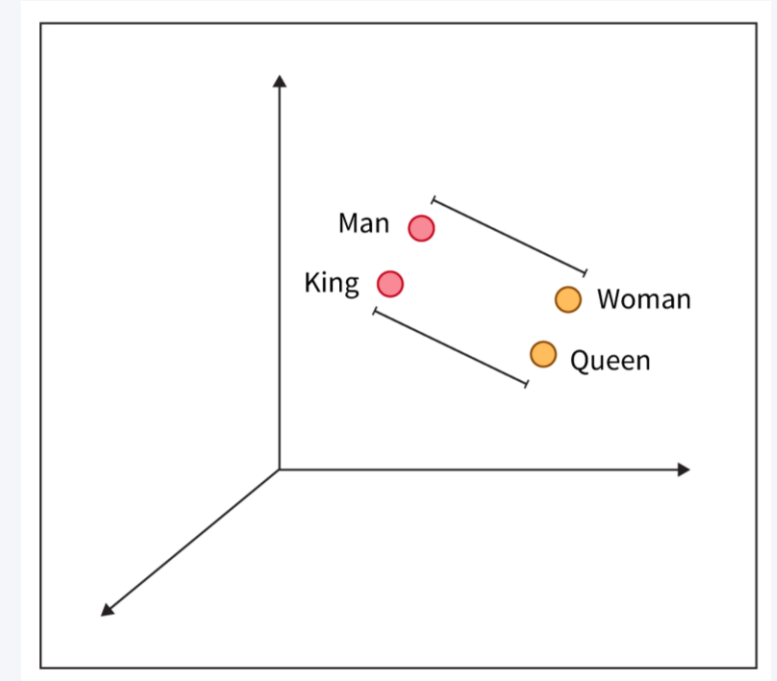
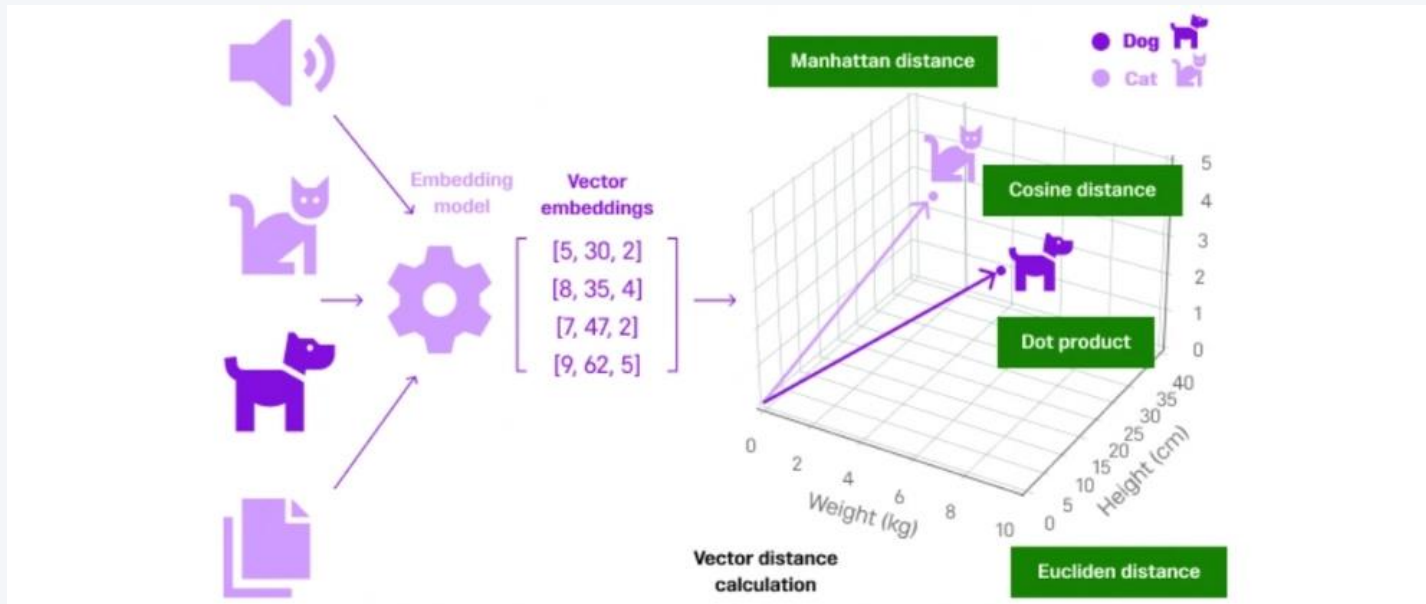
Embeddings are **numerical representations of objects** like text, images and audio that capture their meanings and relationships based on how they appear in a text corpus.

These embeddings are generated by training a ML model on a large amount of data. The model learns to assign each word a unique vector of numbers, which serves as its embedding.



Why vector stores: Semantic Search

Vector stores **enable semantic** search by representing data as embeddings, allowing systems to retrieve information based on meaning and **intent rather than exact keyword matches**.



Semantic (BERT) vs Keyword search (BM25)

Search systems aim to retrieve the most relevant documents based on a user query.

Two major approaches dominate: keyword-based retrieval and semantic retrieval.

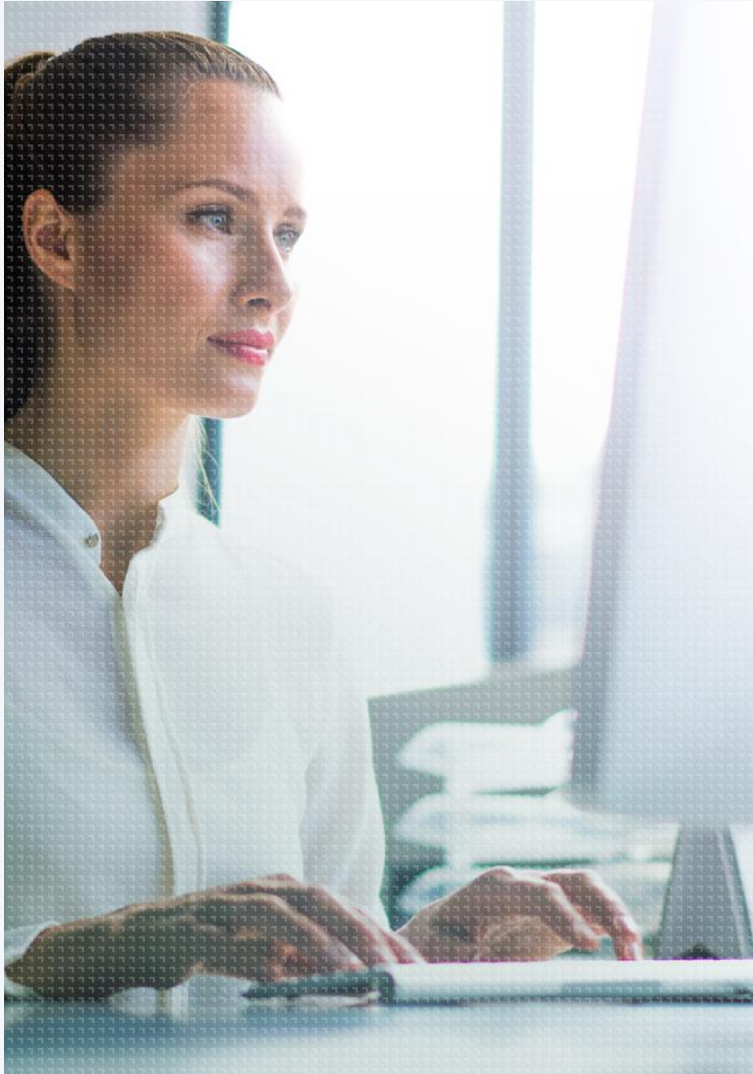
Semantic Search (BERT / Embedding Models)

- Uses deep learning language models to convert text into dense vector embeddings.
- Measures similarity between query and documents in semantic space. **Understands context**, synonyms, paraphrases, and sentence meaning.

Keyword Search (BM25 = Best Matching 25)

- A statistical, **term-frequency-based ranking** algorithm.
- Does not understand meaning, only exact or similar **word matches**.

Try it!! <https://semantic-search.fly.dev/>



Exercise 1 – 2 : Generate embeddings

- You will learn how to generate embeddings
 - Via curl
 - With python
- You will receive an OPENAI_API_KEY from the teacher.
- Open GitHub repository and follow the instructions
<https://github.com/frkroe/state-of-the-art-tools-for-big-data/>
- Let's go!!

Deep Dive & Features



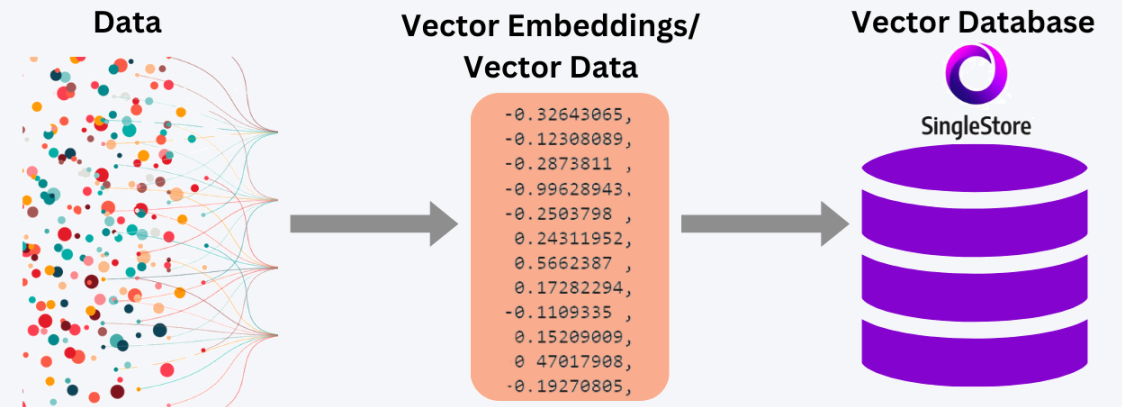
Vector stores

What is a vector store - Features

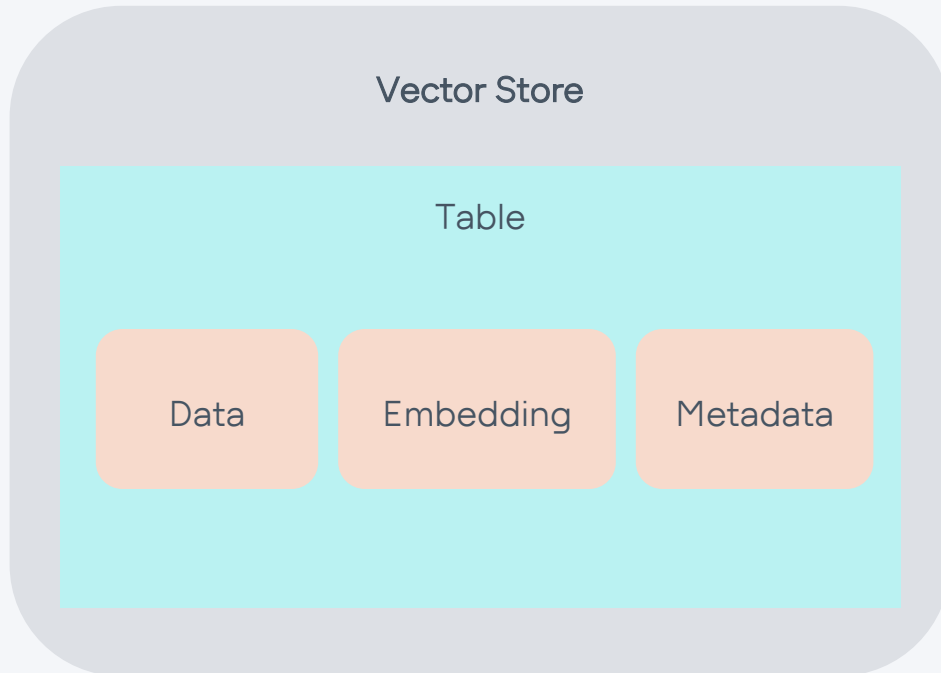
A Vector Database is designed to store and manage high-dimensional data as vectors, produced and utilized by AI and Machine Learning models, particularly in Generative AI, where it handles embeddings — the vectorized representations of images, text, or other complex data.

Features:

- Search capabilities
 - Semantic (must have), keyword, hybrid...
 - Distance types: L2, cosine, Euclidean...
- Metadata handling & filters
- Integrations with leading frameworks (langchain, n8n...)
- License (OS vs commercial) & deployment (cloud, on-prem...)
- Latencies & index creation
- Scalability & performance
- Dashboard UI
- Others: security, access control, monitoring...

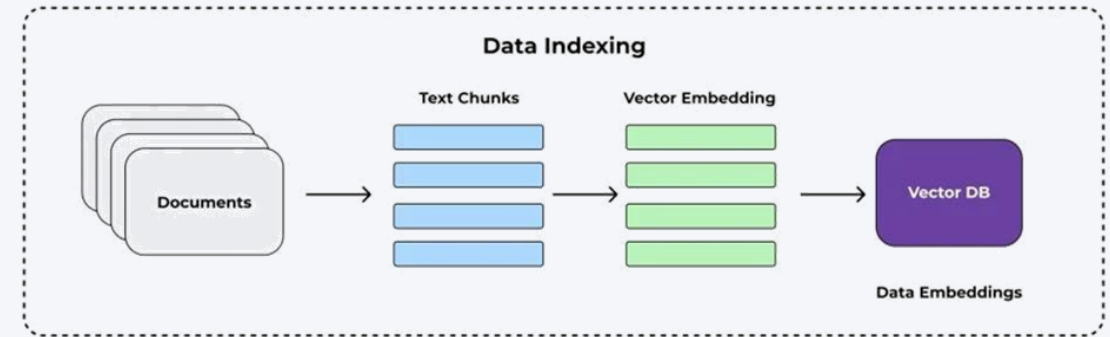


What is a vector store - Schema



- **Data:** Original text (i.e. what we want to search)
- **Embedding:** vector representation of the original text
- **Metadata:** extra information related to the original text

Example



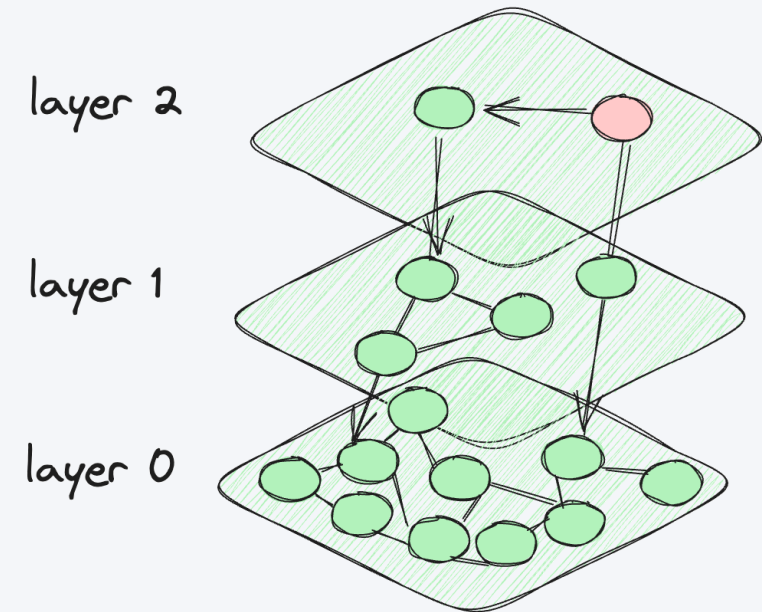
- **Data:** "The cat sat on the mat"
- **Embedding:** [0.823782, 0.11232 0.232372, 0.8234738]
- **Metadata:**
 - Document name: cats.pdf
 - Page number: 3

Vector indexing

A **vector index** is a data structure that organizes embedding vectors for **faster and more efficient similarity searches**. Instead of scanning every vector, the index narrows the search space and jumps directly to the most relevant candidates. This enables much faster queries with lower latency and makes vector search scalable to millions or even billions of items.

Types of Vector Index Structures:

- FLAT / Brute Force Index
- IVF – Inverted File Index
- HNSW – Hierarchical Navigable Small World Graph
- PQ – Product Quantization
- MSTG – Multi-Scale Tree Graph (MyScale)

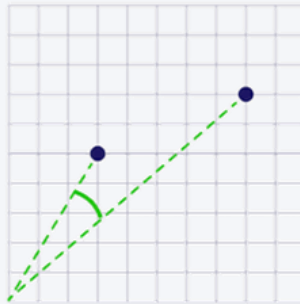


Index Types

- **Flat / Brute-Force:** Scans all vectors for exact nearest-neighbor search.
- **IVF (Inverted File Index):** Clusters vectors and searches only the most relevant clusters.
 - **IVFFLAT:** Uses IVF clustering but stores raw vectors in each cluster for higher accuracy.
 - **IVFPQ:** Combines IVF with product quantization to compress vectors and speed up search.
 - **IVFSQ:** Uses IVF with scalar quantization to reduce storage by quantizing each dimension.
- **HNSW:** Builds a multi-layer navigable graph for extremely fast approximate nearest-neighbor search.
 - **HNSWFLAT:** HNSW graph storing raw vectors for maximum recall.
 - **HNSWSQ:** HNSW graph with scalar-quantized vectors to reduce memory usage.
- **PQ (Product Quantization):** Compresses vectors into compact codes for efficient approximate search.
- **MSTG:** A multi-scale tree-graph hybrid that reduces memory and accelerates large-scale vector search.

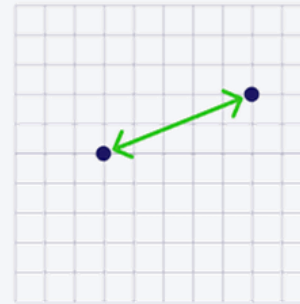
Similarity search & distances

Vector search uses distance metrics like **Euclidean (L2)**, **Cosine Similarity/Distance**, and **Dot Product** for normalized vectors, plus **Manhattan (L1)** and **Hamming** (for binary) to measure how "close" data points are in a high-dimensional space, helping find semantically similar items. The choice depends on whether magnitude or direction matters and if vectors are normalized.



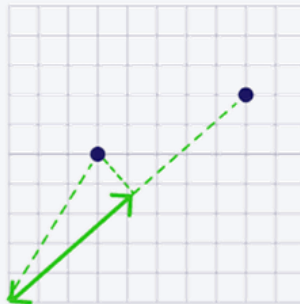
Cosine Distance

$$1 - \frac{A \cdot B}{\|A\| \|B\|}$$



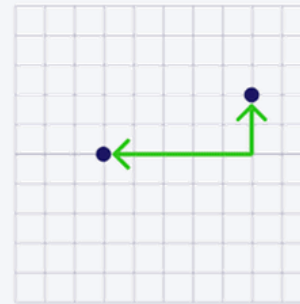
**Squared Euclidean
(L2 Squared)**

$$\sum_{i=1}^n (x_i - y_i)^2$$



Dot Product

$$A \cdot B = \sum_{i=1}^n A_i B_i$$



Manhattan (L1)

$$\sum_{i=1}^n |x_i - y_i|$$

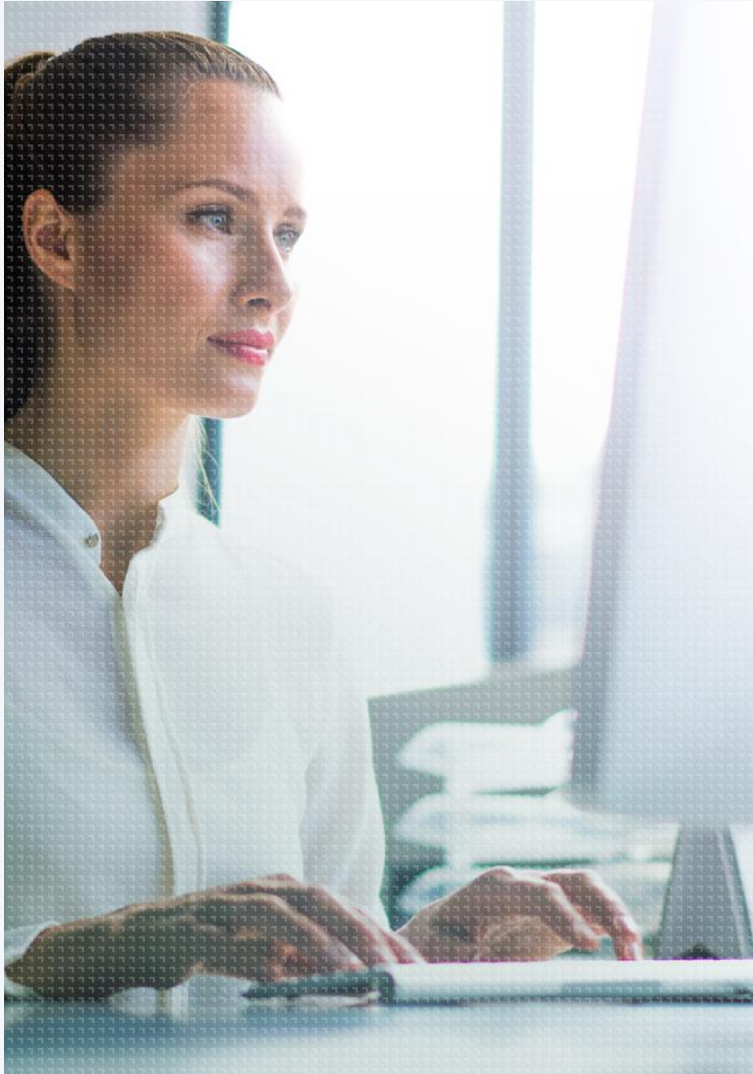
Similarity search & distances (detail)

Common Distance Metrics

- **Euclidean Distance (L2 Norm):** The straight-line distance between two points (vectors).
 - Use: General purpose, good for low-dimensional data but can struggle with high dimensions.
- **Cosine Similarity:** Measures the angle between vectors, focusing on orientation, not magnitude.
 - Use: Text/document similarity where vector lengths (document size) vary; often best for unnormalized vectors.
- **Dot Product:** Calculates the sum of products of corresponding components.
 - Use: Highly efficient for normalized (unit-length) vectors; becomes a measure of similarity.
- **Manhattan Distance (L1 Norm):** Sum of absolute differences between coordinates (like city blocks).
 - Use: Alternative to Euclidean, less sensitive to outliers.

How to Choose

- For Normalized Vectors (Unit Length): Dot Product (fastest), or Cosine Distance.
- For Unnormalized Vectors: Cosine Similarity (ignores magnitude), or Euclidean Distance (if magnitude matters).
- For General Purpose/Uncertainty: Euclidean Distance (L2) or Cosine Similarity.



Exercise 3-5: Create, insert and query embeddings from a database

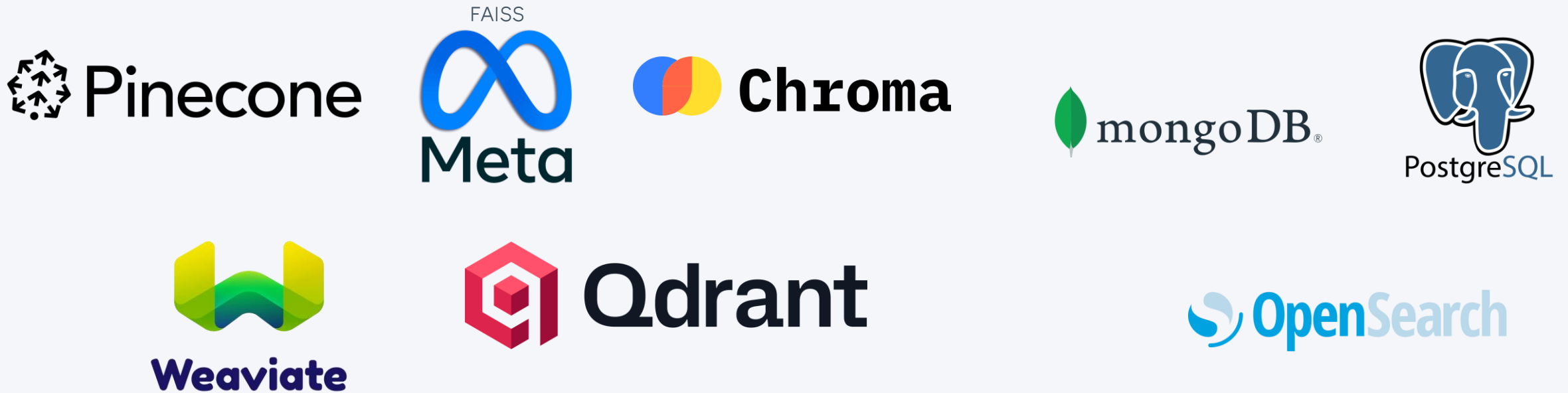
- Let's load embeddings into a PostgreSQL database and run some queries with different distance metrics.
- Open GitHub repository and follow the instructions
<https://github.com/frkroe/state-of-the-art-tools-for-big-data/>
- Let's go!!

Main frameworks and vendors

Vector stores

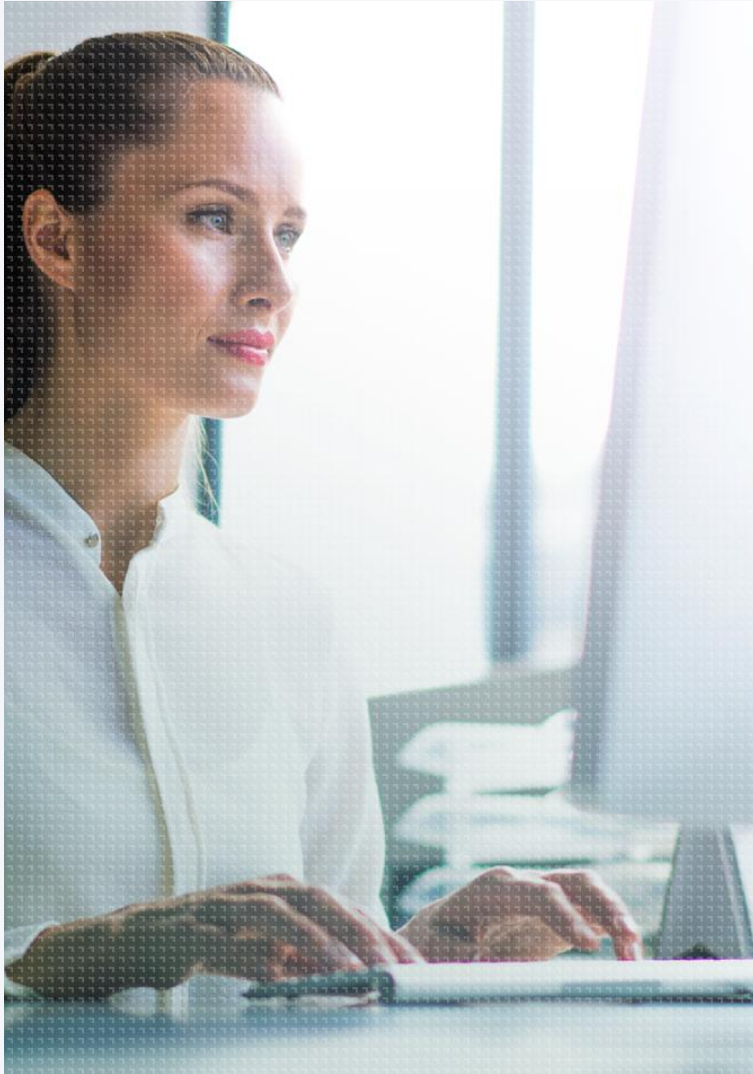
Vector Database Vendors

Nowadays there are **many vendors** which have a vector store/database, and even existing vendors provide such capabilities. **Choosing the right one** is key to the success of the project and decision depends on several factors.



Vector Databases - Comparison

Vector Store	Search Capabilities (Semantic / Keyword / Hybrid)	Distance Types (L2, Cosine...)	Metadata & Filters	Framework Integrations (LangChain, n8n...)	License & Deployment	Latency & Indexing	Scalability & Performance	Dashboard UI
Pinecone	✓ Semantic ✓ Hybrid	✓ Cosine ✓ Dot ✓ L2	✓ Rich filters	✓ Excellent	Commercial Cloud	✓ Low latency, managed	✓ High (managed)	✓ Yes
Chroma	✓ Semantic	✓ Cosine ✓ L2	⦿ Basic	✓ Good	Open Source Self-managed	⦿ OK for small-medium	⦿ Limited	✗ No
Qdrant (<i>"quadrant"</i>)	✓ Semantic ⦿ Hybrid	✓ Cosine ✓ Dot ✓ L2	✓ Strong	✓ Very good	Open Source (self-managed) Commercial (cloud managed)	✓ Fast	✓ High	✓ Yes
Weaviate	✓ Semantic ✓ Hybrid	✓ Cosine ✓ Dot ✓ L2	✓ Strong	✓ Excellent	Open Source (self-managed) Commercial (cloud managed)	✓ Fast	✓ High	✓ Yes
FAISS	✓ Semantic only	✓ L2 ✓ Cosine	✗	⦿ Limited	Open Source Self-managed	✓ Very fast (in-memory)	⦿ Depends on setup	✗ No
PostgreSQL (pgvector)	✓ Semantic ⦿ Hybrid	✓ Cosine ✓ L2	✓ SQL-level	✓ Growing	Open Source (self-managed) Commercial (cloud managed)	⦿ Medium	⦿ Medium	⦿ Third-party
OpenSearch	✓ Semantic ✓ Keyword ✓ Hybrid	✓ Cosine ✓ L2	✓ Strong	✓ Good	Open Source (self-managed) Commercial (cloud managed)	⦿ Medium	✓ High	✓ Yes
MongoDB (Atlas Vector Search)	✓ Semantic ⦿ Hybrid	✓ Cosine ✓ Dot	✓ Strong	✓ Good	Commercial Cloud	⦿ Medium	✓ High	✓ Yes



Exercise 7: Store embeddings in Qdrant

- Let's repeat the same workflow but with a different database, Qdrant!
- We will use our tickets dataset.
- Open GitHub repository and follow the instructions
<https://github.com/frkroe/state-of-the-art-tools-for-big-data/>
- Let's go!!

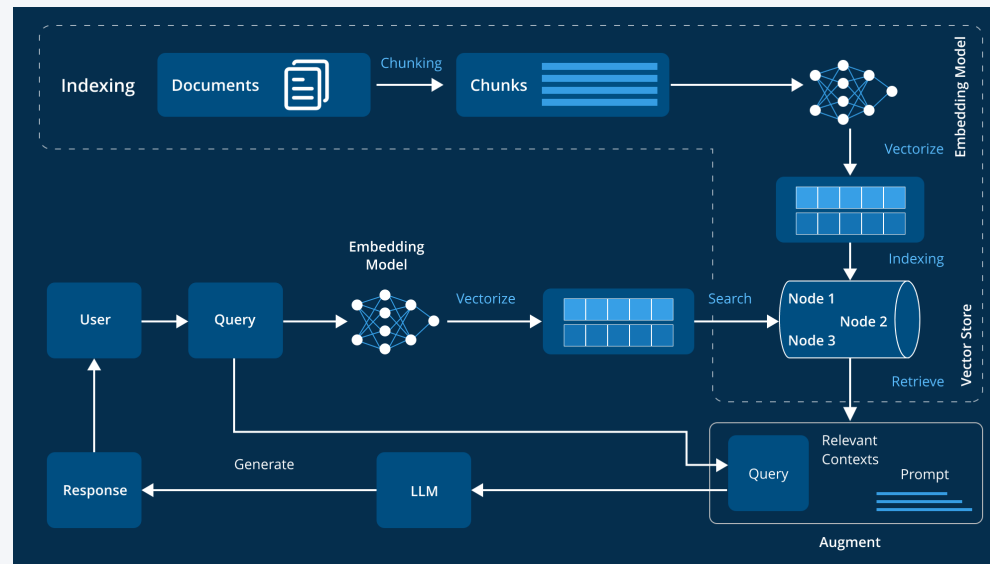
Vector search applied (RAG)

Vector stores

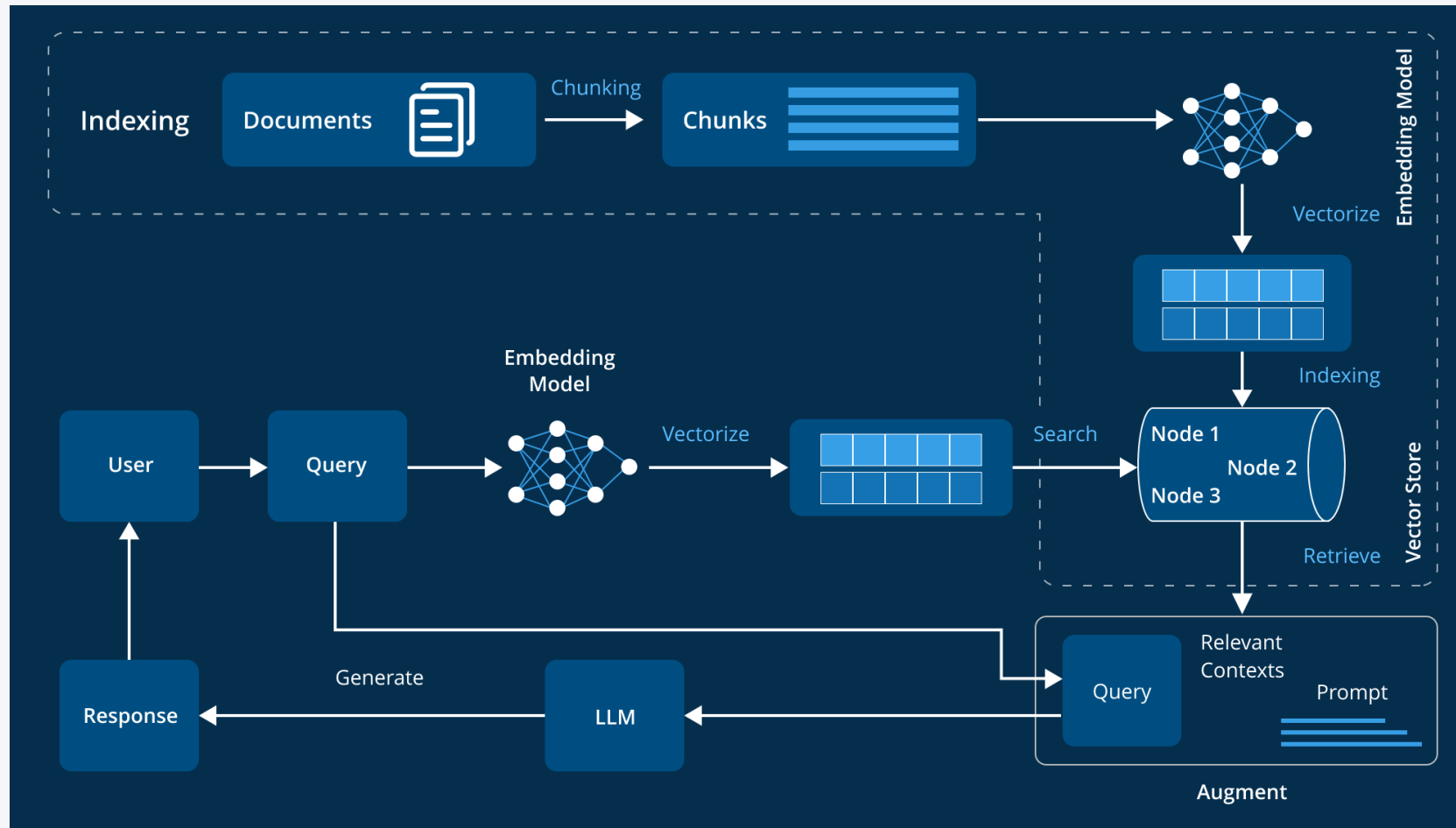
Introduction to Retrieval Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is an architecture that enhances large language models by **retrieving relevant information from a vector store** before generating a response. In RAG, documents are converted into embeddings and stored in a vector database.

When a user asks a question, the query is also embedded and compared against the vector store using semantic similarity. The most relevant results are then provided as context to the language model, enabling it to **generate more accurate, grounded, and up-to-date answers** without retraining.

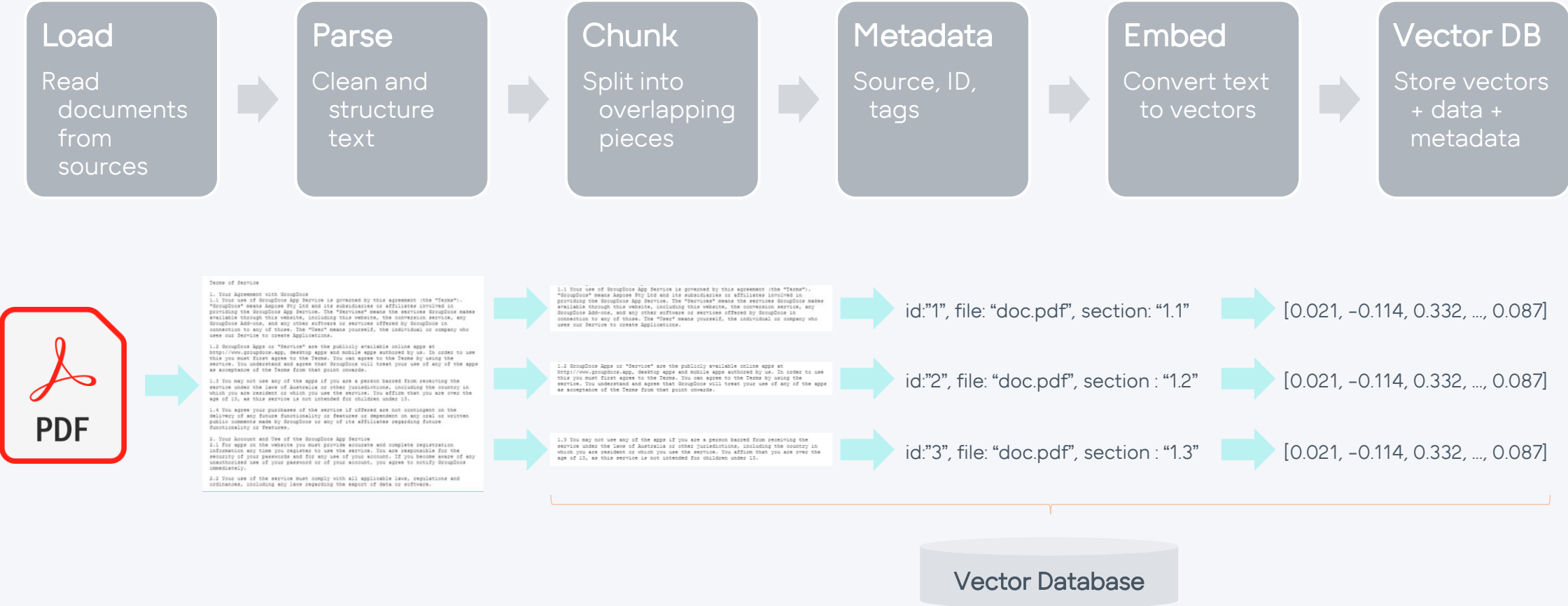


Introduction to Retrieval Augmented Generation (RAG)



RAG Ingestion / Pre-processing

RAG ingestion transforms **raw data into searchable embeddings** stored in a vector database.



RAG Retrieval

Retrieval is the step where **relevant information is selected** from a vector store and provided to the LLM as context before generation. But there are different ways to do so.

Semantic Retrieval (Vector Search)

- Uses embeddings to search by meaning
- Finds relevant content even without exact keywords
- Most common approach in RAG

Keyword Retrieval

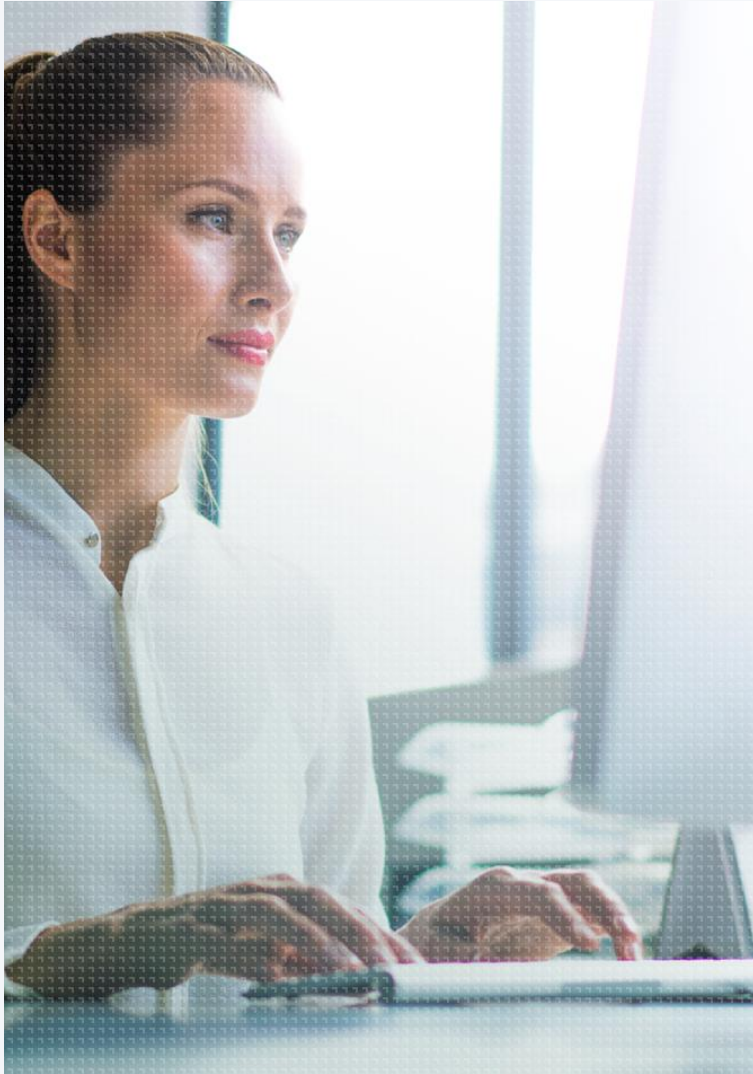
- Matches exact terms or phrases
- High precision for specific words, IDs, or codes
- Often used with traditional search engines

Hybrid Retrieval

- Combines semantic + keyword search. Requires reranking.
- Balances recall and precision
- Common in enterprise RAG systems

Knowledge Graph

- Retrieves information from a graph of entities and relationships
- Preserves structure, hierarchy, and explicit reasoning paths
- Complements vector search for factual and relational queries



Exercise 8-9: LLM & Conversational Agents

- Let's test the OpenAI's GPT large language model.
- We want to build a chatbot in n8n, a workflow automation platform.
- Open GitHub repository and follow the instructions
<https://github.com/frkroe/state-of-the-art-tools-for-big-data/>
- Let's go!!

Success stories at GFT

Vector stores

Automatic ticket resolution

CHALLENGE

- A large supermarket chain's staff identified inefficiencies in the ticket resolution process, highlighting the need for a more streamlined, intelligent approach that would reduce response times and give more consistent solutions.
- To achieve this optimization, the client needed to improve and automate both the ticket assignment and ticket resolution processes.

ENGAGEMENT

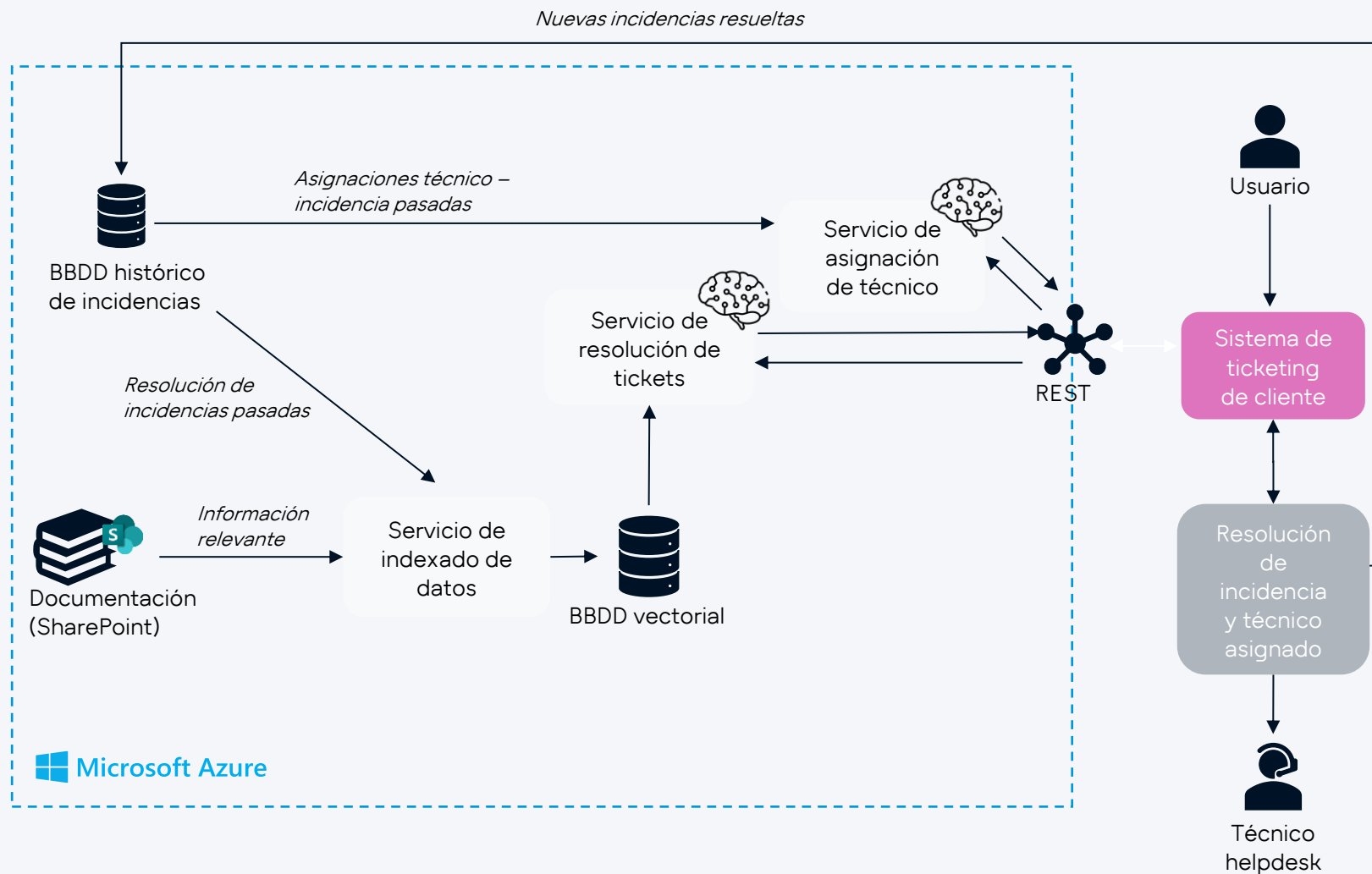
- GFT implemented AI-driven incident resolution services that enable the client to accelerate ticket resolution and assignment processes. Leveraging generative AI and a solid data model, the solution provides natural, human-like responses to tickets arriving to the client's ticket resolution platform. Additionally, ML algorithms automatically assign tickets to the appropriate technical support teams, ensuring accurate handling of issues.

BENEFITS

- **Faster incident resolution:** ML-powered classification and automated assignment significantly reduce response and resolution times, minimizing operational disruptions.
- **Increased consistency and quality:** standardising and automating the ticket resolution process through a single AI-powered based on past tickets and handbooks ensures that all team members can produce homogeneous, high-quality solutions.
- **Robust data foundation:** implementing a dedicated data model to store historical and newly resolved tickets ensures that the system learns from past cases and leverages the full dataset, improving both the resolution and ticket assignment processes.



Automatic ticket resolution



Incident Triage and Resolution

Performance Index and Rates for Spanish Bank

REAL IMPACT OF AI-DRIVEN INCIDENT RESOLUTION: KPIs and Cultural Transformation

It accelerates acknowledgment; drastically shortens assignment, prioritization, and analysis time;

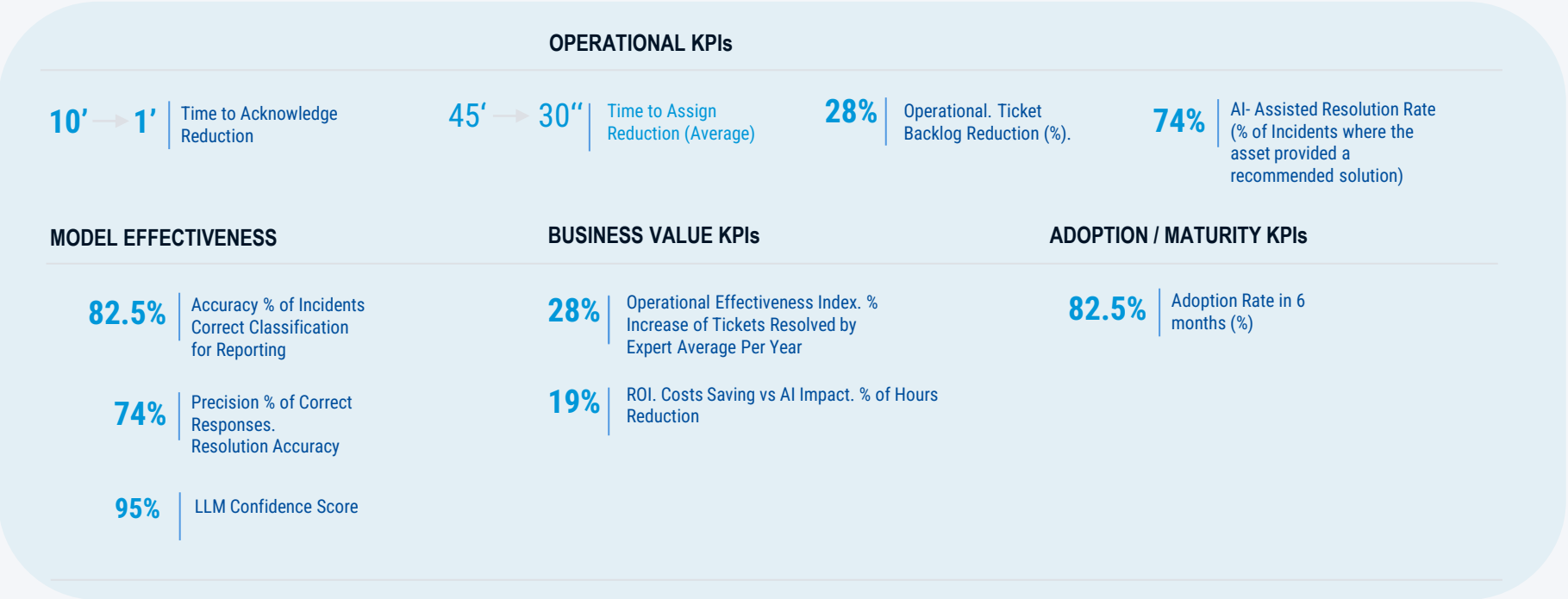
Improves overall resolution time by recommending solutions based on historical incidents.

It delivers a 95% accuracy rate in identifying relevant solutions, with 74% precision in responses.

The implementation is not just technical—it drives a cultural and operational shift.

As a result, we're seeing strong adoption, with a projected 100% adoption rate within the first year.

AI-driven incident resolution recommendations reduce the effective 'Time to Resolve' by pre-identifying the root cause and suggesting validated solutions based on historical incident data. As a result, the resolution phase is limited primarily to approval, deployment, and validation steps—accelerating recovery and minimizing system downtime.



Observations from the SmartOps team:

- ❖ Noticed improvements especially in incidents involving error codes. With the structure "Error code + brief explanation," the system efficiently identifies how to handle them.
- ❖ Using Incident Triage mostly for abends and I find it very helpful in most cases. It allows me to identify other incidents with the same error string or similar abends, which may be resolved in the same way.

SUCCESS CASE

Employee Contact Center

Automating employee support (HR department) with a **Generative AI chatbot** to handle employee inquiries, reducing call center costs and improving response accuracy through real-time integration with HR policies.

Challenge

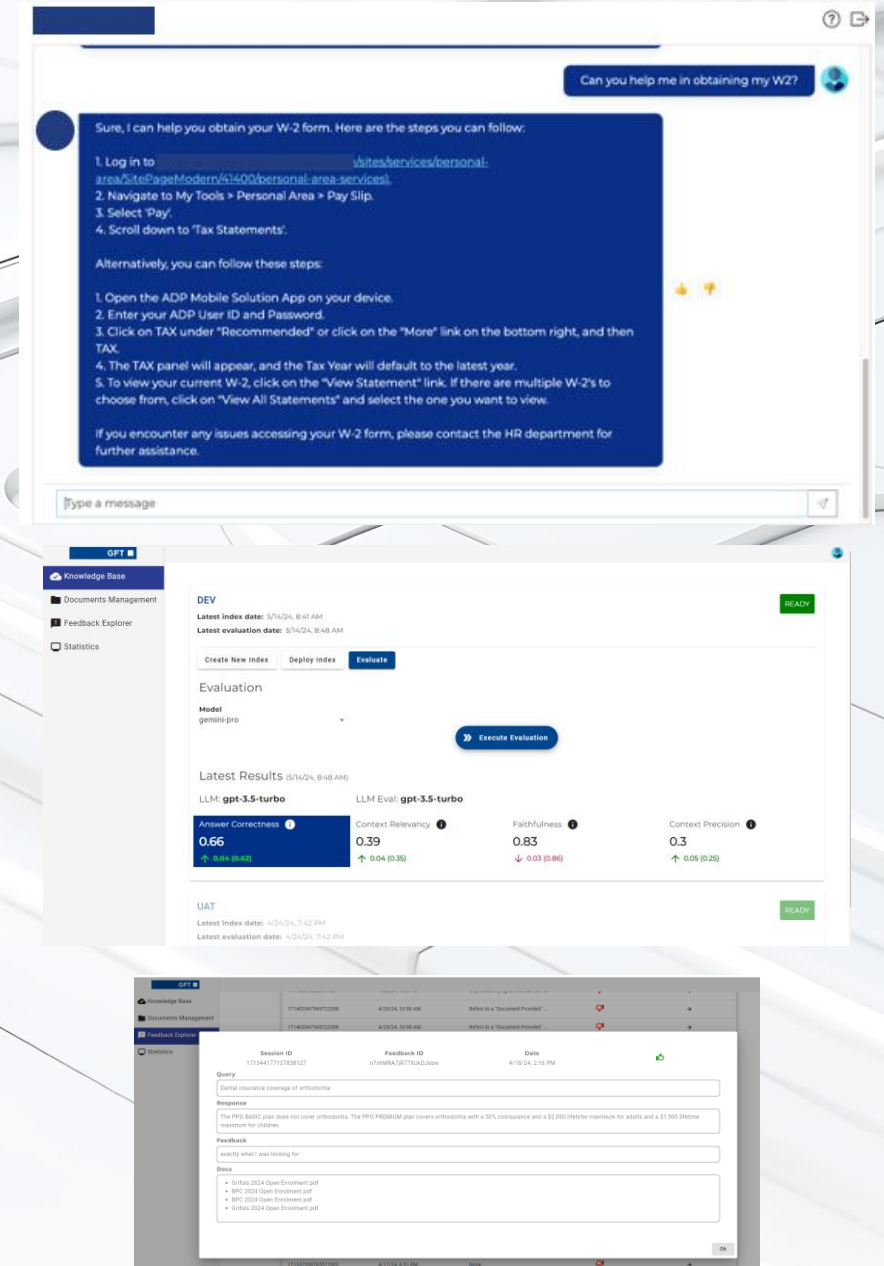
- A leading pharmaceutical company faced **significant costs** due to the **extensive use of a contact center** for handling **HR inquiries** from over 10,000 employees. The manual process of providing general HR answers was inefficient and costly.

Engagement

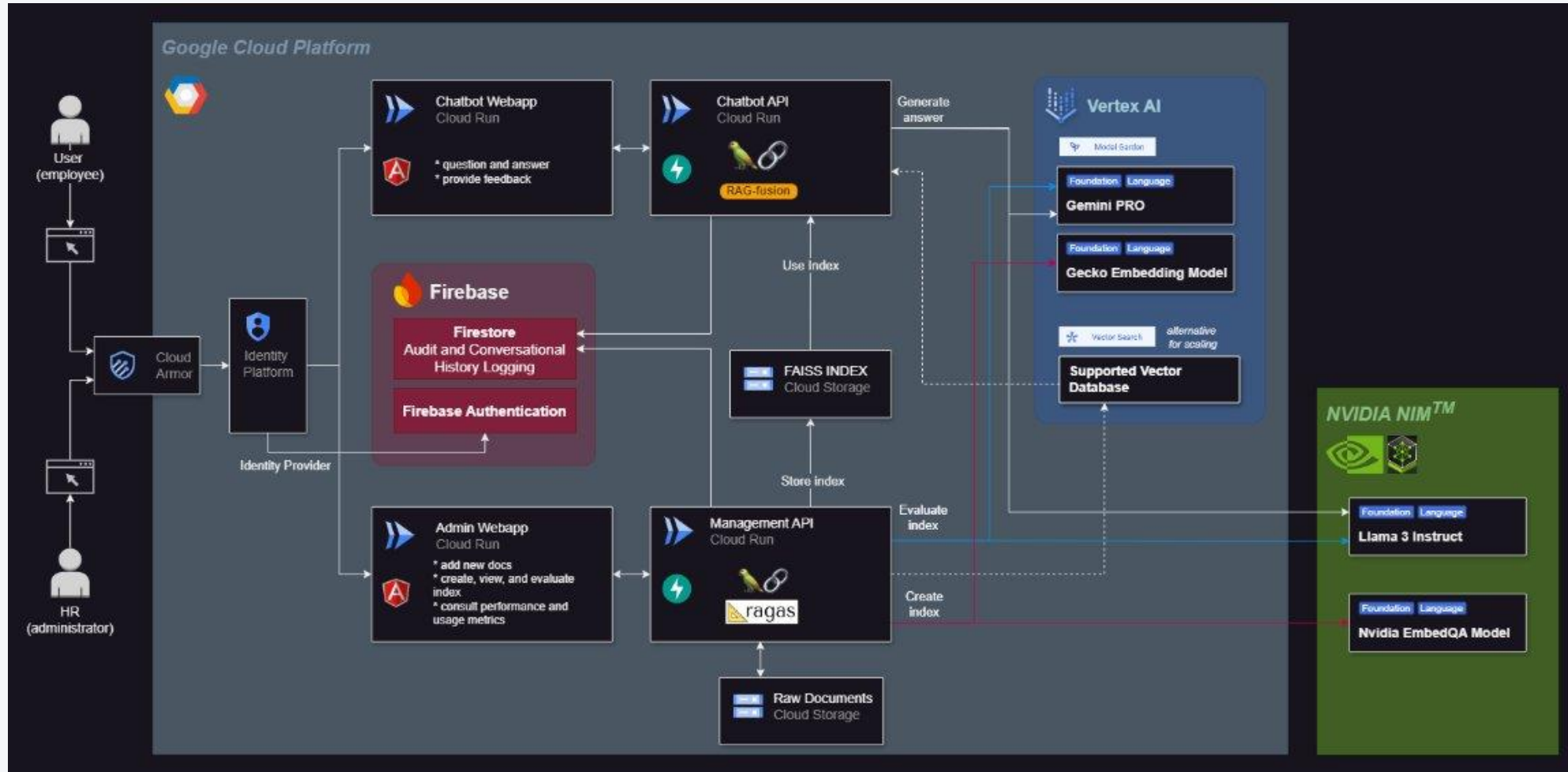
- GFT developed a **retrieval-augmented generation (RAG)** chatbot designed to automate the response process for general HR inquiries.
- The system was **integrated with the company's HR policy documents** to ensure accurate and up-to-date information.
- Additionally, an **administration console** was created for document updates and overall tool administration.

Benefits

- Significant reduction in operational costs** by minimizing manual response requirements.
- The chatbot's **instant response capability** markedly improved the timeliness of HR support.
- By automating responses and enabling easy administration of HR policies, the system enhanced overall HR support efficiency.



Employee Contact Center



Credit Risk Assistant

CHALLENGE

CHALLENGE

- Creating credit memos in banking presents key challenges, including ensuring data accuracy, regulatory compliance, thorough risk assessments, and interdepartmental coordination.
- Efficiency requires effective time management, seamless technology integration, and strict quality control.

ASSET

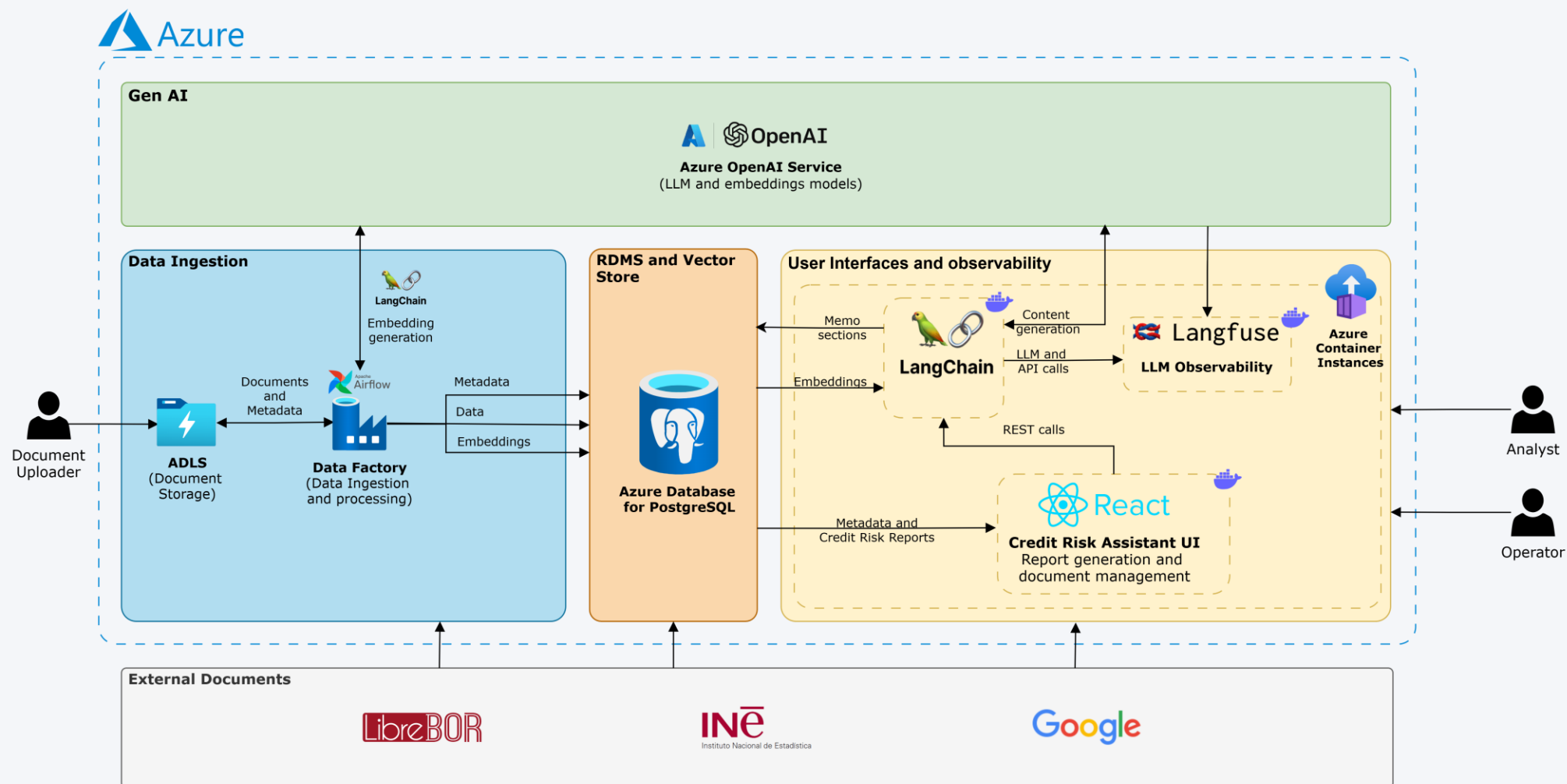
- GFT collaborated with Oliver Wyman to develop an advanced system that automates the integration of multiple data sources and generates the summaries that analysts need to produce these reports efficiently.

BENEFITS

- **Significant time savings:** Automation of key credit memo tasks saves over thousands of hours annually, enabling analysts to focus on strategic activities rather than manual processes.
- **Cost efficiency:** The reduction in manual effort leads to substantial savings, optimising resource allocation.
- **Enhanced quality and consistency:** Automated processes ensure improved report accuracy and uniformity, eliminating discrepancies caused by human error.



Credit Risk Assistant



Outro

GFT Junior Program & Homework

GFT Data Junior Program 2026

¿Quieres unite a nuestro #teamgft?

Empleabilidad

- Contrato indefinido desde el primer día
- Modalidad híbrida o teletrabajo según proyecto
- Horario flexible y paquete de beneficios competitivo

Formación

- Plan formativo intensivo en tecnologías Data
- Acceso a cursos y certificaciones

Acompañamiento

- Mentoría y seguimiento continuo por parte de formadores técnicos y RRHH

Ediciones

- Primavera 2026 (aprox. marzo)
- Otoño 2026 (aprox. septiembre)



Contact Information

GFT Technologies
Franzi Kröger
Data Engineer
Franzi.Kroeger@gft.com

GFT Technologies
Carlos Ferrer
Data Engineer
Carlos.Ferrer@gft.com



Save resources: this presentation is optimized for screen use. Please avoid printing.

© 2025 | GFT Technologies SE and its affiliates. All rights reserved.

